Fundamentos para el Análisis Automático de Lenguaje

Dra. Delia Irazú Hernández Farías
Dr. Luis Villaseñor Pineda
Dr. Manuel Montes y Gómez
Dr. Aurelio López López

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Lenguaje **2**

2. Continuación (Lenguaje) **4**

3. Recuperación de información (IR) **5**

4. Continuación (Vector Space Model) **10**

5. Modelos de lenguaje para IR **13**

6. Medidas de evaluación en IR **17**

7. Expansión de consultas y clustering **21**

8. Representaciones más allá de las palabras **24**

9. Indexado más allá de la palabra **27**

10. Desambiguación e indexado conceptual **31**

11. Representaciones conceptuales **39**

12. Descomposición en valores singulares **43**

13. Semántica distribucional y *word embeddings* **47**

14. Word2Vec, FastText y GloVe **56**

15. De las redes recurrentes al Transformer **68**

16. El Transformer a detalle **80**

---

1. Lenguaje

17 de agosto de 2026

Resumen rápido. En esta sesión vimos una intro de qué es el lenguaje, la lingüística, desde sus bases y cómo se ha estudiado a lo largo de la historia, llegando a la lingüística computacional y el procesamiento de lenguaje natural.

Qué es un lenguaje natural?

Un lenguaje natural es un sistema de comunicación que permite a los seres humanos expresar ideas, emociones y conceptos a través de signos y símbolos.

Idea Random

Un sistema de comunicación entre dos animales puede ser conciderado lenguaje? O entre seres interespecíficos? Por ejemplo, entre un humano y un perro.

Un lenguaje puede ser influenciado por:

- **Geografía**: Por ejemplo torta es pues una torta en México, pero en Argentina es un pastel.
- **Cultura**: Por ejemplo, en México se dice "aguacate", pero palta en sudamerica
- **Historia**
- **Educación**: "Qué pató mamá kuakuak" es una expresión de barrio
- **Cosas que se descubren**

El procesamiento del lenguaje es cómo el cerebro procesa el lenguaje y cómo entiende el mensaje que se quiere transmitir. Esta habilidad se obtiene a través de la experiencia y la exposición al lenguaje desde una edad temprana y, el cerebro comienza a desarrollar la capacidad de comprender y producir lenguaje a medida que se expone a él.

En ciencias de la computación, el procesamiento del lenguaje natural (PLN) es un campo que se enfoca en desarrollar algoritmos y modelos computacionales que permitan a las máquinas comprender, interpretar y generar lenguaje humano de manera efectiva. El PLN combina técnicas de lingüística, aprendizaje automático e inteligencia artificial para abordar tareas como la traducción automática, el análisis de sentimientos, la generación de texto y la comprensión del lenguaje natural.

Conceptos importantes

- **Lingüística**: Estudio científico del lenguaje, incluyendo su estructura, evolución y uso en la sociedad.
- **Lingüística computacional**: Rama de la lingüística que se enfoca en el desarrollo de modelos y algoritmos para procesar y analizar el lenguaje natural mediante computadoras.
- **Procesamiento de lenguaje natural (PLN)**: Campo de estudio que combina técnicas de lingüística, aprendizaje automático e inteligencia artificial para permitir que las máquinas comprendan, interpreten y generen lenguaje humano.

Otras disciplinas que también se relacionan con el estudio del lenguaje son la psicología y las neurociencias.

Para entender el PNL hay varias areas como:

- **Morfología**: Estudio de la estructura de las palabras y cómo se forman a partir de morfemas
- **Léxica**: Estudio del vocabulario de un idioma, incluyendo el significado y uso de las palabras
- **Pragmática**: Estudio del uso del lenguaje en contextos específicos y cómo el significado puede variar según la situación y la intención del hablante.
- **Semántica**: Estudio del significado de las palabras, frases y oraciones, así como las relaciones entre ellos
- **Sintaxis**: Estudio de la estructura de las oraciones y cómo se combinan las palabras para formar oraciones gramaticalmente correctas
- **Fonética**: Estudio de los sonidos del habla, incluyendo su producción, transmisión y percepción
- **Fonología**: Estudio de los sistemas de sonidos en un idioma y cómo se organizan para formar palabras y oraciones

2. Continuación (Lenguaje)

19 de agosto de 2026

Resumen rápido. En esta sesión continuamos con la introduccion al curso, como ha evolucionado el NLP que es el NLP multinodal y los modelos de lenguaje, todos estos conceptos asi muy por encima, nada super formal, solo para tener una idea de lo que es el curso y a donde vamos a llegar. Igual vimos las areas o problemas de investigacion en PLN.

- **1950**: Alan Turing propone el test de Turing.
- **1957**: Chomsky publica su gramática generativa.
- **1966**: ELIZA, el primer chatbot basado en reglas.
- **1980s**: Auge de los métodos estadísticos sobre corpus.
- **1990s**: Modelos ocultos de Márkov y traducción automática estadística.
- **2013**: word2vec populariza los embeddings de palabras.
- **2017**: Llega la arquitectura Transformer.
- **2018**: BERT y GPT inician los modelos preentrenados a gran escala.
- **2020s**: Modelos de lenguaje grandes de uso masivo.

Al inicio se utilizaban solo textos y reglas para los algoritmos de NLP pero con el avance de la tecnología surgió el **NLP multinodal** (cuando usa solo texto se llama mono nodal), que a demas de texto, también utiliza imágenes, audio y video para el procesamiento del lenguaje natural.

En nuestra vida diaria el NLP se usa en muchas cosas como:

- Asistentes virtuales (Siri, Alexa, Google Assistant)
- Traducción automática (Google Translate, DeepL)
- Análisis de sentimientos en redes sociales
- Chatbots y atención al cliente automatizada (aunque a veces es deficiente)
- Resumen automático de textos
- Reconocimiento de voz y transcripción

Modelos de Lenguaje

Idea clave

Un modelo de lenguaje es un modelo probabilístico/estadístico que asigna una probabilidad a una secuencia de palabras (o tokens)

Como curiosidad, la formula que usualmente se utiliza para predecir la siguiente dado un contexto anterior es

$$P(w_n|w_1, w_2, ..., w_{n-1}) = \frac{P(w_1, w_2, ..., w_n)}{P(w_1, w_2, ..., w_{n-1})} \tag{1}$$

Y pues se parece mucho a la formula de Bayes o a la de probabilidad condicional. Hay dos grandes famililias de modelos de lenguaje

- **Basados en n-gramas**: Estiman esa probabilidad contando frecuencias de secuencias de n palabras en un corpus. Simples pero limitados por la escasez de datos (data sparsity) para n grande.
- **Basados en ANNs**: Aprenden una representación continua del contexto y generalizan mejor a secuencias no vistas. Los LLM actuales (GPT, BERT, etc.) son modelos de lenguaje neuronales entrenados sobre enormes corpus.

Areas de investigación en PLN

Hay varias áreas de investigación y/o problemas en PLN:

- Detección de fake news
- Perfilado de autoría
- Detección de discurso de odio (el discurso de odio no siempre es grosero o así, pero es discurso de odio sin ser explícito)
- Detección de humor
- Análisis de sentimiento
- Question answering
- RAG
- Resumen de textos
- Generación de texto
- Text to image
- Image to text (captioning)

3. Recuperación de información (IR) *24 de agosto de 2026*

Resumen rápido. En esta sesión comenzamos con un concepto, que al menos para mi es nuevo, el de la recuperacion de informacion. Vimos que es, para que sirve y un par de ejemplos. El contenido de esta seccion de IR es el siguiente

- Definicion de la tarea
- The Vector Space Model
- Modelos de lenguaje para IR
- Evaluacion de IR (Performance)
- Problemas principales y soluciones basicas (Indexing, Query Expansion, Relevance Feedback, Clustering, Diversification)

Introduccion

Para entender de manera intuitiva que es un Sistema de Recuperación de Informacion (IRS), responderemos las preguntas que el Dr planteo en clase...

- **Qué es un sistema de recuperación de información (IR)?** Es un sistema que busca dentro de una colección grande de documentos no estructurados o semiestructurados
- **Que objetivo tiene?** Su objetivo esrecuperar documentos reelevantes para satisfacer una neceidad de informacion del usuario
- **Que hay dentro de el? Algun sub-proceso?** Pues si, un SRI se descompone tipicamente en sus fases de; Adqisicion, Preprocesamiento del texto, Indexacion, Procesamiento de la query, Recuperacion, Ranking Presentacion de resultados, Retroalimentacion.
- **Como evaular su desempeno?** Se busca maximizar dos cosas a la vez, la presicion (devolver lo que sea relevante), exhaustividad o recall (que no se le escape info relevante que si exista)
- **Poruqe los resultados no siempre son relevantes?** Porque el matching lexico no siempre es lo mismo que el matching semantico

UN ejemplo de SRI es el motor de busqueda de Google, o el buscador dentro de una biblioteca digital. Es importante notar que un SRI es diferente de un sistema de BDD por tres simples razones

- NO hay una respuesta exacta ni unica
- El objeto de trabajo es texto no estructurado
- La consulta no describe formalmente lo que se busca, sino que lo insinua

Componentes de un SRI

Los componentes de un SRI son los siguientes:

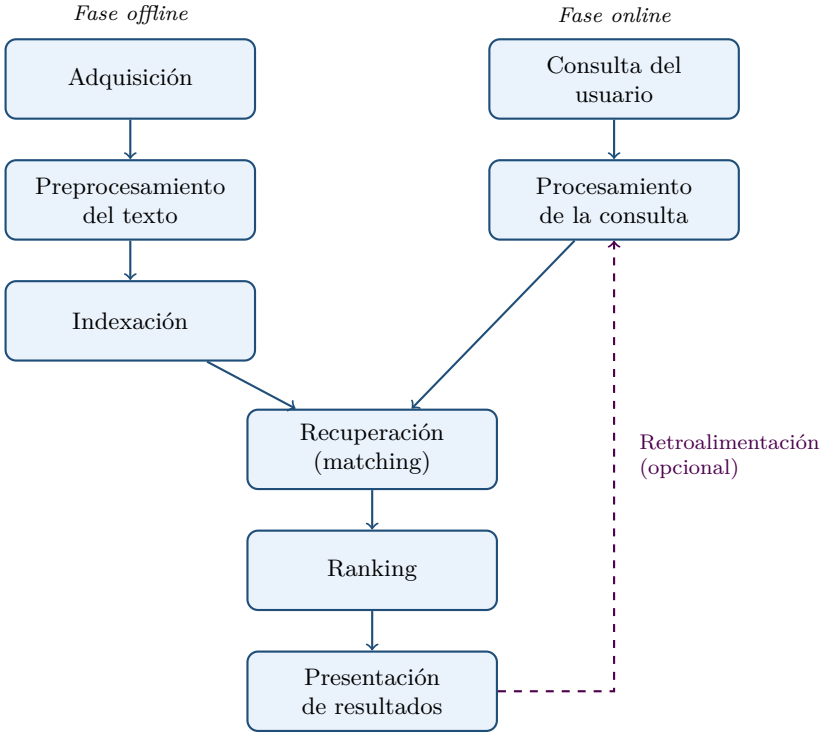
- Adquisición: Recopilación de documentos y construcción de la colección
- Preprocesamiento del texto: Limpieza, normalización, tokenización, stemming, eliminación de stopwords
- Indexación: Construcción de índices invertidos u otras estructuras de búsqueda. ESte paso es de vital importancia porque lo que no se indexa, es como si no existiera.
- Procesamiento de la consulta: Transformación de la consulta del usuario en una forma que pueda ser comparada con los documentos
- Recuperación: Búsqueda de documentos candidatos en el índice
- Ranking: Ordenación de los documentos recuperados según su relevancia
- Presentación de resultados: Mostrar los documentos al usuario
- Retroalimentación: Ajuste de la consulta basado en la relevancia de los documentos mostrados

Cómo se conectan las fases?

Las fases no forman una sola fila, se dividen en dos ramas que corren por separado y convergen en el matching:

- **Rama de la colección (offline)**: Adquisición, Preprocesamiento e Indexación construyen el índice invertido antes de que llegue ninguna consulta.
- **Rama de la consulta (online)**: el usuario escribe su consulta y el sistema le aplica el mismo preprocesamiento que a los documentos, para que ambos lados queden en la misma forma y se puedan comparar.

Ambas ramas se encuentran en la Recuperación (matching), que usa el índice para hallar los documentos candidatos. De ahí en adelante el flujo es único: Ranking los ordena y Presentación de resultados se los muestra al usuario. La Retroalimentación, cuando existe, cierra el ciclo regresando al Procesamiento de la consulta para ajustarla con lo que el usuario marcó como relevante.



Preprocesamiento del texto

El preprocesamiento es la fase de limpieza que corre antes de indexar. Típicamente se quitan:

- Plurales
- Signos de puntuación
- Formato
- Stop words, o sea artículos, conjunciones y preposiciones

Después se hace **stemming** o **lematización**, que es básicamente dejar el texto hablando como cavernícola: se quitan diminutivos y terminaciones hasta quedarse con la raíz.

Idea clave

Hacer stemming es truncar las palabras estadísticamente. No se consulta ningún diccionario, se corta según reglas que en promedio funcionan, así que el pedazo que queda no siempre es una palabra real. *Corriendo*, *corrió* y *correremos* terminan los tres en *corr*.

La lematización hace el mismo trabajo pero por el camino caro: usa diccionario y análisis morfológico, así que devuelve el lema (*correr*, una palabra que sí existe) en lugar de un truncado.

Todo esto se hace por dos razones. La primera es que reduce el vocabulario, y el vocabulario es el que define el tamaño del índice. La segunda es que sin esto el matching falla por pura forma: la consulta dice *corrió* y el documento dice *corriendo*, que para una comparación de cadenas son dos cosas distintas.

El algoritmo de Porter

El truncador estándar para inglés... lo publicó Martin Porter en 1980 y sigue siendo el que se usa por omisión en recuperación de información.

No consulta ningún diccionario. Son reglas de sufijos agrupadas en cinco pasos que se aplican en orden, y dentro de cada paso gana el sufijo más largo que empate: primero los plurales y las terminaciones de verbo, luego los sufijos dobles como *ational* o *izer*, y al final la *e* muda.

La pieza que evita que destroce palabras cortas es la medida m. Cada palabra se modela como

$$[C]\,(VC)^m\,[V],$$

donde C es una racha de consonantes y V una de vocales, así que m mide qué tan larga es la raíz. Casi toda regla lleva una condición sobre m, y por eso el mismo sufijo se trata distinto según la palabra: *feed* se queda igual porque tiene $m = 0$, mientras que *agreed* sí se trunca a *agre*.

Idea clave

El resultado no tiene que ser una palabra real, y no importa: el stem solo es la llave con la que se agrupan las variantes. *Studies*, *studied* y *studying* caen los tres en *studi*, que no existe, pero con eso la consulta ya empata con los tres documentos.

Se equivoca en las dos direcciones. Junta lo que no va junto: *universal*, *university* y *universe* terminan los tres en *univers*. Y deja separado lo que sí va junto: *europe* da *europ*, pero *european* se queda completo. Es ruido conocido y en IR sale a cuenta.

Vector Space Model

Idea clave

Todo modelo de recuperación define dos cosas: como se guarda la información y como se recupera. El modelo de espacio vectorial contesta las dos con la misma herramienta, un vector.

Salton ideó representar los documentos como vectores, y desde entonces los documentos se representan como vectores en un espacio de N dimensiones, donde N es el número de términos de la colección.

Ojito...

Término no es lo mismo que palabra. Un término es lo que quedó después del preprocesamiento, o sea la raíz o el lema, así que varias palabras del texto original pueden colapsar en un solo término y contar como una sola dimensión.

La representación queda como una tabla donde cada renglón es un documento y cada columna un término. La celda guarda el peso de ese término en ese documento, que puede ser la frecuencia cruda o algo como TF-IDF.

	corr	**perr**	**cas**	**com**
d_1	3	0	4	0
d_2	0	2	0	4
d_3	1	1	2	0
q	1	1	0	0

La consulta se trata como cualquier otro documento: se preprocesa igual y se convierte en un vector del mismo espacio. Por eso q aparece como un renglón más de la tabla y no como algo aparte.

Idea clave

Aquí la relevancia se mide como similaridad. Un documento es relevante si su vector se parece al vector de la consulta.

La medida usual es la similitud coseno, que compara la dirección de los dos vectores e ignora su magnitud. Eso importa porque si no, un documento largo ganaría solo por ser largo:

$$\text{sim}(d, q) = \frac{\vec{d} \cdot \vec{q}}{\left\|\vec{d}\right\| \, \|\vec{q}\|} = \frac{\sum_{i=1}^{N} d_i \, q_i}{\sqrt{\sum_{i=1}^{N} d_i^2} \sqrt{\sum_{i=1}^{N} q_i^2}} \tag{2}$$

Con la tabla de arriba, q pide *corr* y *perr*. Sale primero d_3 con 0.58, luego d_1 con 0.42 y al final d_2 con 0.32. Gana d_3 porque cubre los dos términos de la consulta, aunque d_1 mencione *corr* tres veces: lo que d_1 tiene de más es *cas*, que a la consulta no le importa y solo le alarga el vector.

4. Continuación (Vector Space Model)

26 de agosto de 2026

Resumen rápido. Continuación del modelo de espacio vectorial. Primero de dónde viene, o sea las consultas booleanas que había antes. Luego el tema central de la sesión: cómo se llenan los vectores, con los tres esquemas de ponderación y el principio de que la importancia de un término decrece conforme aparece en más documentos. Cerramos con los pros y contras del enfoque y con un repaso de los otros modelos de recuperación.

De dónde viene: las consultas booleanas

Antes del modelo de espacio vectorial estaban las consultas booleanas, que fueron la primera forma de recuperar información. La consulta se armaba con operadores booleanos: quiero esto **y** esto, esto **o** lo otro, esto pero **no** aquello.

El vector space model rompe con eso: ya no representa la consulta ni los documentos como expresiones lógicas, sino como vectores. Al inicio fue con álgebra vectorial simple, y después se extendió a cosas más complejas.

Idea clave

El cambio de fondo no es la notación sino el resultado. Una consulta booleana parte la colección en dos, los documentos que cumplen y los que no. Un vector permite ordenar por qué tanto se parecen, y eso es lo que da el ranking.

Cómo se llenan los vectores

La forma en que se llenan los vectores sirve para describir la importancia de un término en el documento. Puede ser el número de veces que aparece la palabra, o alguna variante de eso.

El principio que rige todo el pesado es este: la importancia general de un término decrece proporcionalmente a su ocurrencia en la colección entera. Hay atributos, palabras y términos que son tan frecuentes que no sirven para discriminar.

Ojito...

Esos términos tan frecuentes no se eliminan en el procesamiento. No hace falta: el esquema de pesado se encarga de dejarlos en un peso cercano a cero, así que se anulan solos.

Hay muchos métodos para llenar la matriz, pero todos se basan en lo mismo, la ponderación por número de veces y la importancia que decrece.

Binary weights

El peso vale 1 si el término aparece en el documento y 0 si no. Solo registra presencia, así que un término que sale una vez pesa lo mismo que uno que sale cincuenta.

Term frequency

Se usa la frecuencia, pero normalizada. Con $f_{i,j}$ el número de veces que el término i aparece en el documento j,

$$\mathrm{tf}_{i,j} = \frac{f_{i,j}}{\max_k f_{k,j}},$$

donde el denominador es la frecuencia del término más frecuente de ese documento. Se normaliza porque si no, un documento largo tendría pesos más grandes solo por ser largo.

TF-IDF

Es el esquema estándar, y combina las dos mitades del principio: qué tan seguido sale el término en *este* documento, contra en cuántos documentos de la colección sale. Con N el total de documentos y n_i cuántos contienen al término i,

$$w_{i,j} = \mathrm{tf}_{i,j} \times \mathrm{idf}_i, \qquad \mathrm{idf}_i = \log \frac{N}{n_i}.$$

Idea clave

El idf es la formalización de que la importancia decrece. Un término que aparece en todos los documentos tiene $n_i = N$, así que $\log(N/n_i) = \log 1 = 0$ y su peso se anula sin que nadie lo haya quitado a mano.

Okapi BM25

Es un refinamiento de TF-IDF, y viene del sistema Okapi. Arregla dos cosas que TF-IDF hace mal.

La primera es que en TF-IDF la frecuencia entra de forma lineal, así que un término que aparece cien veces pesa cien veces más que uno que aparece una. Eso no corresponde a cómo funciona la evidencia: que una palabra pase de aparecer una vez a dos dice muchísimo sobre el tema del documento, pero que pase de cincuenta a cincuenta y uno ya no dice casi nada.

La segunda es que la normalización por longitud de TF-IDF es fija. BM25 la vuelve ajustable, porque no siempre conviene normalizar igual: un documento puede ser largo por tratar más temas, o por ser verboso sobre el mismo.

Con $|d|$ la longitud del documento en términos y avgdl la longitud promedio de la colección,

$$\mathrm{BM25}(d, q) = \sum_{i \in q} \mathrm{idf}_i \cdot \frac{f_{i,d}\,(k_1 + 1)}{f_{i,d} + k_1 \left(1 - b + b\,\dfrac{|d|}{\mathrm{avgdl}}\right)}.$$

Los dos parámetros son los que se ajustan:

- k_1 controla qué tan rápido satura la frecuencia. Con $k_1 = 0$ el modelo se vuelve binario, solo importa que el término esté; conforme k_1 crece se parece más a la frecuencia cruda. Lo típico es entre 1.2 y 2.0.
- b controla la normalización por longitud. Con $b = 0$ no normaliza nada y con $b = 1$ normaliza por completo. Lo típico es 0.75.

Idea clave

La saturación es la idea central. Cuando $f_{i,d}$ crece sin límite, la fracción tiende a $k_1 + 1$ y ahí se queda, así que la contribución de un término está acotada por más veces que aparezca. TF-IDF no tiene tope.

A pesar de ser de los noventa, BM25 sigue siendo el baseline fuerte contra el que se comparan los métodos neuronales de recuperación, y es lo que usan por defecto motores como Lucene y Elasticsearch.

Qué no captura el modelo

Esta representación no considera la semántica de los documentos. El vector solo sabe qué términos hay y con qué peso, no qué significan.

A favor:

- Es fácil de explicar
- Está matemáticamente bien estructurado
- Permitió el *approximate query matching*, o sea recuperar documentos que no calzan exacto con la consulta

En contra:

- No entiende la sinonimia: *carro* y *auto* son dos dimensiones distintas y ortogonales
- No entiende la semántica
- Las consultas estructuradas son difíciles de modelar
- La normalización aumenta el costo computacional
- Hay que pesar los términos, y de ahí salen los esquemas de arriba

Otros modelos de recuperación

- **Modelos booleanos.** Consulta con operadores lógicos, sin ranking: un documento cumple o no cumple.
- **Probabilistic indexing.** Pesa los términos del índice por su probabilidad de ser relevantes.
- **Vector space model.** El que estamos viendo.
- **Probabilistic retrieval.** Ordena los documentos por la probabilidad de que sean relevantes para la consulta.
- **Fuzzy set models.** La pertenencia al conjunto de documentos relevantes es gradual y no de todo o nada.
- **Inference networks.** Modelan la recuperación como inferencia sobre una red bayesiana.
- **Language models.** Cada documento define un modelo de lenguaje, y se ordena por la probabilidad de que ese modelo genere la consulta.
- **Word embeddings y enfoques neuronales.** Representan términos y documentos en un espacio denso aprendido, que sí captura sinonimia y semántica. Es la respuesta directa a las dos contras de arriba.

5. Modelos de lenguaje para IR

31 de agosto de 2026

Resumen rápido. En esta sesion vimos como se utilizan los modelos de lenguaje para la Info Retrieval y como se evalua el desempeno de un sistema de IR, nos quedamos deduciendo una formula de evaluacion de desempeno... la "Arturiana" y se quedo de tarea como desarrollarla completamente.

Modelos de lenguaje estadísticos

Un modelo de lenguaje estadístico asigna una probabilidad a una secuencia de m palabras por medio de una distribución de probabilidad. La secuencia se descompone con la regla de la cadena:

$$P(t_1, t_2, \ldots, t_m) = P(t_1)\, P(t_2 \mid t_1)\, P(t_3 \mid t_1 t_2) \cdots = \prod_{i=1}^{m} P(t_i \mid t_1, \ldots, t_{i-1}).$$

Cada término se condiciona a todo lo que vino antes, que es justo lo que vuelve el cálculo impráctico: para contextos largos casi ninguna secuencia se repite en el corpus. Lo que dice es,

$$P(t_1, t_2, \ldots, t_m)$$

la probabilidad de que aparezca la secuencia de palabras en ese orden $t_1, t_2, \ldots, t_m$.

Es igual a

$$P(t_1)\, P(t_2 \mid t_1) P(t_3 \mid t_1 t_2) \cdots P(t_m \mid t_1 \ldots t_{m-1})$$

La probablidad de que salga la primer palabra, por la probabilidad de que salga la segunda dado la primera y asi... Pero desarrollando los terminos con la definicion de probablidad condicional, resulta

$$P(t_1, t_2, \ldots, t_m) = P(t_1) \frac{P(t_1, t_2)}{P(t_1)} \frac{P(t_1, t_2, t_3)}{P(t_1, t_2)} \cdots \frac{P(t_1, \ldots, t_m)}{P(t_1, \ldots, t_{m-1})} = \prod_{i=1}^{m} P(t_i \mid t_1, \ldots, t_{i-1}).$$

Secuencia de unigramas

La forma más simple de que un modelo de lenguaje deseche el contexto probabilístico es tratar cada término de manera independiente. Esto sirve por si no hay mucho contexto, y se llama secuencia de unigramas:

$$P(t_1, t_2, \ldots, t_m) = \prod_{i=1}^{m} P(t_i).$$

De dónde se sacan esas probabilidades?

Se estiman contando sobre el texto. Para el modelo M_d construido sobre el documento d, la estimación por máxima verosimilitud es la frecuencia relativa del término:

$$P(t \mid M_d) = \frac{\mathrm{tf}_{t,d}}{|d|},$$

o sea las veces que t aparece en d entre el total de términos de d.

Antes de entrar a cómo se usan en recuperación de información, vale notar que un modelo de lenguaje se usa en el día a día: en los autocompletados, traducción de texto, corrección ortográfica y reconocimiento de habla.

Cómo se usan los LM en IR

La idea principal es que los documentos pueden ser rankeados por su similitud de generar la consulta.

A cada documento lo voy a ver como si fuese un mundo, algo independiente, y sobre ese documento se calcula o se construye un modelo de lenguaje. Cuando llega una consulta se hace la pregunta de cuál es la probabilidad de generar la secuencia de la consulta a partir de cada modelo de lenguaje:

$$P(Q \mid M_{d_i}) = \prod_{t \in Q} P(t \mid M_{d_i}).$$

El objetivo es rankear los documentos por $P(d \mid Q)$, donde la probabilidad de un documento se interpreta como qué tan similar o relevante es a la consulta. Aplicando Bayes:

$$P(d_i \mid Q) = \frac{P(Q \mid d_i)\, P(d_i)}{P(Q)} \;\propto\; P(Q \mid d_i)\, P(d_i) \;\approx\; P(Q \mid M_{d_i}).$$

Ojito...

$P(Q)$ se puede tirar porque es la misma para todos los documentos: divide a todos por igual y no cambia el orden. Por eso la igualdad se vuelve proporcionalidad. Y si además se toma $P(d_i)$ uniforme, rankear por $P(d \mid Q)$ es lo mismo que rankear por $P(Q \mid M_{d_i})$.

$P(d)$ es la confianza, algo así como la probabilidad de que ese documento sea recuperado antes de ver la consulta. Ahí es donde entra información que no depende de la consulta, por ejemplo qué tan citado o qué tan reciente es el documento.

Qué pasa cuando veo un término nuevo

Si un término de la consulta no aparece en el documento, su tf es cero, y como el producto multiplica todos los términos, la probabilidad entera se va a cero. Un solo término no visto descarta el documento aunque los demás calcen perfecto.

Para solucionar esto se asume que los términos no vistos se vieron una vez:

$$P(t \mid M_d) = \frac{\mathrm{tf}_{t,d} + 1}{|d| + |V|},$$

con $|V|$ el tamaño del vocabulario.

Pero una forma más robusta es agregar un modelo de lenguaje para toda la colección completa, o sea hacer una mezcla entre el documento y la colección. Con M_C el modelo de la colección y $\lambda \in (0, 1)$ el peso que se le da al documento, la fórmula de ranking queda:

$$P(d \mid Q) \;\propto\; P(d) \prod_{t \in Q} \Big[\lambda\, P(t \mid M_d) + (1 - \lambda)\, P(t \mid M_C) \Big].$$

Idea clave

La mezcla nunca deja un factor en cero, porque si el término no está en el documento el segundo sumando lo rescata con lo que sabe la colección. Y de paso pesa mejor: un término raro en la colección tiene $P(t \mid M_C)$ chica, así que encontrarlo en el documento cuenta más. Es el mismo efecto del idf, pero saliendo de la probabilidad en lugar de estar puesto a mano.

Evaluación de IR

Lo primero que se puede medir es la **eficiencia**: qué tanto tiempo se tarda el modelo. Otra cosa que se puede medir es cuántos relevantes trajo y cuántos relevantes olvidó.

Se puede decir que el IR es subjetivo, porque la relevancia de algo es subjetiva, pero se hace el etiquetado con muchas personas para poder llegar a un consenso.

Cómo funciona el sistema?

La pregunta se contesta en tres niveles:

- **Procesamiento.** Qué tanto tiempo se tardó y el espacio que usó.
- **Búsqueda.** Qué tan buenos son los resultados.
- **Sistema.** La satisfacción del usuario.

Nos enfocaremos en evaluar la **efectividad** de recuperación, o sea el segundo nivel.

El problema de la relevancia

El problema de la efectividad es que está relacionada con la relevancia de los ítems recuperados, y la relevancia es escurridiza:

- No es binaria, es un valor continuo
- Incluso si fuese binaria, podría ser muy difícil de juzgar

Además depende de varias cosas a la vez:

- La **necesidad** de información del usuario
- **Situacional**, relacionado con esa necesidad y con el contexto en el que busca
- **Cognitiva**, depende de la percepción humana y del comportamiento
- **Dinámica**, cambia en el tiempo

Ojito...

La relevancia es subjetiva y depende de muchos valores. Por eso no se evalúa preguntándole a una sola persona, sino juntando juicios de varias hasta que haya consenso.

Paradigma de Cranfield

Es la forma estándar de evaluar, y consiste en armar una colección de prueba (*test collection*). Para que sirva hace falta:

- Muchos documentos, entre más mejor
- Muchas consultas
- *Relevance judgments* para todas las consultas

Se pone a una persona a hacer la recuperación sobre esa colección y sus resultados se van evaluando contra los juicios de relevancia. Como la colección, las consultas y los juicios quedan fijos, dos sistemas distintos se pueden comparar de forma justa.

Colecciones de prueba estándar

- TREC
- CLEF
- NTCIR

Tipos de medidas

Hay dos tipos:

- **Basadas en conjuntos.** Tenía que recuperar estos, pero me dio estos. Solo miran qué documentos salieron, no en qué orden.
- **Basadas en ranking.** Cuál es más importante que cuál. Aquí sí importa la posición, porque no es lo mismo que el relevante salga primero a que salga en el lugar cincuenta.

6. Medidas de evaluación en IR

2 de septiembre de 2026

Resumen rápido. En esta clase vimos de q las medidas basadas en conjuntos mas en profundidad y las basadas en ranking. Comenzamos desde la arturiana, presente mi modificacion pero no fue tan buena, despues pasamos a las métricas basadas en ranking igual tratando de deducirlas y vemis la diferencia entre presicion y recall

Métricas basadas en conjuntos

Todo parte de partir la colección en dos por dos criterios a la vez: qué documentos son relevantes para la consulta, y cuáles devolvió el sistema. La intersección es lo que salió bien (pero estos no tienen un orden en especifico, tampoco los del conjunto de train)

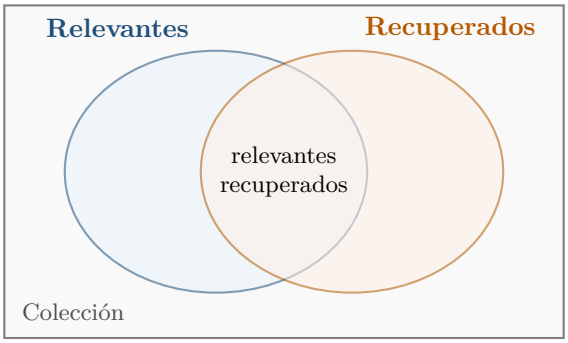


Figura 1: La intersección es lo que el sistema hizo bien. Recall mide qué tanto cubre del círculo izquierdo, precisión qué tanto del derecho

De ahí salen las dos medidas:

$$\text{recall} = \frac{\text{relevantes recuperados}}{\text{número de documentos relevantes}}, \qquad \text{precision} = \frac{\text{relevantes recuperados}}{\text{total de recuperados}}.$$

Ojito...

Cuando el sistema devuelve cero documentos, la precisión queda como una división entre cero, que no está determinada. En IR eso no es un problema: un sistema que no devuelve nada no está diciendo algo, pero tampoco está diciendo tonterías ni cosas equivocadas, entonces se toma como 1, el principio de "En boca cerrada no entran moscas"

F1

Una medida más robusta es F_1, que es la media armónica de precisión y recall:

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}.$$

Idea clave

Media armonica es el inverso del promedio de los inversos. Para dos números a y b:

$$H(a,b) = \frac{2}{\frac{1}{a} + \frac{1}{b}} = \frac{2ab}{a+b}.$$

El 1 del nombre de F_1 es el peso que se le da a cada una, y en este caso las pondera igual. La forma general es

$$F_\beta = (1 + \beta^2) \frac{P \cdot R}{\beta^2 \cdot P + R},$$

donde $\beta > 1$ favorece al recall y $\beta < 1$ a la precisión.

Idea clave

La media armónica se usa en lugar del promedio simple porque castiga los desbalances. Un sistema con $P = 1$ y $R = 0.01$ tiene promedio 0.5, que suena decente, pero $F_1 = 0.02$. Con el promedio se puede hacer trampa devolviendo un solo documento seguro; con la armónica no.

Métricas basadas en ranking

Las anteriores son métricas de conjunto: no les importa el orden en que salieron los documentos. Pero un sistema real devuelve un ranking, y ahí la posición importa.

El orden es por relevancia, aunque puede haber otros criterios como el número de citas, la distancia o la calificación en estrellas de un lugar. Esos son independientes de la relevancia semántica.

P@N

Precisión hasta la posición N: se corta el ranking en los primeros N resultados y se calcula la precisión ahí nada más. P@10 es la típica, porque es lo que cabe en la primera página. No dice nada de lo que quedó más abajo.

MRR

Mean reciprocal rank. Se fija solo en la posición del primer documento relevante: si aparece en la posición k, el rango recíproco es $1/k$. Se promedia sobre el conjunto de consultas Q:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{k_i}.$$

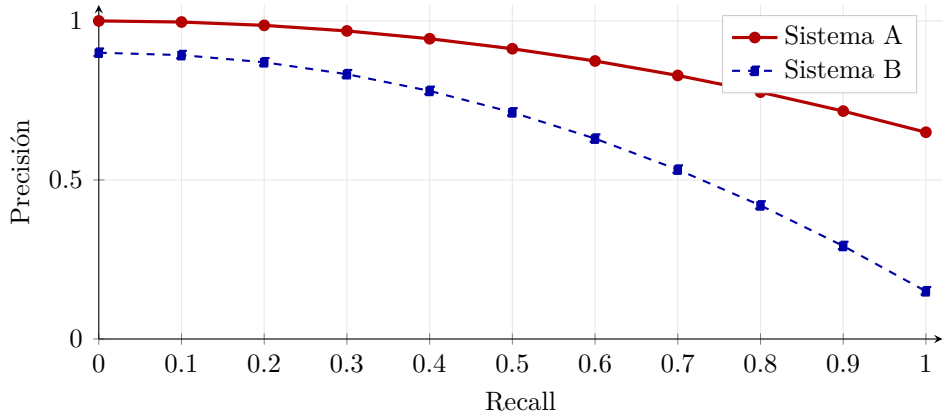
Se usaba en *question answering*, donde solo importa que la respuesta correcta salga hasta arriba y lo demás da igual.

Curva de recall-precisión

En lugar de un número suelto, se grafica la precisión en función del recall. Cada punto es un corte distinto del ranking, y la curva se promedia sobre todas las consultas. Sirve para comparar sistemas: el que quede más arriba es mejor en todo el rango.

Ojito...

La precisión casi siempre baja conforme sube el recall. Para recuperar más documentos relevantes hay que bajar el umbral, y al bajarlo entran también más irrelevantes. Ese compromiso es lo que la curva dibuja.



MAP

Mean average precision es de las medidas más usadas. Para cada consulta se promedia la precisión medida justo en las posiciones donde apareció un documento relevante, y después se promedian esas consultas:

$$\mathrm{AP}(q) = \frac{1}{R}\sum_{k=1}^{n} P@k \cdot \mathrm{rel}(k), \qquad \mathrm{MAP} = \frac{1}{|Q|}\sum_{q\in Q} \mathrm{AP}(q),$$

donde R es el número de documentos relevantes, n el tamaño del ranking y $\mathrm{rel}(k)$ vale 1 si el documento en la posición k es relevante y 0 si no.

Esa suma es una aproximación discreta del área bajo la curva de recall-precisión: cada término aporta el pedazo que corresponde a un documento relevante.

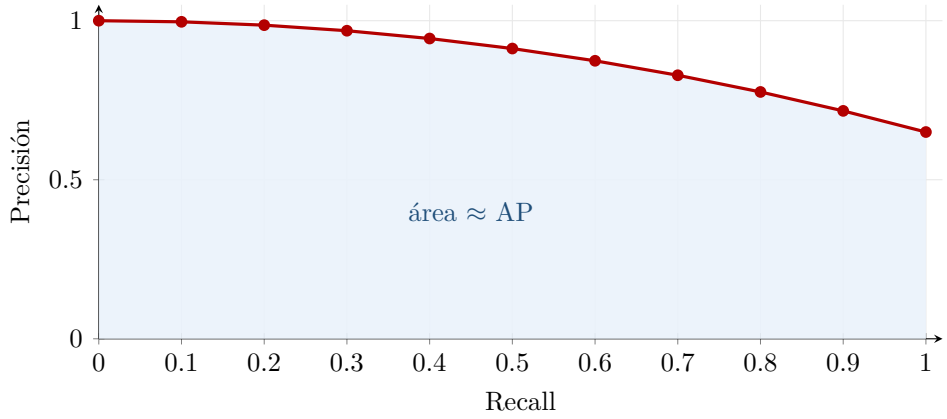


Figura 2: La average precision de una consulta es el área bajo su curva de recall-precisión.

AP como suma de Riemann

La correspondencia con el área no es una analogía suelta, la suma que define AP es literalmente una suma de Riemann. Si la consulta tiene R documentos relevantes, cada uno que aparece sube el recall en exactamente $1/R$, así que le toca un rectángulo de ese ancho y de altura $P@k$. Sumar las áreas de los rectángulos es calcular

$$\mathrm{AP}(q) = \sum_{k=1}^{n} \underbrace{P@k \cdot \mathrm{rel}(k)}_{\text{altura}} \cdot \underbrace{\frac{1}{R}}_{\text{ancho}}.$$

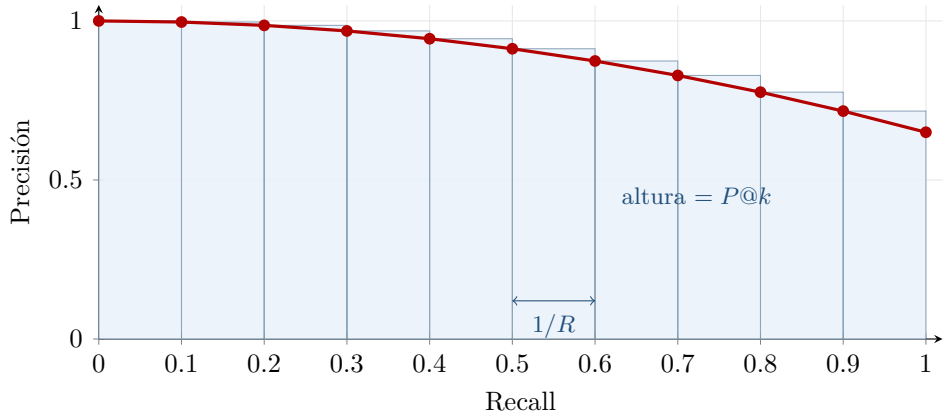


Figura 3: Cada documento relevante aporta un rectángulo de ancho $1/R$ y altura $P@k$. La suma de sus áreas es AP, con $R = 10$ en el dibujo.

Ojito...

Por eso AP no es el promedio de las precisiones de todo el ranking, sino solo de las posiciones donde hubo un relevante. Las demás posiciones no aportan rectángulo, porque no mueven el recall.

Ventajas y desventajas

A favor:

- Es un solo valor, así que compara sistemas de un vistazo
- Se ajusta muy bien y no se fija tanto en las colas del ranking
- Es muy buena para métricas binarias, o sea cuando un documento simplemente es relevante o no lo es

En contra:

- No es tan buena para *ratings* numéricos de grano fino, porque asume que todos los relevantes son igualmente relevantes, y eso no es cierto

Pendiente por resolver

Preguntas que dejo el Dr para la sig clase:

- pq todo lo que recupera un sistema de IR no es relevante?
- pq es difícil tener 100 de recall?
- ideas para solucionarlo?

7. Expansión de consultas y clustering

7 de septiembre de 2026

Resumen rápido. Arrancamos respondiendo las tres preguntas que quedaron pendientes de la clase pasada, y las tres tienen la misma raíz: la consulta es ambigua. De ahí salió la expansión de consultas, con sus tres vías (tesauros, retroalimentación del usuario y automática), el método de Rocchio y la semántica distribucional. Cerramos con clustering, que ataca el mismo problema pero del lado del resultado.

Por qué no se llega al 100

Las tres preguntas que dejó el Dr. la clase pasada se responden con lo mismo.

Es difícil tener 100 de precisión porque la consulta es ambigua. Basta imaginar una consulta de una sola palabra: el sistema no tiene con qué saber a cuál de sus sentidos se refiere el usuario, así que devuelve documentos de todos, y los que sobran cuentan como error.

Es difícil tener 100 de recall por la misma razón. Si la consulta dice *coche* y el documento relevante dice *automóvil*, el sistema no los empata y ese documento se queda fuera.

Idea clave

La salida es ser lo más específico posible en la consulta. El problema es que el usuario no sabe serlo: no conoce la colección ni el vocabulario con el que está escrita. Entonces el sistema lo hace por él, y eso es la expansión de consultas.

Expansión de consultas

Expandir una consulta es agregarle términos que el usuario no escribió pero que apuntan a lo mismo que quiso decir: sinónimos, variantes morfológicas, términos relacionados. La consulta original no se sustituye, se le suman términos para que empate con más documentos relevantes.

Sirve sobre todo para el recall, que es lo que la ambigüedad rompe primero. El riesgo va en la otra dirección: cada término que se agrega puede traer documentos que no vienen al caso, y eso baja la precisión.

Hay tres vías para decidir qué términos agregar:

- **Tesauros.** Un recurso externo ya construido, que dice qué palabras se relacionan con cuáles.
- **Retroalimentación del usuario.** Se le pregunta cuáles de los resultados le sirvieron y se aprende de ahí.
- **Expansión automática.** Las asociaciones se sacan de la propia colección, sin recurso externo ni usuario.

Basada en tesauros

A cada término de la consulta se le agregan sus sinónimos y términos relacionados según un tesauro, que puede ser general, como WordNet, o del dominio.

Incrementa el recall, porque la consulta empieza a empatar con documentos escritos con otro vocabulario. Pero baja la precisión, y la razón es que el tesauro no sabe de cuál sentido se está hablando: si la consulta trae *banco*, le agrega lo de la institución financiera y lo del asiento por igual.

Retroalimentación de relevancia

El sistema devuelve una primera tanda de resultados, el usuario marca cuáles le sirvieron y cuáles no, y con eso el sistema arma una consulta nueva y vuelve a buscar.

La ventaja es que la evidencia viene del usuario, que es el único que sabe qué necesita. La desventaja es que hay que pedírsela, y en un buscador real casi nadie se toma la molestia.

Método de Rocchio

Es la forma estándar de aplicar esa retroalimentación. Como todo son vectores, la consulta se puede mover en el espacio: la idea es acercarla a los documentos que el usuario marcó como relevantes y alejarla de los que marcó como irrelevantes.

$$\vec{q}_m = \alpha\,\vec{q}_0 + \beta\,\frac{1}{|D_r|}\sum_{\vec{d}\in D_r}\vec{d} - \gamma\,\frac{1}{|D_{nr}|}\sum_{\vec{d}\in D_{nr}}\vec{d}$$

donde $\vec{q}_0$ es la consulta original, D_r y D_{nr} son los conjuntos de documentos marcados como relevantes e irrelevantes, y α, β y γ pesan cuánto cuenta cada parte.

Ojito...

En la práctica $\beta > \gamma$. La evidencia positiva es más confiable que la negativa: que al usuario le haya servido un documento dice mucho, que no le haya servido puede deberse a que ya lo conocía o a que no lo abrió.

Retroalimentación pseudo relevante

Igual que la anterior, pero sin preguntarle nada al usuario: se asume que los primeros n resultados son relevantes y se aplica Rocchio con ellos.

Funciona bien cuando la primera tanda ya es decente. Cuando no lo es, refuerza el error: la consulta se mueve hacia documentos equivocados y la segunda tanda sale peor que la primera.

Semántica distribucional

La tercera vía, la automática. En lugar de un tesauro externo o del usuario, las asociaciones entre palabras salen de la propia colección.

Se parte de la matriz de términos contra documentos y se multiplica por su transpuesta. El resultado es una matriz cuadrada de términos contra términos, donde cada celda dice en cuántos contextos aparecen juntas dos palabras.

$$C = AA^T, \qquad A \in \mathbb{R}^{|V|\times|D|}, \qquad C \in \mathbb{R}^{|V|\times|V|}$$

Idea clave

El tamaño del contexto cambia qué tipo de relación sale. Con un contexto grande se correlacionan palabras relacionadas por tema: médico con hospital. Con un contexto chico la correlación se acerca a la sinonimia, porque dos palabras rodeadas de las mismas pocas palabras suelen ser intercambiables.

Clustering en recuperación de información

Todo lo anterior ataca la ambigüedad sobre la marcha, antes de buscar. El clustering la ataca del otro lado, en el momento de presentar el resultado: se agrupan los resultados, por ejemplo por tema, y el usuario escoge el grupo que le interesa. Ahí desambigua él, sin tener que reescribir la consulta.

Idea clave

Hipótesis del cluster: los documentos que caen en el mismo grupo se comportan de forma similar respecto a la relevancia. Si uno sirve para una necesidad de información, es probable que los demás del grupo también.

Tiene dos usos principales:

- **Clustering de la colección.** Se agrupa antes de buscar. Da más eficiencia, porque la búsqueda se restringe a los grupos prometedores en vez de recorrer todo, y tiende a mejorar el recall.
- **Clustering de resultados.** Se agrupa lo que se va a devolver. No cambia qué se recuperó, cambia cómo se presenta, y con eso la información le llega al usuario de forma más efectiva.

8. Representaciones más allá de las palabras

9 de septiembre de 2026

Resumen rápido. En esta clase impartió el Dr. Aurelio. Vimos repaso de conceptos que ya habíamos visto antes, como la ambigüedad y los problemas de sinonimia y polisemia, y cómo afectan la recuperación de información. También se introdujo la pregunta de fondo del indexado: cuál es la mejor unidad de representación para un documento y el POS tagging

Repaso: ambigüedad

La ambigüedad es la condición donde una información puede entenderse o interpretarse en más de un sentido. El contexto la mitiga, no la elimina.

- **Léxica.** Palabras o frases con más de un significado posible: *banco*, *gato*.
- **Sintáctica.** La oración se puede parsear de varias formas, y cada árbol da una interpretación distinta.
- **Semántica.** El significado de una palabra o concepto es vago o queda abierto, aunque la sintaxis sea clara.

Qué subyace: sinonimia y polisemia

Detrás de la ambigüedad hay dos problemas del lenguaje natural, y cada uno rompe una métrica diferente.

Problema	Qué es	Qué degrada
Sinonimia	Varias palabras, un significado	Recall
Polisemia	Una palabra, varios significados	Precisión

Idea clave

La sinonimia baja el recall porque el documento relevante usa otra palabra y nunca se recupera. La polisemia baja la precisión porque la palabra empata con documentos de otro sentido y sí se recupera, pero sobra. Son direcciones opuestas, por eso no hay una sola solución que arregle las dos.

Indexado de documentos

El objetivo del indexado es encontrar los significados importantes de un documento y construir con ellos una representación interna. Lo que no se indexe no existe para el sistema.

Tres factores para juzgar una representación:

- **Precisión semántica.** Qué tan bien captura el significado.
- **Exhaustividad.** Si cubre todo el contenido del documento o solo una parte.
- **Manejabilidad.** Qué tan fácil le resulta a la computadora manipularla.

Cuál es la mejor unidad de representación

Unidad	Cobertura	Precisión
Caracter	Total	Nula
Palabra	Buena	Baja
Frase	Pobre	Mayor
Concepto	Pobre	Alta

Idea clave

Ninguna gana. Al subir de unidad se gana precisión y se pierde cobertura: hay pocas frases y menos conceptos, así que casi nada empata. Al bajar pasa lo contrario, todo empata y nada discrimina. Por eso el modelo de espacio vectorial se quedó en la palabra: es el punto donde las dos curvas todavía se cruzan.

Etiquetado gramatical (POS tagging)

Tarea del procesamiento de lenguaje que consiste en asignarle a cada palabra de una oración la categoría gramatical que asume *en esa secuencia*, no la que tendría aislada.

Entrada	Una cadena de palabras más un conjunto de etiquetas
Salida	Una sola etiqueta, la mejor, para cada palabra

La salida es una secuencia única: el etiquetador no propone alternativas, se compromete con una.

Ejemplo 8.1. Sobre una oración de la colección Time:

The	*allies*	*saw*	*the*	*first*	*outlines*
DT	NNS	VBD	DT	JJ	NNS

The sale determinante, *saw* verbo en pasado. Fuera de contexto *saw* también es el sustantivo *sierra*, y es justo la ambigüedad léxica que el etiquetado resuelve usando la secuencia.

Los conjuntos de etiquetas

Los dos de referencia salen de sendos corpus anotados a mano. El de **Brown** es el grande, del orden de 87 etiquetas, con distinciones muy finas. El de **Penn Treebank** lo simplificó a 36 categorías gramaticales, más los signos de puntuación, y es el que se usa por omisión hoy.

Etiqueta	Categoría	Ejemplo
NN, NNS	Sustantivo singular, plural	*force, allies*
NNP	Nombre propio	*Kennedy*
VB, VBD	Verbo base, pasado	*help, saw*
VBG, VBN	Gerundio, participio	*suppressing*
JJ	Adjetivo	*nuclear*
RB	Adverbio	*finally*
DT	Determinante	*the*
IN	Preposición	*after*
PRP	Pronombre personal	*they*
CD	Número cardinal	*1960*

Observación. El etiquetado gramatical se empezó a trabajar en los años sesenta, en la misma época que el corpus de Brown.

9. Indexado más allá de la palabra

14 de septiembre de 2026

Resumen rápido. Se cerró el etiquetado gramatical con las dos formas de resolverlo, la de la etiqueta más frecuente y la de modelos ocultos de Markov, y con el veredicto de para qué sirve en indexado. De ahí salieron las otras dos propuestas para representar un documento con algo distinto de la palabra suelta: las unidades compuestas (frases, entidades nombradas, n-gramas, secuencias frecuentes maximales) y el indexado por sentidos.

Cómo se resuelve el etiquetado

Primer enfoque: la etiqueta más frecuente

A cada palabra se le asigna la etiqueta que más veces recibe en un corpus anotado. Si w admite las etiquetas $t_1, \ldots, t_k$, se escoge

$$\hat{t}(w) = \arg\max_{t_i} P(t_i \mid w), \qquad P(t_i \mid w) = \frac{C(t_i, w)}{C(w)},$$

donde $C(t_i, w)$ es el número de veces que w aparece etiquetada t_i en el corpus. El contexto no entra en la cuenta: la decisión es la misma en toda oración donde aparezca w.

Aun así acierta alrededor del 91 por ciento en inglés. La razón es que la mayoría de las palabras no son ambiguas, y las que sí lo son tienen una distribución muy sesgada hacia una de sus etiquetas. Por ejemplo *heat* se usa mucho más como sustantivo que como verbo, así que este método siempre lo etiqueta sustantivo y casi siempre le atina.

Ojito...

Ese 91 por ciento es el piso contra el que se compara, no un buen resultado. Es una palabra mal etiquetada de cada once, y los errores no se reparten: caen justo sobre las palabras ambiguas, que son las frecuentes y las que más peso tienen en la interpretación.

Segundo enfoque: HMM tagging

El etiquetado con modelos ocultos de Markov no decide palabra por palabra, escoge la secuencia completa de etiquetas. Dada la secuencia de palabras w_1^n, busca

$$\hat{t}_1^n = \arg\max_{t_1^n} P(t_1^n \mid w_1^n) = \arg\max_{t_1^n} P(w_1^n \mid t_1^n)\, P(t_1^n),$$

donde la segunda igualdad es la regla de Bayes, quitando $P(w_1^n)$ porque es la misma para todas las secuencias candidatas y no cambia cuál gana.

Así como está no se puede calcular, hay $|T|^n$ secuencias posibles. Se meten dos suposiciones:

- **Emisión.** Cada palabra depende solo de su propia etiqueta, no de las vecinas: $P(w_1^n \mid t_1^n) \approx \prod_i P(w_i \mid t_i)$.
- **Transición.** Cada etiqueta depende solo de la anterior, o sea un bigrama: $P(t_1^n) \approx \prod_i P(t_i \mid t_{i-1})$.

Con las dos, el criterio queda

$$\hat{t}_1^n = \arg\max_{t_1^n} \prod_{i=1}^{n} P(w_i \mid t_i)\, P(t_i \mid t_{i-1}).$$

Las dos probabilidades se estiman contando sobre el corpus anotado, igual que antes. Y para no enumerar las $|T|^n$ secuencias se usa Viterbi, programación dinámica que resuelve el máximo en $O(n \cdot |T|^2)$.

Ejemplo 9.1. En *to heat the water*, el método de la etiqueta más frecuente pone *heat* como sustantivo, porque ese es su uso más común. El HMM lo corrige sin saber nada especial de *heat*: la transición manda. Después de TO casi siempre viene un verbo en forma base, así que $P(\text{VB} \mid \text{TO})$ es órdenes de magnitud mayor que $P(\text{NN} \mid \text{TO})$, y el producto se voltea a favor de VB aunque la emisión $P(\text{heat} \mid \text{NN})$ sea la más grande de las dos.

Idea clave

Ahí está la diferencia entre los dos enfoques. El primero pregunta qué es esta palabra normalmente. El segundo pregunta qué secuencia de etiquetas hace más probable a la oración entera, y por eso puede ir contra la etiqueta favorita de una palabra cuando el contexto lo pide.

Sirve el POS para indexar?

Indexar con la etiqueta pegada al término separa *saw* verbo de *saw* sustantivo, y con eso sube la precisión. Pero baja el recall: un documento que usa la palabra con la otra etiqueta deja de empatar, aunque hable de lo mismo.

Ojito...

El veredicto de la clase fue que no vale la pena como técnica de indexado por sí sola, ni siquiera si el etiquetado se hace a mano. El límite no es la calidad del etiquetador, es el intercambio: lo que gana en precisión lo paga en recuerdo. El POS tagging se queda como paso intermedio, insumo para extraer frases, no como el índice mismo.

Segunda propuesta: unidades compuestas

Usar palabras sueltas como términos de indexado da buena exhaustividad y especificidad pobre: se cubre todo el documento, pero cada término discrimina poco. Y hay combinaciones cuyo significado no se recupera separando las palabras, *apple pie* no es una manzana más un pastel.

De ahí la idea de usar términos de indexado de más de una palabra, para reducir la ambigüedad.

Ejemplo 9.2. Sobre *this apple pie looks good and is a real treat*:

Relación	Término
Adjetivo, sustantivo	*real treat*
Sustantivo, sustantivo	*apple pie*
Sujeto, verbo	*pie looks*
Verbo, objeto	*is treat*

La complicación es de dónde salen: hay que extraerlas del texto ya etiquetado con POS o del árbol sintáctico, y el análisis sintáctico completo es caro.

La alternativa barata es el **chunking**, o análisis superficial. En vez de construir el árbol entero, reconoce bloques no recursivos (sobre todo sintagmas nominales) con patrones sobre las etiquetas POS. La operación complementaria es el **chinking**, que define qué sacar de un bloque ya formado en lugar de qué meter.

Entidades nombradas

Son las entidades que el texto menciona por nombre. Las tres clases aceptadas son personas, lugares y organizaciones. Suelen agregarse fecha, hora, medidas, correos y direcciones.

La ambigüedad sigue presente aunque se trabaje con entidades: un nombre como *Charlie Kirk* puede ser una persona o un lugar, porque a los lugares se les pone nombre de persona.

Se procesan en dos pasos:

1. **Identificación.** Detectar los límites, dónde empieza y dónde termina la entidad nombrada.
2. **Clasificación.** Asignarle una de las categorías predefinidas.

Cómo se ha atacado, en orden histórico:

Enfoque	**Cómo**	**Limitación**
Reglas basadas en conocimiento	Expertos se sientan a escribir patrones y a etiquetar a mano	No escala, y se rehace entero para cada dominio
Aprendizaje automático y estadística (HMM, CRF)	Explota rasgos de los elementos de interés: mayúsculas, patrones, POS, prefijos y sufijos	Requiere corpus extensos ya anotados
Aprendizaje profundo y transformers (BERT, RoBERTa)	Embeddings contextuales, la representación se aprende sola, sin ingeniería manual de rasgos	Mucho cómputo y conjuntos grandes para el ajuste fino por dominio

N-gramas

Un n-grama es una subsecuencia de n términos de una secuencia dada. Sobre *apple pie looks good*, los bigramas son *apple pie*, *pie looks* y *looks good*.

Lo que los hace atractivos:

- Se calculan fácil, una pasada con una ventana deslizante. No hacen falta etiquetador ni analizador sintáctico.
- Se pueden combinar n-gramas de distinto tamaño, y eso da flexibilidad al momento de buscar: la consulta empata con la granularidad que tenga.

El problema es la alta dimensionalidad. El número de n-gramas crece muy por encima del tamaño del vocabulario, y el índice se vuelve inmanejable.

Secuencias frecuentes maximales

Otro enfoque que también sale de las frecuencias. Una secuencia es frecuente si aparece en al menos cierto número de documentos de la colección, y es **maximal** si no está contenida en ninguna otra secuencia frecuente más larga.

Ejemplo 9.3. Si *united states of america* es frecuente, entonces *united states* también lo es, porque está contenida en ella. Solo se queda la larga.

Esa poda es justo su fortaleza: el índice que forma es muy compacto. La extracción de las MFS se hace normalmente combinando métodos bottom-up con métodos voraces.

Funciona el indexado compuesto?

El veredicto es mixto. Mejora los resultados en niveles bajos de recuerdo, o sea en la cabeza del ranking, los primeros documentos que regresa el sistema. Pero conforme la colección crece, la anotación de entidades nombradas se vuelve el cuello de botella y la ganancia se diluye.

Idea clave

La recomendación de la clase no es reemplazar la bolsa de palabras. Es considerar las frases como términos suplementarios del espacio vectorial, encima de las palabras, no en lugar de ellas.

Tercera propuesta: indexado por sentidos

La tercera idea es ir más allá del empatamiento de términos, hacia el indexado por sentidos. La motivación son la sinonimia y la polisemia de la clase pasada: empatar cadenas de caracteres no arregla ninguna de las dos, porque una es un problema de recall y la otra de precisión. Lo que se busca entonces es capturar conceptos.

Pero qué es un sentido? Es uno de los significados de una palabra. Las palabras tienen más de un significado, y cuál de ellos aplica lo decide el contexto.

10. Desambiguación e indexado conceptual

21 de septiembre de 2026

Resumen rápido. Continuación del indexado por sentidos. Se separó el significado del sentido, se definió la desambiguación del sentido de las palabras y se contrastó con la discriminación. De ahí salieron el proceso de tres pasos, los dos enfoques para resolverlo, WordNet como inventario de sentidos y el algoritmo de Lesk. Al final entró la cuarta propuesta de indexado, la conceptual, con DOR.

Significado y sentido

Se usan casi como sinónimos, pero no son lo mismo:

- El **significado** es lo que está en el diccionario o en el tesauro. Es una lista fija, escrita de antemano, que no depende del texto que se esté procesando.
- El **sentido** lo da el contexto, los términos que rodean a la palabra en ese documento. Es lo que selecciona cuál de los significados aplica.

Indexar así cambia la tabla del índice: una palabra deja de tener una entrada y pasa a tener dos o más, una por cada sentido. *bank* como orilla del río y *bank* como institución financiera pasan a ser términos de indexado distintos, aunque la cadena de caracteres sea la misma.

Idea clave

Ese desdoblamiento es lo que ataca la polisemia: dos documentos que comparten la cadena pero no el sentido dejan de empatar. Y si además los sinónimos se colapsan en una sola entrada, se ataca la sinonimia. Las dos de un golpe, que es justo lo que no logra el empatamiento de cadenas.

Desambiguación y discriminación

La **desambiguación del sentido de las palabras** (WSD, por *word sense disambiguation*) es la tarea de seleccionar el sentido correcto de una palabra a partir de un conjunto predefinido. El inventario de sentidos existe antes de ver el texto.

La **discriminación** está relacionada, pero es diferente. La pregunta aquí es otra: dado un conjunto de textos, qué sentidos distintos surgen en ellos. No hay inventario, y un sistema encaminado a la discriminación tiene que distinguir los sentidos con esa base de documentos. Los sentidos son la salida, no la entrada.

	Desambiguación	**Discriminación**
Inventario	Predefinido y externo, el mismo para toda colección	Se induce de los datos, cambia con la colección
Salida	Una etiqueta de sentido por ocurrencia	Grupos de ocurrencias, sin nombre que los identifique
Método	Clasificación	Agrupamiento
Necesita	Un recurso léxico, o un corpus ya anotado	Solo los documentos de la colección

El proceso de WSD

1. Elegir un inventario de sentidos

Es la decisión que condiciona todo lo demás, porque fija cuáles son los sentidos posibles. Un sentido que no esté en el inventario no se puede asignar nunca, por bueno que sea el método.

Las opciones van de un diccionario o un tesauro a una base como WordNet o a una ontología del dominio. Lo que se negocia al escoger es la **granularidad**:

- Un inventario fino distingue matices, y ahí la tarea se vuelve tan difícil que ni los anotadores humanos coinciden entre sí.
- Uno grueso separa solo los sentidos muy distintos, los homógrafos. Da acuerdo alto, pero captura menos.

Para recuperación conviene el grueso: interesan los sentidos que cambian el tema del documento, no los que cambian el matiz.

2. Diseñar y aplicar el procedimiento

Se decide el alcance y el método. El alcance puede ser de dos tipos:

- **Todas las palabras**. Etiquetar cada palabra de contenido del texto. Es lo que hace falta para indexar una colección entera.
- **Muestra léxica**. Solo unas cuantas palabras objetivo, escogidas de antemano. Es el escenario de los concursos de evaluación.

El método cae en una de dos familias, basada en conocimiento o basada en aprendizaje automático, que van más abajo.

3. Evaluar el desempeño

Hay dos maneras de medir, y no responden la misma pregunta:

- **In vitro**. La exactitud contra un corpus anotado a mano. Mide la tarea aislada.
- **In vivo**. Meter el desambiguador en el SRI y ver si sube el MAP. Es la que importa aquí, porque un desambiguador puede ser bueno y aun así no mover la recuperación.

Y hay dos referencias contra las cuales comparar el número que salga:

- El **piso**: asignar siempre el sentido más frecuente de la palabra. Es sorprendentemente difícil de superar, porque la distribución de sentidos está muy sesgada.
- El **techo**: el acuerdo entre anotadores humanos. Si dos personas solo coinciden el 75 por ciento de las veces, no tiene sentido exigirle 90 al sistema.

Los dos enfoques

Basados en conocimiento

Se apoyan en recursos léxicos ya construidos, diccionarios, tesauros o redes semánticas, y no necesitan datos etiquetados. La información sale del recurso, no de ejemplos.

Cómo deciden:

- **Traslape de glosas**. Comparar la definición de cada sentido con el contexto y quedarse con la que más palabras comparte. Es Lesk, que va abajo.
- **Métodos de grafo**. Armar el subgrafo de WordNet que conecta a los sentidos candidatos de todas las palabras de la oración y quedarse con los nodos mejor conectados.
- **Restricciones de selección**. Un verbo restringe la clase semántica de su objeto: *comer* pide algo comestible, y con eso se descartan sentidos.

A favor tienen la cobertura: alcanzan cualquier palabra que el recurso incluya, sin pagar anotación. En contra, dependen por completo de la calidad y la cobertura de ese recurso.

Basados en aprendizaje automático

Tratan la desambiguación como un problema de clasificación. Cada ocurrencia se convierte en un vector de rasgos tomados de su contexto, y un clasificador predice el sentido.

Los rasgos típicos salen de una ventana alrededor de la palabra objetivo:

- La bolsa de palabras de la ventana, las k palabras de cada lado sin importar el orden.
- Las colocaciones, o sea las palabras exactas en las posiciones -1, $+1$, -2, $+2$, que sí conservan el orden.
- La etiqueta gramatical de los vecinos, y las relaciones sintácticas si hay analizador.

Según cuántos datos etiquetados haya, hay tres variantes:

Variante	Qué necesita	Qué entrega
Supervisada	Un corpus anotado con sentidos, palabra por palabra	La mejor exactitud de las tres
No supervisada	Solo texto crudo	Grupos de ocurrencias, o sea discriminación, no desambiguación
Semisupervisada	Unas cuantas ocurrencias semilla por sentido	Un clasificador que se amplía solo, etiquetando lo que ya reconoce con confianza

La semisupervisada se apoya en dos regularidades del lenguaje que se cumplen casi siempre: un sentido por colocación, la palabra acompañante fija el sentido, y un sentido por discurso, dentro de un mismo documento la palabra conserva el sentido con el que apareció la primera vez.

Ojito...

El cuello de botella del enfoque supervisado es la adquisición de conocimiento: hace falta un corpus anotado por cada palabra que se quiera desambiguar, anotar sentidos a mano es caro y los anotadores no siempre coinciden. Por eso el enfoque basado en conocimiento sigue vivo aunque sea menos exacto.

WordNet

Es una base de datos léxica del inglés organizada en torno a los significados de las palabras y a las relaciones semánticas entre ellos. Incluye sustantivos, verbos, adjetivos y adverbios.

Su unidad no es la palabra. Las palabras con el mismo significado o muy cercano se agrupan en un **synset**, un conjunto de sinónimos, y cada synset representa un concepto particular. Los synsets están conectados por relaciones semánticas.

Relación	Qué liga	Ejemplo
Sinonimia	Las palabras dentro de un mismo synset	*coche*, *automóvil*, *carro*
Hiperonimia	Del caso al tipo general, es-un	*perro* es un *cánido*
Hiponimia	La inversa, del tipo al caso	*cánido* tiene por caso a *perro*
Holonimia	Del todo a la parte, tiene-parte	*coche* tiene por parte a *rueda*
Meronimia	La inversa, de la parte al todo	*rueda* es parte de *coche*
Antonimia	Opuestos, sobre todo entre adjetivos	*caliente* y *frío*

Hiperonimia

Y es hiperónimo de X si todo X es un tipo de Y. Es la relación es-un, y lo importante es que es transitiva: si *perro* es un *cánido* y *cánido* es un *carnívoro*, entonces *perro* es un *carnívoro*.

Esa transitividad es la que organiza los sustantivos de WordNet en una jerarquía con raíz única. Subiendo desde *perro* se llega a la raíz:

$$\textit{perro} \to \textit{cánido} \to \textit{carnívoro} \to \textit{mamífero} \to \textit{vertebrado} \to \textit{animal} \to \textit{organismo} \to \textit{entidad}$$

La relación inversa, bajando, es la hiponimia.

Holonimia

Y es holónimo de X si X es parte de Y. Es la relación parte-de, vista desde el todo; vista desde la parte, la inversa, se llama meronimia. WordNet distingue tres tipos:

- **Componente**. *rueda* es parte de *coche*.
- **Miembro**. *árbol* es miembro de *bosque*.
- **Sustancia**. *oxígeno* es sustancia de *agua*.

Ojito...

A diferencia de la hiperonimia, esta no se puede encadenar sin cuidado. La manija es parte de la puerta y la puerta es parte de la casa, pero decir que la manija es parte de la casa ya suena raro. La transitividad solo se sostiene dentro del mismo tipo de meronimia.

Idea clave

Para indexar, lo que se gana con el synset es directo: los sinónimos se colapsan en una entrada, los sentidos de una polisémica se separan en varias, y la jerarquía de hiperónimos da una forma controlada de expandir la consulta. Si nada empata con *sedán*, se sube un nivel a *coche*. Es la expansión de consultas de la clase del 7 de septiembre, pero con un recurso que dice a dónde subir en vez de adivinarlo de las coocurrencias.

El algoritmo de Lesk

Lesk (1986) es el representante clásico de los métodos basados en conocimiento, y la idea cabe en una frase: el sentido correcto de una palabra es aquel cuya definición comparte más palabras con el contexto en el que la palabra aparece.

Algoritmo 10.1 Lesk simplificado

```
1: procedimiento LESK(w, contexto)
2:     mejor ← sentido más frecuente de w
3:     mayor ← 0
4:     para cada sentido s de w en el inventario hacer
5:         firma ← palabras de la glosa y los ejemplos de s
6:         traslape ← |firma ∩ contexto|
7:         si traslape > mayor entonces
8:             mayor ← traslape
9:             mejor ← s
10:        fin si
11:    fin para
12:    devolver mejor
13: fin procedimiento
```

La versión original de Lesk no compara la glosa contra el contexto, compara las glosas entre sí: toma todas las palabras de contenido de la ventana, considera todas las combinaciones de un sentido por palabra y se queda con la combinación que maximiza el traslape entre las glosas elegidas. Desambigua todas las palabras a la vez, y es mucho más cara.

Ejemplo 10.1. El caso clásico es la frase *pine cone*. Las glosas del diccionario son:

pine

1. kinds of evergreen tree with needle-shaped leaves
2. waste away through sorrow or illness

cone

1. solid body which narrows to a point
2. something of this shape whether solid or hollow
3. fruit of certain evergreen trees

Con dos sentidos de *pine* y tres de *cone* hay seis combinaciones que revisar:

Combinación	Palabras en común	Traslape
pine 1, *cone* 1	ninguna	0
pine 1, *cone* 2	ninguna	0
pine 1, *cone* 3	*evergreen*, *tree*	2
pine 2, *cone* 1	ninguna	0
pine 2, *cone* 2	ninguna	0
pine 2, *cone* 3	ninguna	0

Gana la pareja *pine* 1 con *cone* 3: el árbol y su fruto, que es la lectura correcta. Las dos palabras quedaron desambiguadas al mismo tiempo.

Desventajas

Son dos, y las dos se ven en el ejemplo:

1. **Son muchas combinaciones a analizar.** Con dos palabras fueron seis. Con una ventana de k palabras de contenido donde cada una tiene s sentidos en promedio, son s^k combinaciones. Una oración de diez palabras de contenido con tres sentidos cada una ya da $3^{10} = 59\,049$ combinaciones que evaluar. Por eso en la práctica se usa la versión simplificada, que es lineal en el número de sentidos de la palabra objetivo.

2. **Puede no haber suficiente traslape entre las definiciones.** Las glosas son cortas, una docena de palabras, y es muy común que dos definiciones correctas no compartan ni una. Cuando todos los traslapes dan cero el algoritmo no tiene con qué decidir y se cae al sentido más frecuente.

Los dos parches habituales para lo segundo: expandir la firma de cada sentido con las glosas de los synsets vecinos en WordNet, hiperónimos e hipónimos incluidos, y pesar el traslape con idf para que las palabras comunes no dominen la cuenta.

Idea clave

La desambiguación es benéfica cuando las consultas son muy cortas. Dos o tres palabras no alcanzan para desambiguarse entre sí, y es justo ahí donde el sentido equivocado hace más daño. En una consulta larga los demás términos ya desambiguan solos y el desambiguador aporta poco. Con todo, la desambiguación sigue siendo un problema actual, no resuelto.

Ojito...

Un desambiguador imperfecto puede dejar la recuperación peor de como estaba. Sanderson (1994) midió eso: por debajo de una exactitud cercana al 90 por ciento, los errores de sentido meten más ruido del que quitan y conviene no desambiguar. Es el mismo intercambio del etiquetado gramatical de la clase pasada, lo que se gana en precisión se paga en recuerdo.

Cuarta propuesta: indexado conceptual

La formulación del problema es que la bolsa de palabras ignora toda la información semántica, trabaja solo con las formas superficiales de las palabras. Esta cuarta representación busca el tipo conceptual.

El concepto está relacionado con el sentido, pero desde un punto de vista práctico: no hace falta un inventario escrito por lexicógrafos ni un desambiguador. La idea es representar los documentos en función de las palabras que ocurren en ellos, y que la semántica del documento se obtenga a partir de eso.

Idea clave

Es un cambio de dónde viene el significado. En la propuesta anterior lo aportaba un recurso externo, WordNet. Aquí lo aporta la colección misma: dos palabras significan algo parecido si aparecen en los mismos documentos. Es la semántica distribucional de la clase del 7 de septiembre, llevada al indexado.

DOR

La primera propuesta es **DOR**, representación por ocurrencia en documentos (*document occurrence representation*).

El giro está en qué se toma como concepto: cada documento de la colección es una dimensión, un concepto primitivo. Una palabra se describe entonces por los documentos en los que ocurre, y dos palabras se parecen si ocurren en los mismos.

Ojito...

DOR es una representación de palabras, no de documentos. Lo que construye directamente es un vector por cada término del vocabulario. La representación del documento viene después, y es derivada.

El peso de la dimensión d_j en la representación del término t_i se calcula como el dual de tf-idf, intercambiando los papeles de término y documento:

$$w_{ij} = \mathrm{tf}_{i,j} \cdot \log \frac{|T|}{|T_j|},$$

donde $|T|$ es el tamaño del vocabulario y $|T_j|$ el número de términos distintos que contiene d_j. La lectura es tf-idf al revés: un documento que toca muchísimos términos distintos describe mal cualquier concepto, igual que un término que aparece en muchísimos documentos discrimina mal.

De palabras a documentos

Con los vectores de palabra ya construidos, el documento se arma sumando: la representación de un documento es la suma de los vectores de las palabras que contiene. Las consultas se representan igual, sumando los vectores de sus palabras.

El efecto sobre la matriz es el que importa. La matriz palabra-documento, de $m \times n$, se convierte en una matriz documento-documento, de $n \times n$, y esa segunda es la que se usa como índice para la recuperación.

Palabra-documento
representación de palabras

	d_1	d_2	$\cdots$	d_n
t_1				
t_2				
$\vdots$		w_{ij}		
t_m				

suma →

Documento-documento
índice para recuperación

	d_1	d_2	$\cdots$	d_n
d_1				
d_2				
$\vdots$		w_{ij}		
d_n				

Ejemplo 10.2. Una colección de cuatro documentos, dos de fútbol y dos de medicina, con pesos crudos para que la cuenta se siga a mano: d_1 contiene *gol* y *equipo*, d_2 *gol* y *árbitro*, d_3 *virus* y *fiebre*, y d_4 *virus* y *paciente*.

Cada renglón de la matriz palabra-documento es el vector DOR de una palabra:

	d_1	d_2	d_3	d_4
gol	1	1	0	0
equipo	1	0	0	0
árbitro	0	1	0	0
virus	0	0	1	1
fiebre	0	0	1	0
paciente	0	0	0	1

Sumando los vectores de las palabras de cada documento sale la matriz documento-documento:

$$d_1 = (1,1,0,0) + (1,0,0,0) = (2,1,0,0)$$
$$d_2 = (1,1,0,0) + (0,1,0,0) = (1,2,0,0)$$
$$d_3 = (0,0,1,1) + (0,0,1,0) = (0,0,2,1)$$
$$d_4 = (0,0,1,1) + (0,0,0,1) = (0,0,1,2)$$

Queda diagonal por bloques: los dos documentos de fútbol se parecen entre sí y no se parecen nada a los de medicina, y nadie tuvo que declarar cuáles eran los temas.

Ahora la consulta *equipo*, que se representa con el vector de esa palabra, $(1,0,0,0)$. Los cosenos contra los cuatro documentos son

$$\cos(q,d_1) = \frac{2}{\sqrt{5}} = 0.894, \qquad \cos(q,d_2) = \frac{1}{\sqrt{5}} = 0.447,$$

y cero contra d_3 y d_4.

Idea clave

Lo interesante del ejemplo está en d_2: la palabra *equipo* no aparece en d_2. Con la bolsa de palabras su similitud sería exactamente cero y ese documento no se recuperaría nunca. DOR lo trae porque *equipo* y *gol* coocurren en d_1, y *gol* sí está en d_2. La conexión que en la propuesta anterior tenía que poner el diccionario, aquí la puso la colección.

Ojito...

El costo es la dimensionalidad. El vector de una palabra tiene tantas componentes como documentos haya, así que la representación crece con la colección y no con el vocabulario. Y al sumar, los documentos dejan de ser dispersos: la matriz documento-documento se llena. Para una colección grande eso pesa más que el índice invertido de siempre.

11. Representaciones conceptuales

23 de septiembre de 2026

Resumen rápido. Continuación del indexado conceptual. Después de DOR vino su pareja, TCOR, que describe cada palabra por las palabras con las que coocurre. Luego random indexing, una forma barata de construir bolsas de conceptos, qué tanto funcionan en IR y la representación más usada, LSI.

TCOR

En WSD una palabra se representa por los términos que ocurren en su contexto: el significado de una palabra lo transmiten las palabras que suelen aparecer junto a ella. Esa es la base de **TCOR**, representación por coocurrencia de términos (*term co-occurrence representation*).

Cambia el concepto primitivo respecto a DOR. Allá cada documento era una dimensión; aquí cada palabra del vocabulario lo es. La matriz es palabra-palabra, de $m \times m$, y la celda w_{ij} pesa la coocurrencia de t_i con t_j.

	t_1	t_2	$\cdots$	t_m
t_1				
t_2				
$\vdots$		w_{ij}		
t_m				

Las dos intuiciones sobre los pesos son las mismas de tf-idf, con palabras en lugar de documentos:

- Entre más contextos compartan t_i y t_j, más importa t_j para caracterizar la semántica de t_i. Es el tf.
- Entre más palabras distintas acompañen a t_j, menos aporta a caracterizar a t_i. Es el idf: una palabra que aparece con todo no dice nada de nadie.

Juntas dan el peso

$$w_{ij} = \mathrm{tf}_{i,j} \cdot \log \frac{|T|}{|T_j|},$$

donde $\mathrm{tf}_{i,j}$ es el número de contextos en que coocurren t_i y t_j, y $|T_j|$ el número de términos distintos con los que coocurre t_j.

Igual que DOR, TCOR es una representación de palabras. El documento se obtiene sumando los vectores de sus palabras, y la consulta igual. La matriz palabra-palabra se vuelve una matriz documento-palabra, de $n \times m$, que es el índice para recuperar.

Ejemplo 11.1. La misma colección de fútbol y medicina de DOR, con conteos crudos y el documento como contexto. Cada palabra coocurre consigo misma una vez, por eso la diagonal vale 1. Los vectores de las palabras de fútbol, sobre las dimensiones (*gol*, *equipo*, *árbitro*, *virus*, *fiebre*, *paciente*), son

$$\begin{aligned} \textit{gol} &= (1,1,1,0,0,0) \\ \textit{equipo} &= (1,1,0,0,0,0) \\ \textit{árbitro} &= (1,0,1,0,0,0) \end{aligned}$$

y sumando salen los documentos:

$$\begin{aligned} d_1 &= \textit{gol} + \textit{equipo} = (2,2,1,0,0,0) \\ d_2 &= \textit{gol} + \textit{árbitro} = (2,1,2,0,0,0) \end{aligned}$$

La consulta *equipo* es $q = (1, 1, 0, 0, 0, 0)$, y

$$\cos(q, d_1) = \frac{4}{\sqrt{2} \cdot 3} = 0.943, \qquad \cos(q, d_2) = \frac{3}{\sqrt{2} \cdot 3} = 0.707.$$

Otra vez d_2 se recupera sin contener *equipo*, ahora porque *equipo* y *gol* coocurren. La diferencia con DOR es el tamaño: el vector tiene una componente por palabra del vocabulario, no por documento, así que no crece con la colección.

Otras representaciones de bolsa de conceptos

La bolsa de palabras estándar casi nunca se usa tal cual. Antes se refina de una de dos formas:

- **Selección de características:** se quitan palabras con base en medidas estadísticas, como la frecuencia en documentos o la ganancia de información.
- **Extracción de características:** se crean características artificiales a partir de las originales, con agrupamiento distribucional de palabras o con métodos de análisis factorial, como LSI.

El problema de las dos es que son caras. Random indexing es una forma sencilla de generar representaciones de bolsa de conceptos (BoC).

Random indexing

Es una metodología de espacio vectorial que acumula vectores de contexto para las palabras a partir de la coocurrencia. Son tres pasos.

1. Vectores índice

A cada contexto, que puede ser un documento, un párrafo o una oración, se le asigna un **vector índice** aleatorio y único de dimensión $k \ll n$. Es casi todo ceros, con unas pocas entradas en $+1$ y -1 en posiciones al azar.

Idea clave

El truco es que en dimensión alta los vectores aleatorios dispersos son casi ortogonales entre sí. Se comportan como si cada contexto tuviera su propia dimensión, que es lo que hace DOR, pero en k componentes en lugar de n. La reducción de dimensión sale gratis, sin factorizar ninguna matriz.

2. Vectores de contexto

Se recorre el texto y cada vez que una palabra aparece en un contexto se le suma el vector índice de ese contexto. El vector de contexto de la palabra es la suma de los vectores índice de todos los contextos donde aparece.

Ejemplo 11.2. Dos documentos con $k = 10$:

$$D_1 : \text{Towards an Automata Theory of Brain} \qquad r_1 = (0, 1, 0, 0, 0, -1, 0, 0, 0, 0)$$
$$D_2 : \text{From Automata Theory to Brain Theory} \qquad r_2 = (1, 0, 0, 0, 0, 0, 0, -1, 0, 0)$$

Brain aparece en los dos, así que su vector de contexto es

$$\text{CV}_{brain} = r_1 + r_2 = (1, 1, 0, 0, 0, -1, 0, -1, 0, 0).$$

Automata y *Theory* también aparecen en los dos y reciben el mismo vector. Con dos documentos no hay más información que esa: las tres palabras ocurren exactamente en los mismos contextos.

3. Vectores de documento

El documento se representa con la suma ponderada de los vectores de contexto de sus palabras, con el idf como peso. Para d_i: *From Automata Theory to Brain Theory*,

$$d_i = a_1\mathrm{CV}_1 + a_2\mathrm{CV}_2 + a_3\mathrm{CV}_3 + a_2\mathrm{CV}_2,$$

con CV_1, CV_2 y CV_3 los vectores de *Automata*, *Theory* y *Brain*, y a_1, a_2, a_3 sus idf. *Theory* aparece dos veces, por eso su término se suma dos veces.

Funcionan las representaciones conceptuales?

- Resuelven varios problemas de emparejamiento conceptual: capturan información de relación entre palabras, incluida la causal, la orientada a metas y la taxonómica.
- En IR hay poco trabajo. Los experimentos muestran que TCOR, DOR y random indexing superan al VSM tradicional; en las colecciones de CLEF la mejora ronda el 7 por ciento.
- El enfoque más usado es el de indexado semántico latente, LSI, pero es computacionalmente caro.

Indexado semántico latente (LSI)

El concepto clave es la **coocurrencia**: dos o más términos que aparecen en los mismos documentos más seguido de lo que daría el azar.

	t_1	t_2	t_3	t_4
Consulta	user	interface		
Documento 1	user	interface	HCI	interaction
Documento 2			HCI	interaction

El documento 2 no comparte ni un término con la consulta y su coseno es cero. Intuitivamente también es un buen candidato: el documento 1 muestra que *user* e *interface* van con *HCI* e *interaction*.

LSI proyecta consultas y documentos a un espacio con dimensiones semánticas "latentes":

- Los términos que coocurren van a las mismas dimensiones, así que dos documentos pueden tener coseno alto sin compartir ningún término.
- Los que no coocurren van a dimensiones distintas.
- El espacio latente tiene menos dimensiones que el original: es una técnica de reducción de dimensión.

La proyección sale de la descomposición en valores singulares de la matriz término-documento A, de $m \times n$, truncada a los k valores singulares más grandes:

$$A \approx A_k = U_k \Sigma_k V_k^T.$$

Las columnas de U_k son las dimensiones latentes, y las de $\Sigma_k V_k^T$ los documentos en ese espacio. Una consulta nueva se proyecta con $\hat{q} = \Sigma_k^{-1} U_k^T q$.

Ejemplo 11.3. La tabla de arriba, con términos en los renglones (*user*, *interface*, *HCI*, *interaction*) y los dos documentos en las columnas:

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad A^T A = \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix}.$$

Los valores propios de $A^T A$ son $3 \pm \sqrt{5}$, así que $\sigma_1 = 2.288$ y $\sigma_2 = 0.874$. Quedándose con $k = 1$, con $v_1 = (0.851, 0.526)$,

$$A_1 = \sigma_1 u_1 v_1^T = \begin{pmatrix} 0.724 & 0.447 \\ 0.724 & 0.447 \\ 1.171 & 0.724 \\ 1.171 & 0.724 \end{pmatrix}.$$

El documento 2 ahora tiene peso 0.447 en *user* y en *interface*, términos que no contiene. Con la consulta $q = (1, 1, 0, 0)$ el coseno contra el documento 2 pasa de 0 a 0.53.

Ojito...

La similitud nueva la crea el truncamiento, no la SVD. Con $k = 2$ no se pierde nada, $A_2 = A$ y el documento 2 vuelve a coseno cero. El k controla cuánto se mezclan los términos: muy chico y todo se parece a todo, como aquí donde los dos documentos quedan con el mismo coseno. El costo es la SVD de una matriz de vocabulario por colección, y por eso LSI es caro.

12. Descomposición en valores singulares

28 de septiembre de 2026

Resumen rápido. La maquinaria que hay detrás de LSI. Se abrió la descomposición en valores singulares: qué contiene cada una de las tres matrices, qué significa quedarse con las primeras k columnas y por qué esa truncada es la mejor aproximación posible. El ejemplo de cinco términos reducidos a dos dimensiones se siguió con números, y la clase cerró con las ventajas y las desventajas del método.

La factorización

LSI es la aplicación de la SVD a la matriz término-documento. Lo que hace la descomposición es proyectar un espacio de n dimensiones sobre uno de k, con $n > k$.

La matriz A, de t términos por d documentos, se parte en el producto de tres:

$$A_{t\times d} = T_{t\times n}\, S_{n\times n}\, (D_{d\times n})^T,$$

donde n es el rango de A, a lo más mín(t, d).

Matriz	Forma	Qué contiene
T	$t \times n$	Los términos en el espacio latente, una columna por dimensión. Sus columnas son ortonormales
S	$n \times n$	Diagonal, con los valores singulares de A en orden decreciente
D	$d \times n$	Los documentos en el espacio latente, también con columnas ortonormales

Observación. Es la misma factorización que se escribió como $A = U\Sigma V^T$ en la clase del 23 de septiembre. Solo cambian los nombres: $T = U$, $S = \Sigma$ y $D = V$. La notación con letras es la de las diapositivas, y tiene la ventaja de que se recuerda cuál es la de términos y cuál la de documentos.

Qué contiene S

El i-ésimo valor singular dice cuánta variación hay a lo largo del i-ésimo eje. El primero recoge la dirección en la que la colección se dispersa más, el segundo la siguiente, y así hasta el último.

Que vengan ordenados es lo que hace posible el truncamiento: las dimensiones no valen lo mismo, y la descomposición las entrega de la más informativa a la menos sin que nadie tenga que decidirlo.

Una identidad que sirve para verificar una SVD a mano:

$$\sum_{i=1}^{n} \sigma_i^2 = \|A\|_F^2 = \sum_{i,j} a_{ij}^2.$$

De cinco a dos dimensiones

Ejemplo 12.1. Cinco términos y seis documentos:

$A =$

	d_1	d_2	d_3	d_4	d_5	d_6
cosmonaut	1	0	1	0	0	0
astronaut	0	1	0	0	0	0
moon	1	1	0	0	0	0
car	1	0	0	1	1	0
truck	0	0	0	1	0	1

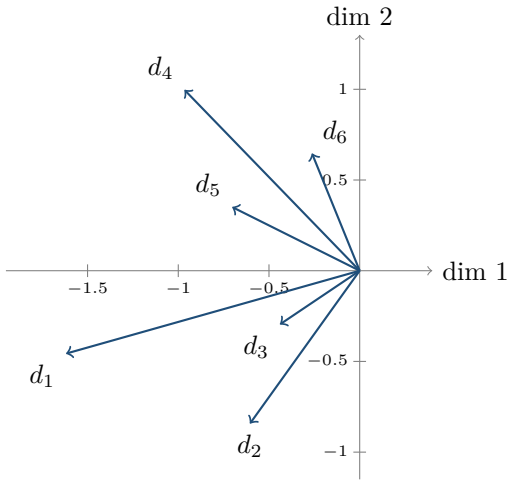
Su descomposición da cinco valores singulares:

$$S = \mathrm{diag}(2.16,\ 1.59,\ 1.28,\ 1.00,\ 0.39).$$

La identidad de arriba cuadra: la suma de sus cuadrados es 10, que es justo el número de unos de A.

Los seis documentos proyectados a las dos primeras dimensiones, o sea las columnas de $S_2 D_2^T$:

	d_1	d_2	d_3	d_4	d_5	d_6
dim 1	−1.62	−0.61	−0.44	−0.97	−0.70	−0.26
dim 2	−0.46	−0.84	−0.30	+1.00	+0.35	+0.65



La figura se parte sola en dos grupos. Abajo quedan d_1, d_2 y d_3, que son los del espacio; arriba d_4, d_5 y d_6, que son los de vehículos. Nadie declaró los dos temas.

Y ahí está el punto: d_2 contiene *astronaut* y *moon*, y d_3 solo *cosmonaut*. No comparten ni un término, así que en el espacio original su coseno es exactamente cero. En el de dos dimensiones su coseno es 0.94.

Ojito...

Los signos de una SVD son arbitrarios. Si se le cambia el signo a una columna de T y a la misma columna de D, el producto no se entera. Aquí el eje 1 va con el signo de las diapositivas, con los seis documentos del lado negativo.

El truncamiento

Restringiendo T, S y D a sus primeras k columnas, con $k < n$, quedan $T_{t\times k}$, $S_{k\times k}$ y $(D_{d\times k})^T$. Su producto

$$\hat{A} = T_k\, S_k\, D_k^T$$

es la mejor aproximación por mínimos cuadrados de A mediante una matriz de rango k. Mejor quiere decir que, entre todas las B de rango a lo más k, $\hat{A}$ es la que minimiza $\|A - B\|_F$.

El error que se comete se lee directo de los valores singulares que se tiraron:

$$\left\|A - \hat{A}\right\|_F = \sqrt{\sum_{i=k+1}^{n} \sigma_i^2}.$$

Ejemplo 12.2. Con $k = 2$ en el ejemplo de arriba se tiran 1.28, 1.00 y 0.39, así que el error es $\sqrt{1.28^2 + 1.00^2 + 0.39^2} = 1.67$, y la fracción de la energía que se conserva es $(2.16^2 + 1.59^2)/10 = 0.72$.

La matriz reconstruida llena huecos. En $\hat{A}$, el término *cosmonaut* tiene peso 0.52 en d_2, un documento donde esa palabra nunca aparece. Eso es lo que hace que el coseno deje de ser cero.

Ojito...

En esa misma reconstrucción aparecen pesos negativos, por ejemplo -0.39 para *truck* en d_2. Un peso negativo no significa nada como frecuencia de un término. Es el primer aviso de la segunda desventaja del método, más abajo.

Qué valor de k

Los valores que se usan con frecuencia son 100 o 150, y pueden ser más altos. No hay regla cerrada: k se escoge probando, y el criterio práctico es la fracción de energía retenida, o directamente el MAP sobre un conjunto de consultas de prueba.

Ventajas

- **Marco formal fuerte y completamente automático.** No hace falta ningún recurso externo, ni tesauro, ni diccionario, ni corpus anotado. Todo sale de la matriz y de un teorema de álgebra lineal.
- **No requiere truncamiento, y tolera errores de escritura.** *connect*, *connected* y *connection* aparecen en los mismos documentos, así que la SVD las manda a las mismas dimensiones sin que nadie corra a Porter. Con una palabra mal escrita pasa lo mismo: la rodean los mismos vecinos que a la forma correcta, y cae junto a ella.
- **Sirve para recuperación multilingüe.** Va abajo.
- **Mejora el recuerdo, lo que se llama IR conceptual.** Se pueden recuperar documentos relevantes que no contienen ni un término de la consulta. Es exactamente lo que hizo d_2 con d_3 en el ejemplo.

Ojito...

Lo de los errores de escritura tiene un límite que conviene tener claro: funciona si la palabra mal escrita aparece lo suficiente como para tener un contexto estable. Un dedazo que ocurre una sola vez no le da a la SVD de dónde agarrarse para colocarlo.

Por qué sirve para multilingüe

El truco no está en la descomposición, está en cómo se arma la matriz antes de descomponerla.

Se necesita una colección paralela o comparable, donde cada documento exista en los dos idiomas. Se pega la versión de un idioma con la versión del otro y el par se trata como **un solo documento**.

Con eso, los renglones de A traen términos de los dos idiomas a la vez, y dos palabras que son traducción una de la otra coocurren en todos esos documentos dobles. La SVD las manda a las mismas dimensiones latentes, igual que haría con dos sinónimos del mismo idioma. Para el método no hay diferencia entre sinonimia y traducción: las dos son palabras distintas que aparecen siempre juntas.

Ya entrenado el espacio, la consulta se escribe en un idioma, se proyecta, y recupera documentos escritos en el otro. Al momento de consultar no interviene ningún diccionario bilingüe ni ninguna traducción automática.

Ojito...

Lo que sí hace falta es el corpus paralelo para entrenar, y eso no es gratis. LSI no descubre la correspondencia entre dos idiomas de la nada: la aprende de documentos que alguien ya alineó. Lo que ahorra es el diccionario en tiempo de consulta, no el trabajo de conseguir los datos.

Desventajas

1. **El cálculo es caro.** La SVD de una matriz densa de $m \times n$ cuesta del orden de $\text{mín}(m,n)^2 \cdot \text{máx}(m,n)$, y el vocabulario por la colección da matrices enormes. Hay dos agravantes: $\hat{A}$ es densa aunque A fuera dispersa, así que el índice crece en vez de encogerse, y la descomposición no es incremental, un documento nuevo obliga a recalcular o a proyectarlo de forma aproximada.

2. **Los métodos basados en la distribución normal continua no son apropiados para datos de conteo.** Minimizar la norma de Frobenius es mínimos cuadrados, y eso supone por debajo que los errores son normales y de varianza constante. Las celdas de A son conteos: enteros no negativos, muy sesgados, y con varianza que crece con la media. De ahí salen los pesos negativos del ejemplo, y de ahí vienen pLSA y LDA, que modelan los conteos con distribuciones discretas en lugar de forzar una normal.

3. **Muchas veces importa más subir la precisión,** y para eso hacen falta desambiguar la consulta y desambiguar los sentidos. LSI ayuda al recuerdo, pero un término polisémico recibe un solo vector que promedia todos sus sentidos. *bank* de río y *bank* de dinero quedan en un único punto intermedio, y documentos de los dos sentidos se mezclan.

Idea clave

Esa última desventaja es el espejo de la clase del 21 de septiembre. LSI colapsa sinónimos muy bien, que era el problema de recuerdo, y trata mal la polisemia, que era el de precisión. La desambiguación hacía justo lo contrario.

Idea clave

Vale la pena ver cómo encaja con las dos clases anteriores. DOR y TCOR arman el espacio conceptual a mano: uno toma los documentos como dimensiones, el otro los términos. LSI no lo decide, lo deduce de los datos.
Y la relación es literal. De $A = TSD^T$, usando que las columnas de T y de D son ortonormales, sale

$$AA^T = TS^2T^T, \qquad A^TA = DS^2D^T.$$

O sea que T diagonaliza la matriz término-término y D la documento-documento, que son las dos que aparecieron en las clases del 21 y del 23. LSI es la versión espectral de ambas.

13. Semántica distribucional y *word embeddings* *30 de septiembre de 2026*

Resumen rápido. Inicio de las representaciones basadas en redes neuronales. Se partió de la hipótesis distribucional: el significado de una palabra está en cómo se usa. De ahí salen los vectores semánticos, primero contando (matriz término-documento, matriz término-término, one-hot, SVD truncada) y después prediciendo, con un modelo de lenguaje neuronal que aprende los vectores como efecto secundario. A esos vectores densos y de baja dimensión se les llama *word embeddings*.

Semántica distribucional

Hasta ahora una palabra era un índice: la posición i del vocabulario, o la dimensión i de un vector de tf-idf. Eso sirve para contar, pero no dice nada de lo que la palabra significa. Para la máquina, *gato* y *perro* están tan lejos como *gato* y *hipoteca*.

La idea que resuelve esto viene de la lingüística de los años cincuenta, y se conoce como la **hipótesis distribucional**:

Definición 13.1 (Hipótesis distribucional)**.** El significado de una palabra está en el uso que se le da en el lenguaje. Las palabras que ocurren en contextos similares tienden a tener significados similares.

La frase clásica es de Firth (1957): *"you shall know a word by the company it keeps"*, a una palabra se la conoce por las compañías que frecuenta. No hace falta un diccionario que diga que *ocelote* es un felino: si aparece rodeado de *selva*, *manchas*, *cazar* y *presa*, igual que *jaguar*, algo tienen en común.

Ejemplo 13.2. Nadie sabe qué es *tesgüino*. Pero con estas oraciones:

- Una botella de *tesgüino* está sobre la mesa.
- A todos les gusta el *tesgüino*.
- El *tesgüino* te pone borracho.
- El *tesgüino* se hace con maíz.

queda bastante claro que es una bebida alcohólica de maíz. Todo lo que se sabe salió del contexto. Eso es exactamente lo que se le pide a la máquina que haga.

El contexto captura varios tipos de relación a la vez:

- **Palabras relacionadas.** No necesariamente sinónimos: *café* y *taza* no significan lo mismo, pero están relacionadas y aparecen juntas.
- **Campo semántico.** Conjuntos de palabras que cubren un mismo dominio: *mesero*, *menú*, *plato*, *cuenta* y *propina* pertenecen al de restaurante.
- **Connotaciones.** El tono afectivo: *feliz* y *triste* son opuestos, pero los dos son palabras de emoción. *Tacaño* y *ahorrador* describen lo mismo con valoración distinta.

Idea clave

Esto no es nuevo en el curso. TCOR, de la clase del 23 de septiembre, ya describía una palabra por las palabras con las que coocurre, y LSI juntaba términos que aparecen en los mismos documentos. Todo eso era semántica distribucional sin decirle así. Lo nuevo de hoy es darle nombre y llevarla a redes neuronales.

El vector semántico

Para representar el significado se usa un vector: el **vector semántico**. Cada palabra queda asociada a un punto en un espacio de muchas dimensiones, y con esos vectores se representan después los textos.

Dos palabras se comparan midiendo qué tan cerca están sus vectores, y la medida de siempre es el coseno:

$$\cos(v, w) = \frac{v \cdot w}{\|v\| \, \|w\|} = \frac{\sum_i v_i w_i}{\sqrt{\sum_i v_i^2} \sqrt{\sum_i w_i^2}}.$$

Vale 1 si los vectores apuntan igual y 0 si son perpendiculares. Con conteos, que nunca son negativos, se queda entre 0 y 1.

Ojito...

Se usa el coseno y no la distancia euclidiana porque las palabras frecuentes tienen vectores largos. *good* aparece mucho más que *wit*, y su vector es enorme aunque signifique cosas parecidas. El coseno solo mira la dirección, no el tamaño, y así compara el patrón de uso sin castigar a las palabras raras.

La pregunta de la clase es cómo se construyen esos vectores. Hay dos familias, que se ven en orden: los que **cuentan** y los que **predicen**.

Matriz término-documento

Es la matriz del modelo vectorial de las primeras clases: cada renglón es una palabra del vocabulario y cada columna un documento de la colección. La celda cuenta cuántas veces aparece la palabra en el documento.

Ejemplo 13.3 (Cuatro palabras en cuatro obras de Shakespeare)**.**

	As You Like It	*Twelfth Night*	*Julius Caesar*	*Henry V*
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

La misma tabla se lee de dos maneras:

- **Por columnas:** cada documento es un vector de longitud $|V|$. Es lo que se hizo en IR, donde se comparaban documentos contra consultas. Las comedias (*As You Like It*, *Twelfth Night*) tienen mucho *fool* y *wit*; las históricas (*Julius Caesar*, *Henry V*) tienen más *battle*.
- **Por renglones:** cada palabra es un vector de longitud igual al número de documentos. Esta es la lectura nueva: la palabra queda descrita por **los documentos en los que tiende a aparecer**.

Ejemplo 13.4. Con los renglones como vectores, *fool* $= (36, 58, 1, 4)$, *wit* $= (20, 15, 2, 3)$ y *battle* $= (1, 0, 7, 13)$.

***fool* contra *wit*.**

$$\begin{aligned} \mathit{fool} \cdot \mathit{wit} &= 36 \cdot 20 + 58 \cdot 15 + 1 \cdot 2 + 4 \cdot 3 = 720 + 870 + 2 + 12 = 1604, \\ \|\mathit{fool}\| &= \sqrt{1296 + 3364 + 1 + 16} = \sqrt{4677} = 68.39, \\ \|\mathit{wit}\| &= \sqrt{400 + 225 + 4 + 9} = \sqrt{638} = 25.26, \\ \cos(\mathit{fool}, \mathit{wit}) &= \frac{1604}{68.39 \cdot 25.26} = 0.93. \end{aligned}$$

***fool* contra *battle*.**

$$\mathit{fool} \cdot \mathit{battle} = 36 + 0 + 7 + 52 = 95, \quad \|\mathit{battle}\| = \sqrt{219} = 14.80, \quad \cos = \frac{95}{68.39 \cdot 14.80} = 0.09.$$

fool y *wit* se parecen porque viven en las mismas obras; *fool* y *battle* casi nada. Nadie le dijo al método qué es una comedia.

Ojito...

El documento es un contexto muy grueso. Dos palabras en la misma obra pueden estar a cincuenta páginas una de otra y cuentan igual que si estuvieran juntas. Además, el vector de cada palabra tiene tantas componentes como documentos haya, que es el mismo problema de dimensionalidad que tenía DOR.

Matriz término-término

Para afinar el contexto, en lugar de documentos se usan las palabras de alrededor. La matriz es de $|V| \times |V|$: los renglones son las palabras objetivo, las columnas las palabras de contexto, y cada celda cuenta cuántas veces la palabra del renglón y la de la columna coocurren en el corpus de entrenamiento.

Coocurrir quiere decir caer dentro de una **ventana**. Con ventana de tamaño L, el contexto de una palabra son las L palabras a su izquierda y las L a su derecha.

Ejemplo 13.5 (Ventana de tamaño 2)**.** En *"... I saw a cute grey **cat** playing in the garden ..."*, los contextos de *cat* son

$$\underbrace{\mathit{cute} \quad \mathit{grey}}_{\text{izquierda}} \quad \textbf{cat} \quad \underbrace{\mathit{playing} \quad \mathit{in}}_{\text{derecha}}$$

Así que esta aparición suma 1 a las celdas (*cat*, *cute*), (*cat*, *grey*), (*cat*, *playing*) y (*cat*, *in*). Recorriendo todo el corpus, la celda queda en

$$N(w, c) = \text{número de veces que la palabra } w \text{ aparece en el contexto } c.$$

Ejemplo 13.6 (Ventana de tamaño 4 sobre Wikipedia)**.** Fragmentos del corpus:

is traditionally followed by	**cherry**	pie, a traditional dessert
often mixed, such as	**strawberry**	rhubarb pie. Apple pie
computer peripherals and personal	**digital**	assistants. These devices usually
a computer. This includes	**information**	available on the internet

y una parte de la matriz resultante:

	aardvark	⋯	*computer*	*data*	*result*	*pie*	*sugar*
cherry	0	⋯	2	8	9	442	25
strawberry	0	⋯	0	0	1	60	19
digital	0	⋯	1670	1683	85	5	4
information	0	⋯	3325	3982	378	5	13

Se ve a ojo: *cherry* y *strawberry* tienen su masa en *pie* y *sugar*; *digital* e *information*, en *computer* y *data*. Con los seis contextos visibles, los cosenos dan

Par	**Coseno**
digital, *information*	0.996
cherry, *strawberry*	0.969
cherry, *digital*	0.019
strawberry, *information*	0.003

Observación. Es la matriz de TCOR. Allá el contexto era el documento; aquí es una ventana, que es más fino. El tamaño de la ventana cambia qué tipo de similitud se captura: con ventanas chicas, de ± 1 a ± 3, las palabras cercanas son del mismo tipo gramatical y salen parecidas las intercambiables (*perro*, *gato*); con ventanas grandes sale más la relación de tema (*perro*, *correa*, *veterinario*).

Ojito...

La matriz tiene $|V|^2$ celdas. Con un vocabulario de 50 000 palabras son 2 500 millones, y casi todas valen cero: la columna *aardvark* del ejemplo es un buen retrato. Los vectores son largos y dispersos, y eso es lo que motiva todo lo que sigue.

Otras formas de generar vectores de palabras

- **Random indexing.** De la clase del 23 de septiembre: en lugar de una dimensión por contexto, cada contexto recibe un vector aleatorio corto, y la palabra acumula los de sus contextos. Reduce dimensión sin factorizar nada.
- **Latent Semantic Analysis (LSA).** Es LSI, de las clases del 23 y 28 de septiembre, cuando se usa para representar palabras y no para recuperar documentos. Se aplica SVD truncada a la matriz de conteos.
- **Latent Dirichlet Allocation (LDA).** Un modelo probabilístico de temas: cada documento es una mezcla de temas y cada tema una distribución sobre palabras. La palabra se puede representar por qué tanto pertenece a cada tema. Es la respuesta a la segunda desventaja de LSI, la de forzar una normal sobre conteos.
- **One-hot encoding.** La más simple de todas, abajo.

One-hot encoding

Cada palabra es un vector de longitud $|V|$ con un 1 en su posición y ceros en todo lo demás.

Ejemplo 13.7. Con $V = \{home, cat, dog, song, car\}$:

home	0	0	0	0	1
cat	0	0	0	1	0
dog	0	0	1	0	0
song	0	1	0	0	0
car	1	0	0	0	0

Las preguntas de la diapositiva: ¿qué tan parecidas son *home* y *cat*? ¿Y *cat* y *dog*? ¿Y *cat* y *car*? El producto punto de dos vectores one-hot distintos es siempre 0, porque el 1 de uno cae sobre un 0 del otro. Las tres similitudes valen exactamente cero. *cat* está igual de lejos de *dog* que de *car*.

Ojito...

One-hot tiene los dos defectos a la vez. No captura ninguna similitud, porque todos los vectores son perpendiculares entre sí. Y la dimensión es el tamaño del vocabulario: con 50 000 palabras, cada vector tiene 50 000 componentes para guardar un solo bit de información. Es un índice disfrazado de vector.

Aun así no es inútil. Es la forma de *entrar* a una red neuronal, como se ve al final de la clase.

Métodos basados en conteo

La receta general tiene dos pasos. Primero se arma una matriz donde los renglones son palabras, las columnas son contextos potenciales, y cada elemento dice algo de la asociación entre esa palabra y ese contexto. Después se reduce su dimensión con SVD truncada:

$$\underbrace{M}_{|V|\times|C|} \approx \underbrace{V_d}_{\substack{\text{vectores de}\\ \text{palabra}}} \times \Sigma_d \times \underbrace{U_d^T}_{\substack{\text{vectores de}\\ \text{contexto}}}$$

Cada renglón de V_d es el vector de una palabra, ya con solo d componentes. Las columnas de U_d^T son los vectores de los contextos.

Observación. En la diapositiva los factores se llaman V_d y U_d^T, al revés de la notación $U\Sigma V^T$ de la clase del 28. Es solo el nombre: el factor de la izquierda es el de los renglones de la matriz, que aquí son las palabras.

Para usar la receta hay que decidir dos cosas:

1. **Qué es el contexto.** Las palabras que rodean a la objetivo dentro de una ventana de tamaño L, o el documento completo, o una oración.

2. **Cómo se calcula cada elemento.** Lo más simple es el conteo $N(w, c)$. En la práctica los conteos crudos se repesan, por la misma razón que en IR se usa tf-idf y no tf: palabras como *the* coocurren con todo y dominan los vectores sin decir nada.

Idea clave

El repesado más usado con matrices término-término es la **información mutua puntual positiva** (PPMI):

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)\,P(c)}, \qquad \text{PPMI}(w, c) = \text{máx}\,(\text{PMI}(w, c),\, 0).$$

Compara qué tan seguido aparecen juntas contra qué tan seguido lo harían si fueran independientes. Si *cherry* y *pie* coocurren mucho más de lo que explicaría el azar, el PMI es alto; si *cherry* y *the* coocurren justo lo esperado por lo frecuente que es *the*, el PMI es cercano a cero. Los valores negativos se cortan porque con corpus finitos no son confiables.

Qué son los *word embeddings*

Definición 13.8 (Word embedding)**.** Un *word embedding* representa cada palabra como un vector de **baja dimensión**, típicamente entre 50 y 1000 componentes, que conserva la distribución semántica de las palabras: palabras de uso parecido quedan con vectores cercanos.

Se les llama también:

- **Representaciones distribuidas.** El significado no vive en una sola componente, como en one-hot, sino repartido entre todas. Ninguna dimensión por sí sola quiere decir "animal" o "comida"; es el patrón completo el que significa.
- **Modelos semánticos distribucionales.** La definición de Baroni, Dinu y Kruszewski (2014): usan vectores que registran los contextos, o sea las palabras que coocurren, en los que aparecen los términos objetivo en un corpus grande, como sustitutos de la representación del significado, y les aplican técnicas geométricas para medir la similitud de significado entre las palabras correspondientes.

	Vectores de conteo	**Embeddings**
Dimensión	$\|V\|$ o número de documentos	50 a 1000
Entradas	Casi todas cero (dispersos)	Casi ninguna cero (densos)
Cada componente	Un contexto concreto	Nada interpretable sola

La motivación está en la diapositiva 19: representar las palabras como índices de un vocabulario es común, pero no captura la rica estructura relacional del léxico. Los embeddings codifican la similitud entre palabras como distancia o ángulo entre vectores de un espacio, y esa similitud es continua: no es "igual" o "distinta", sino qué tan parecida.

La matriz de embeddings

En la práctica todos los embeddings se guardan en una sola matriz E, con un renglón por palabra del vocabulario y una columna por dimensión del embedding. Para obtener el vector de una palabra se toma su índice en el vocabulario y se lee ese renglón.

Ejemplo 13.9. La oración *"I saw a cat ."* se convierte primero en índices del vocabulario:

I	*saw*	*a*	*cat*	*.*
39	1592	10	2548	5

El embedding de *saw* es el renglón 1592 de E. La oración completa queda como cinco vectores densos, uno por token.

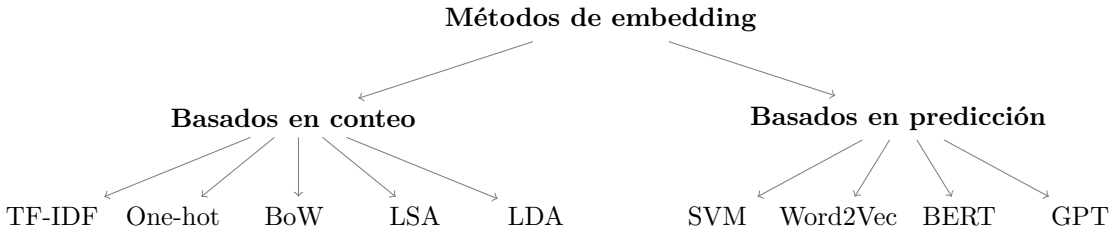
Idea clave

Leer el renglón i de E es lo mismo que multiplicar el vector one-hot de la palabra por E. Si x es one-hot con el 1 en la posición i, el producto $x^T E$ suma los renglones de E pesados por las entradas de x, y la única que no es cero es la i:

$$\begin{pmatrix}0 & 1 & 0\end{pmatrix}\begin{pmatrix}2 & 7\\ 5 & 1\\ 3 & 8\end{pmatrix} = \begin{pmatrix}5 & 1\end{pmatrix}.$$

Por eso one-hot sigue apareciendo: es la entrada formal de una red, y la primera capa de la red es justo E. En el código nadie hace la multiplicación, se hace la búsqueda por índice, que da lo mismo y es mucho más barata.

Conteo contra predicción



- **Basados en conteo.** Cuentan cuántas veces aparece una palabra en relación con otras y construyen la codificación a partir de esos conteos. Primero recolectan los vectores de contexto y luego los repesan con algún criterio: tf-idf, PPMI, SVD. LSA y LDA son enfoques estadísticos, pero siguen infiriendo las propiedades del conjunto de datos por heurísticas más que por un modelo que se entrena.

- **Basados en predicción.** No cuentan. Entrenan un modelo para predecir una palabra a partir de su contexto, o el contexto a partir de la palabra, y los embeddings salen como parámetros de ese modelo.

Estos son los dos tipos de embeddings que se mencionaron al inicio de la clase. El título del artículo de Baroni et al. resume la conclusión de su comparación: *"Don't count, predict!"*. En sus experimentos los vectores entrenados prediciendo ganaron en casi todas las tareas.

Ojito...

La SVM aparece en la diapositiva del lado de predicción, pero no genera embeddings: es un clasificador que *usa* representaciones. Está ahí porque se entrena para predecir, no porque produzca vectores de palabras.

Modelo de lenguaje basado en redes neuronales

Con técnicas de aprendizaje profundo los embeddings se pueden aprender sin contar nada. La red recibe palabras y, para hacer bien su tarea, tiene que aprender vectores que capturen su semántica. El primer modelo de este tipo fue un modelo de lenguaje.

Recordando la clase del 31 de agosto, un modelo de lenguaje asigna probabilidad a la siguiente palabra dada la historia. La historia completa es demasiado larga, así que se aproxima con las últimas $N-1$

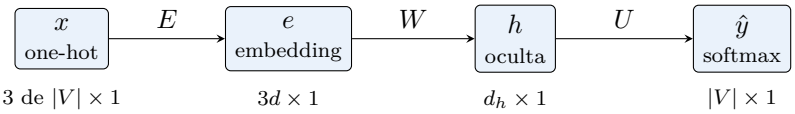
palabras:

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-N+1}, \ldots, w_{t-1}).$$

El modelo de n-gramas estimaba esa probabilidad contando. El modelo neuronal la *calcula*: es una red *feed-forward* que en el tiempo t recibe una representación de las palabras previas (w_{t-1}, w_{t-2}, etc.) y devuelve una distribución de probabilidad sobre todas las posibles palabras siguientes.

La arquitectura, capa por capa

Ejemplo 13.10 (Predecir la palabra después de *"for all the"*)**.** Con $N = 4$ la red ve las tres palabras previas. Sean $|V|$ el tamaño del vocabulario, d la dimensión del embedding y d_h el número de neuronas ocultas.



Entrada. Cada palabra previa entra como vector one-hot de $|V| \times 1$: *for* con el 1 en la posición 35, *all* en la 992 y *the* en la 451.

Capa de embedding. Cada one-hot se multiplica por la matriz E, de $d \times |V|$, que es la misma para las tres posiciones. Por la idea de la sección anterior, eso solo selecciona una columna de E por palabra. Los tres vectores de d componentes se concatenan en e, de $3d \times 1$.

Capa oculta. Una capa densa con función de activación σ no lineal:

$$h = \sigma(We + b), \qquad W \text{ de } d_h \times 3d.$$

Salida. Un puntaje por palabra del vocabulario, $z = Uh$ con U de $|V| \times d_h$, y luego softmax para volverlo probabilidad:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}}.$$

$\hat{y}_{42}$ es $P(\textit{fish} \mid \textit{for all the})$, $\hat{y}_{|V|}$ es $P(\textit{zebra} \mid \textit{for all the})$, y así con cada palabra.

Ejemplo 13.11 (Softmax con tres palabras)**.** Si el vocabulario fuera solo *day*, *fish* y *zebra*, y la red diera $z = (2,\ 1,\ 0.1)$:

$$e^2 = 7.389, \qquad e^1 = 2.718, \qquad e^{0.1} = 1.105, \qquad \text{suma} = 11.212,$$

$$\hat{y} = \left(\frac{7.389}{11.212}, \frac{2.718}{11.212}, \frac{1.105}{11.212}\right) = (0.659,\ 0.242,\ 0.099).$$

Las tres suman 1 y respetan el orden de los puntajes. La exponencial hace que ninguna salga negativa y exagera las diferencias: un punto más de puntaje multiplica la probabilidad por $e \approx 2.7$.

De dónde salen los embeddings

Al principio E, W y U son números al azar. La red se entrena sobre un corpus grande: en cada posición del texto ve las tres palabras previas, predice la siguiente, compara contra la palabra que realmente siguió y ajusta todos los pesos con retropropagación para que esa palabra reciba más probabilidad. E también se ajusta.

Al terminar, a nadie le importa tanto el modelo de lenguaje: lo que se queda es E. Cada una de sus columnas es el embedding de una palabra. Los modelos de lenguaje neuronales representan las palabras por sus embeddings, y los aprenden como efecto secundario de predecir.

Idea clave

Por qué los vectores salen con significado: si *cat* y *dog* suelen ir seguidos de las mismas palabras (*is sleeping*, *ate the*), a la red le conviene darles embeddings parecidos, porque así las dos producen predicciones parecidas. La hipótesis distribucional no se programó; la impone el objetivo de entrenamiento.

Por qué es mejor que los n-gramas

- **Generaliza a combinaciones que nunca vio.** Si en el entrenamiento apareció *"the cat is walking in the bedroom"* pero nunca *"a dog was running in a room"*, el modelo de n-gramas le da probabilidad casi cero a la segunda. El neuronal no, porque *cat* y *dog*, *walking* y *running*, *bedroom* y *room* tienen embeddings cercanos y producen activaciones cercanas.
- **Soporta historias mucho más largas.** Los parámetros de un n-grama crecen como $|V|^N$: cada historia posible es una fila de la tabla, y casi todas quedan con conteo cero. En la red, agregar una palabra más de historia solo agrega d entradas a e y $d \cdot d_h$ pesos a W.

Ejemplo 13.12 (Cuántos parámetros)**.** Con $|V| = 10\,000$, $d = 100$, tres palabras de historia y $d_h = 500$:

$$\begin{aligned} E &: 100 \times 10\,000 = 1\,000\,000, \\ W &: 500 \times 300 = 150\,000, \\ U &: 10\,000 \times 500 = 5\,000\,000. \end{aligned}$$

Un trigrama sobre el mismo vocabulario tendría hasta $10\,000^3 = 10^{12}$ probabilidades. La mayor parte del costo de la red está en U, la capa de salida, porque tiene que dar un puntaje a cada palabra del vocabulario.

Ojito...

El softmax es el cuello de botella: para cada ejemplo de entrenamiento hay que calcular e^{z_j} para las $|V|$ palabras solo para obtener el denominador. Con vocabularios de cientos de miles esto es muy lento, y buena parte de lo que después hizo Word2Vec fue esquivar ese cálculo.

14. Word2Vec, FastText y GloVe

5 de octubre de 2026

Resumen rápido. Continuación de los *word embeddings*. Primero cómo se entrena la red que los produce: con autosupervisión, usando el texto mismo como etiqueta. Luego los métodos que hicieron práctico el entrenamiento: Word2Vec con sus dos arquitecturas, CBOW y skip-gram, y los dos atajos para no pagar el softmax completo, negative sampling y softmax jerárquico. FastText agrega subpalabras para cubrir palabras que nunca vio, y GloVe cambia la predicción por un ajuste directo a la matriz de coocurrencias. Se cerró con las propiedades semánticas de los vectores y con su límite: un solo vector por palabra no distingue sentidos.

Cómo se entrena la red de embeddings

La clase pasada terminó con el modelo de lenguaje neuronal: tres palabras entran como one-hot, pasan por E, por una capa oculta y por un softmax que da la probabilidad de cada palabra siguiente. Faltaba decir cómo se ajustan sus pesos y de dónde salen los datos para ajustarlos.

Autosupervisión

Definición 14.1 (Aprendizaje autosupervisado)**.** Un modelo es autosupervisado (*self-supervised*) cuando las etiquetas con las que se entrena salen de los propios datos, sin que nadie las anote. Se esconde una parte de la entrada y se le pide al modelo que la recupere a partir del resto.

En el modelo de lenguaje la parte escondida es la siguiente palabra. Se toma un corpus y se recorre con una ventana: las palabras de la ventana son la entrada y la que sigue es la respuesta correcta. No hace falta ninguna etiqueta especial (*gold label*), solo la secuencia natural de las palabras.

Ejemplo 14.2 (Ejemplos de entrenamiento a partir de una frase)**.** Con $N = 4$, la red ve tres palabras y predice la cuarta. La frase *"and thanks for all the fish"* da tres ejemplos:

Entrada	Respuesta
and thanks for	*all*
thanks for all	*the*
for all the	*fish*

La ventana avanza una palabra a la vez. Un corpus de mil millones de palabras da casi mil millones de ejemplos etiquetados gratis.

Idea clave

Esto es lo que hace posibles los embeddings. Un clasificador normal necesita que alguien etiquete cada ejemplo, y eso limita los datos a miles o millones. El texto se etiqueta solo, así que el límite es cuánto texto hay.

La pérdida y el ajuste

Para cada ejemplo la red devuelve $\hat{y}$, una probabilidad por palabra del vocabulario. La pérdida mide qué probabilidad le dio a la palabra que de verdad seguía. Es la entropía cruzada, que con una sola respuesta correcta se reduce a

$$L = -\log \hat{y}_{\text{correcta}}.$$

Si la red le dio probabilidad 1, la pérdida es 0; si le dio casi 0, la pérdida se dispara.

Ejemplo 14.3 (Pérdida con el softmax de la clase pasada)**.** Con el vocabulario *day*, *fish*, *zebra* la red dio $\hat{y} = (0.659,\ 0.242,\ 0.099)$. Si la palabra que seguía era *fish*:

$$L = -\ln 0.242 = 1.42.$$

Si hubiera sido *day*, $L = -\ln 0.659 = 0.42$: la red ya apostaba por ella y se equivocó poco.

El gradiente de la pérdida respecto a los puntajes z tiene una forma muy limpia, $\hat{y} - y$, donde y es el one-hot de la respuesta:

$$\hat{y} - y = (0.659,\ 0.242 - 1,\ 0.099) = (0.659,\ -0.758,\ 0.099).$$

El descenso de gradiente resta este vector: sube el puntaje de *fish*, la única componente negativa, y baja el de las demás, cada una en proporción a la probabilidad que se llevó sin merecerla.

Ese gradiente se propaga hacia atrás por U y por W hasta llegar a E. En E solo cambian tres columnas, las de *for*, *all* y *the*, porque son las únicas que participaron en este ejemplo. Se repite con cada posición del corpus, varias pasadas, y al final E contiene los embeddings.

Ojito...

Nadie le dice a la red cómo debe quedar E. Solo se le pide predecir bien, y predecir bien obliga a darles vectores parecidos a las palabras que se usan parecido. Es la hipótesis distribucional de la clase pasada, impuesta por la pérdida.

Word2Vec

Cuando se empezó a trabajar con embeddings se propusieron varios algoritmos. El más influyente fue Word2Vec (Mikolov et al., 2013). Su idea, en la diapositiva 33:

- Entrenar un clasificador sobre una tarea de predicción binaria: ¿es probable que la palabra w_t aparezca cerca de otra palabra w_c?
- Aprender vectores que sirvan para predecir contextos.
- Los pesos que aprende el clasificador **son** los embeddings.

La tarea siempre es generar una representación de las palabras; la predicción es un pretexto para obtener los pesos. Comparado con el modelo de lenguaje neuronal, Word2Vec quita todo lo que no ayuda a los vectores y se queda con un modelo mucho más barato, y eso le permitió entrenar con miles de millones de palabras.

Definición 14.4 (Embeddings estáticos)**.** Word2Vec aprende **un solo embedding fijo para cada palabra** del vocabulario. El vector de *banco* es el mismo en *"me senté en el banco"* y en *"abrí una cuenta en el banco"*.

Tiene dos arquitecturas, que son la misma idea al revés: **continuous bag-of-words** (CBOW) y **skip-gram**.

Continuous bag-of-words (CBOW)

Predice la palabra central a partir de las que la rodean. Se parece al modelo de lenguaje neuronal, con tres cambios:

1. **El contexto va a los dos lados.** No solo ve las palabras anteriores, también las siguientes: C a la izquierda y C a la derecha de la palabra objetivo.
2. **Se quita la capa oculta no lineal.** Ya no hay W ni σ.
3. **La proyección es compartida y se promedia.** Los embeddings del contexto no se concatenan; se promedian en un solo vector.

- **Entrada:** las palabras del contexto de w_t, definido por una ventana de tamaño C.
- **Salida:** una sola palabra, w_t.

En fórmulas, con v_w el embedding de w (su columna de E):

$$h = \frac{1}{2C} \sum_{\substack{-C \le j \le C \\ j \neq 0}} v_{w_{t+j}}, \qquad P(w_t \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}) = \text{softmax}(Uh)_{w_t}.$$

Ojito...

Como se promedia, el orden de las palabras del contexto no influye en la proyección: *"el perro muerde"* y *"muerde el perro"* dan el mismo h. De ahí el nombre, la misma bolsa de palabras de las primeras clases de IR. Lo de *continuous* es porque la bolsa es de vectores densos y no de conteos.

Skip-gram

Es al revés: dada la palabra central, predice las que la rodean.

- **Entrada:** una sola palabra, w_t.
- **Salida:** las palabras del contexto de w_t, definido por una ventana de tamaño C.

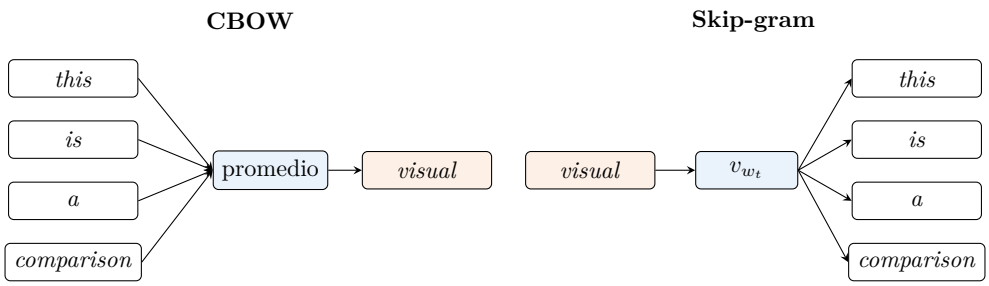
La palabra actual entra a un clasificador log-lineal con una capa de proyección, y se predicen las palabras que caen en cierto rango antes y después de ella. Log-lineal quiere decir que, sin capa oculta, el logaritmo de la probabilidad es lineal en los vectores:

$$h = v_{w_t}, \qquad P(w_{t+j} \mid w_t) = \text{softmax}(Uh)_{w_{t+j}} \quad \text{para } -C \le j \le C,\ j \neq 0.$$

Cada posición del contexto es un ejemplo aparte, y el entrenamiento maximiza la suma de $\log P(w_{t+j} \mid w_t)$ sobre todas las posiciones del corpus y todos los j de la ventana.

CBOW contra skip-gram

Ejemplo 14.5 (*"This is a visual comparison"*)**.** Con *visual* como palabra objetivo y una ventana que alcanza toda la frase:



CBOW junta las cuatro palabras del contexto en un solo vector y hace **una** predicción: *visual*. Skip-gram parte de *visual* y hace **cuatro** predicciones, una por cada palabra del contexto, o sea cuatro pares de entrenamiento: (*visual*, *this*), (*visual*, *is*), (*visual*, *a*) y (*visual*, *comparison*).

	CBOW	**Skip-gram**
Predice	la palabra central	el contexto
Ejemplos por posición	1	$2C$
Velocidad	más rápido	más lento
Palabras raras	se diluyen en el promedio	salen mejor

La última fila es consecuencia de la segunda. En CBOW una palabra rara es una de $2C$ que se promedian, y su señal se pierde. En skip-gram cada aparición de la palabra rara genera $2C$ ejemplos donde ella es la entrada, así que su vector se ajusta más veces.

Negative sampling

Skip-gram tal cual hereda el problema con el que cerró la clase pasada. Cada paso de entrenamiento tiene tres partes (diapositiva 41):

1. Buscar el embedding de la palabra de entrada.
2. Calcular la predicción.
3. Proyectar al vocabulario de salida: un puntaje y una exponencial por **cada** palabra, para el denominador del softmax.

El tercer paso es el caro. Con $|V| = 100\,000$ son cien mil productos punto por cada par de entrenamiento, y un corpus grande tiene miles de millones de pares.

La salida de Mikolov et al. fue cambiar la tarea. En lugar de preguntar *"¿cuál de las $|V|$ palabras es la vecina?"*, se pregunta *"¿son vecinas estas dos palabras, sí o no?"*. Es la tarea binaria de la diapositiva 33. Para entrenarla hacen falta ejemplos de las dos clases:

- **Positivos**, con etiqueta 1: los pares de skip-gram de siempre, la palabra y uno de sus vecinos reales.
- **Negativos**, con etiqueta 0: la palabra y otra sacada al azar del vocabulario, que casi seguro no es su vecina.

Por cada positivo se sacan k negativos. De ahí el nombre: muestreo de negativos.

Ejemplo 14.6 (Los datos de entrenamiento)**.** Frase: *"thou shalt not make a machine"*, palabra de entrada *make* y ventana de ± 2.

Skip-gram genera cuatro pares, todos positivos:

Entrada	**Salida**
make	*shalt*
make	*not*
make	*a*
make	*machine*

Negative sampling toma cada par positivo y le agrega k negativos al azar. Con $k = 2$, el par (*make*, *shalt*) queda así:

Palabra	Otra palabra	Etiqueta
make	*shalt*	1
make	*aaron*	0
make	*taco*	0

aaron y *taco* no aparecen en la frase; salieron del vocabulario al azar.

El clasificador

Cada palabra tiene ahora **dos** vectores: w cuando es la palabra de entrada y c cuando es la otra palabra del par. Viven en dos matrices, la de embeddings y la de contexto. La probabilidad de que el par sea real sale del producto punto y la sigmoide:

$$P(+ \mid w, c) = \sigma(c \cdot w) = \frac{1}{1 + e^{-c \cdot w}}, \qquad P(- \mid w, c) = 1 - \sigma(c \cdot w) = \sigma(-c \cdot w).$$

Producto punto alto quiere decir vectores parecidos, y entonces probabilidad alta de ser vecinas. Para un positivo c_+ y sus k negativos $c_1, \ldots, c_k$, la pérdida es

$$L = -\log \sigma(c_+ \cdot w) - \sum_{i=1}^{k} \log \sigma(-c_i \cdot w).$$

Pide que el positivo tenga probabilidad alta de ser real y que cada negativo tenga probabilidad alta de no serlo.

Ejemplo 14.7 (Un paso de entrenamiento con vectores de dos componentes)**.** Sean

$$w_{make} = (1,\ 0.5), \quad c_{shalt} = (0.8,\ 0.6), \quad c_{aaron} = (0.2,\ -0.4), \quad c_{taco} = (0.6,\ 0.4).$$

Probabilidades.

Par	$c \cdot w$	$\sigma(c \cdot w)$	Debería ser
make, *shalt*	$0.8 + 0.3 = 1.1$	0.750	cerca de 1
make, *aaron*	$0.2 - 0.2 = 0$	0.500	cerca de 0
make, *taco*	$0.6 + 0.2 = 0.8$	0.690	cerca de 0

El modelo cree que *taco* es vecino de *make* con probabilidad 0.69. Es el error más grande.

Pérdida.

$$L = -\ln 0.750 - \ln(1 - 0.500) - \ln(1 - 0.690) = 0.287 + 0.693 + 1.171 = 2.152.$$

Gradientes. Derivando L:

$$\frac{\partial L}{\partial c_+} = \big(\sigma(c_+ \cdot w) - 1\big)\, w, \qquad \frac{\partial L}{\partial c_i} = \sigma(c_i \cdot w)\, w,$$

$$\frac{\partial L}{\partial w} = \big(\sigma(c_+ \cdot w) - 1\big)\, c_+ + \sum_{i=1}^{k} \sigma(c_i \cdot w)\, c_i.$$

Los coeficientes son $0.750 - 1 = -0.25$ para *shalt*, 0.5 para *aaron* y 0.69 para *taco*.

Actualización con tasa de aprendizaje $\eta = 1$, restando el gradiente:

$$\begin{aligned}
c_{shalt} &\leftarrow (0.8,\ 0.6) + 0.25\,(1,\ 0.5) = (1.05,\ 0.725),\\
c_{aaron} &\leftarrow (0.2,\ -0.4) - 0.5\,(1,\ 0.5) = (-0.3,\ -0.65),\\
c_{taco} &\leftarrow (0.6,\ 0.4) - 0.69\,(1,\ 0.5) = (-0.09,\ 0.055),\\
w_{make} &\leftarrow (1,\ 0.5) - \big[-0.25\,(0.8,\ 0.6) + 0.5\,(0.2,\ -0.4) + 0.69\,(0.6,\ 0.4)\big]\\
&= (1,\ 0.5) - (0.314,\ -0.074) = (0.686,\ 0.574).
\end{aligned}$$

Después del paso.

Par	Antes	Después
make, *shalt*	0.750	0.757
make, *aaron*	0.500	0.359
make, *taco*	0.690	0.492

La pérdida baja de 2.152 a 1.402. El vecino real se acercó un poco a *make* y los dos falsos se alejaron bastante, que es donde estaba el error. Todo con tres productos punto, sin tocar el resto del vocabulario.

Idea clave

El costo por par pasa de $|V|$ productos punto a $k+1$. Con $|V| = 100\,000$ y $k = 5$, de cien mil a seis. Los valores usuales son k entre 5 y 20 con pocos datos, y entre 2 y 5 con corpus grandes.

Los negativos no se sacan con la frecuencia tal cual. Si así fuera, casi todos serían *the*, *of* y *a*. Se usa la frecuencia elevada a 0.75, que les quita peso a las palabras muy frecuentes y se lo da a las raras:

$$P_{0.75}(w) = \frac{\text{count}(w)^{0.75}}{\sum_u \text{count}(u)^{0.75}}.$$

Ejemplo 14.8. Con dos palabras de probabilidad 0.99 y 0.01:

$$0.99^{0.75} = 0.992, \qquad 0.01^{0.75} = 0.032,$$

$$P_{0.75} = \left(\frac{0.992}{1.024},\ \frac{0.032}{1.024}\right) = (0.969,\ 0.031).$$

La rara pasa de salir 1 de cada 100 veces a 3 de cada 100.

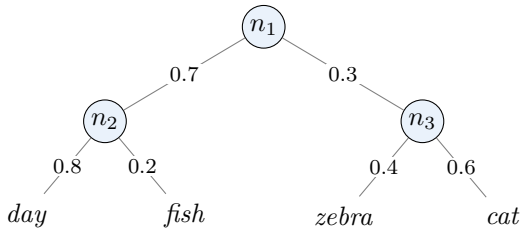
Al terminar quedan dos matrices. Lo común es quedarse con la de embeddings y tirar la de contexto, o sumar los dos vectores de cada palabra.

Softmax jerárquico

La otra forma de esquivar el costo conserva el softmax, pero lo calcula de otra manera. CBOW y skip-gram estándar usan una capa softmax en la salida, y es muy cara: para calcular y normalizar cada probabilidad hace falta el puntaje de todas las demás palabras del vocabulario, en cada contexto y en cada paso de entrenamiento.

El softmax jerárquico pone las palabras en las hojas de un árbol binario. Cada nodo interno tiene su propio vector n, y en cada nodo se decide ir a la izquierda con probabilidad $\sigma(n \cdot h)$ o a la derecha con $1 - \sigma(n \cdot h)$. La probabilidad de una palabra es el producto de las decisiones en el camino de la raíz a su hoja.

Ejemplo 14.9 (Cuatro palabras)**.** Un vocabulario de cuatro palabras y su árbol, con la probabilidad de tomar cada rama:



Con $\sigma(n_1 \cdot h) = 0.7$, $\sigma(n_2 \cdot h) = 0.8$ y $\sigma(n_3 \cdot h) = 0.4$:

$$P(day) = 0.7 \cdot 0.8 = 0.56, \qquad P(fish) = 0.7 \cdot 0.2 = 0.14,$$
$$P(zebra) = 0.3 \cdot 0.4 = 0.12, \qquad P(cat) = 0.3 \cdot 0.6 = 0.18.$$

Suman 1 sin dividir entre nada: en cada nodo las dos ramas suman 1, así que la probabilidad se reparte entera hasta las hojas. Para la de *fish* se calcularon dos sigmoides, no cuatro exponenciales.

Idea clave

El camino a cualquier hoja mide unos $\log_2 |V|$ nodos. Con un millón de palabras son 20 sigmoides en lugar de un millón de exponenciales. Word2Vec usa un árbol de Huffman, el de compresión: las palabras frecuentes quedan cerca de la raíz con caminos cortos, y como son las que más se entrenan, el promedio baja todavía más.

Ojito...

Los dos atajos persiguen lo mismo por caminos distintos. El softmax jerárquico sigue dando una distribución completa sobre el vocabulario. Negative sampling ya no: cambia la pregunta por una binaria y no le importa que el modelo deje de ser un modelo de lenguaje, porque lo único que se quería eran los vectores.

FastText

Word2Vec deja dos problemas abiertos (diapositiva 47):

- **Palabras fuera del vocabulario** (*OOV*). Si en el entrenamiento aparecieron *tensor* y *flow*, hay vector para cada una, pero si después llega *tensorflow* no hay nada. La palabra no existe para el modelo.
- **Morfología.** *eat*, *eats*, *eaten*, *eater* y *eating* son cinco palabras distintas con cinco vectores independientes. Que compartan raíz no le sirve de nada al modelo: cada una se aprende por su cuenta, y las formas raras se aprenden mal.

El segundo pesa más en español que en inglés. Un verbo regular tiene decenas de formas conjugadas, y muchas aparecen muy pocas veces en cualquier corpus.

FastText (Bojanowski, Grave, Joulin y Mikolov, 2017) los resuelve entrenando por debajo de la palabra. Es una extensión de skip-gram que toma en cuenta información de subpalabras.

Generación de subpalabras

Se usa una ventana, pero de caracteres. La palabra se encierra entre < y > para marcar dónde empieza y dónde termina, y se parte en todos sus n-gramas de caracteres.

Ejemplo 14.10 (Los n-gramas de *eating*)**.** Con los marcadores la palabra queda `<eating>`.

n	**N-gramas de caracteres**
3	`<ea, eat, ati, tin, ing, ng>`
4	`<eat, eati, atin, ting, ing>`
5	`<eati, eatin, ating, ting>`
6	`<eatin, eating, ating>`

Los marcadores distinguen posiciones: `<ea` es *ea* al principio de la palabra y `ng>` es *ng* al final. Sin ellos, el `her` de *where* sería el mismo n-grama que la palabra *her*, que con marcadores es `<her>`.

Cada n-grama tiene su propio vector, y la palabra se representa como la **suma** de los vectores de sus n-gramas, más uno para la palabra completa `<eating>`. En la práctica se usan todos los n-gramas con n de 3 a 6. Lo demás es skip-gram con negative sampling, con esa suma en el lugar donde antes estaba v_{w_t}.

Ejemplo 14.11 (Una palabra que nunca vio)**.** Con $n = 3$:

tensor	`<te, ten, ens, nso, sor, or>`
flow	`<fl, flo, low, ow>`
tensorflow	`<te, ten, ens, nso, sor, orf, rfl, flo, low, ow>`

De los 10 n-gramas de *tensorflow*, 8 ya tienen vector porque salieron en *tensor* o en *flow*; solo `orf` y `rfl` no están en esas dos palabras. Sumando lo que hay, la palabra nueva recibe un vector que cae entre los de sus dos partes, sin haberla visto nunca.

Con la morfología pasa lo mismo: *eats*, *eaten* y *eating* comparten `<ea` y `eat`, así que sus vectores tienen una parte en común y lo que se aprende de una ayuda a las otras.

Ojito...

El precio es que se multiplican las operaciones. Cada palabra ya no es un vector sino la suma de diez o veinte, y hay muchos más vectores que entrenar, porque los n-gramas distintos de un corpus son millones. Para que la memoria no crezca sin límite, FastText no guarda un vector por n-grama: los reparte con una función hash en un número fijo de cubetas, unos dos millones, y dos n-gramas pueden terminar compartiendo vector. A cambio, puede generar vectores para cualquier palabra.

GloVe

GloVe, de **Glo**bal **Ve**ctors (Pennington, Socher y Manning, 2014), regresa a contar. Está basado en las coocurrencias de palabras en un corpus y no en una red neuronal: es la matriz término-término de la clase pasada, con una ventana, contada una sola vez sobre todo el corpus. La diapositiva 50 usa justo la tabla de *cherry*, *strawberry*, *digital* e *information*.

La diferencia con Word2Vec está en qué ve el modelo. Skip-gram recorre el corpus ventana por ventana, y cada paso ve un solo contexto local. GloVe ve las estadísticas **globales**: cuántas veces coocurrió cada par en todo el corpus.

El significado está en las razones

La idea de partida es que las probabilidades de coocurrencia pueden codificar algo de significado, pero no por sí solas, sino comparadas. Sea X_{ij} el número de veces que j aparece en el contexto de i, y $P(j \mid i) = X_{ij} / \sum_k X_{ik}$.

Ejemplo 14.12 (*ice* contra *steam*)**.** Del artículo original, sobre un corpus de 6 000 millones de palabras:

k	*solid*	*gas*	*water*	*fashion*
$P(k \mid ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
Razón	8.9	0.085	1.36	0.96

Las probabilidades sueltas son todas chiquitas y difíciles de leer. La razón sí dice algo: *solid* va con *ice* y no con *steam*, razón grande; *gas* al revés, razón chica; *water* va con las dos y *fashion* con ninguna, y las dos dan cerca de 1. Las razones del artículo salen de los conteos sin redondear; por eso 1.9/0.22 no da exactamente 8.9.

El modelo

GloVe busca vectores cuyo producto punto reproduzca el logaritmo del conteo:

$$w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}.$$

w_i es el vector de la palabra i, $\tilde{w}_j$ el de j como contexto, y b_i, $\tilde{b}_j$ son sesgos que absorben qué tan frecuente es cada palabra. Con el logaritmo, una razón de probabilidades se vuelve una resta, y la resta de productos punto es el producto punto con la resta de vectores:

$$\log \frac{P(k \mid ice)}{P(k \mid steam)} \approx (w_{ice} - w_{steam}) \cdot \tilde{w}_k + \text{términos que no dependen de } k.$$

Así, la diferencia entre los vectores de *ice* y *steam* queda apuntando en la dirección que separa lo sólido de lo gaseoso. Por eso se le llama modelo **log-bilineal**: el logaritmo del conteo se modela con una expresión lineal en cada uno de los dos vectores.

Definición 14.13 (Objetivo de GloVe)**.**

$$J = \sum_{i,j} f(X_{ij}) \left(w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2,$$

con la función de peso

$$f(x) = \begin{cases} (x/x_{\text{máx}})^{3/4} & \text{si } x < x_{\text{máx}}, \\ 1 & \text{si } x \geq x_{\text{máx}}, \end{cases} \qquad x_{\text{máx}} = 100.$$

Es mínimos cuadrados ponderados: el error al cuadrado entre lo que predice el modelo y el logaritmo del conteo, pesado por f. El peso resuelve tres cosas:

- $f(0) = 0$, así que las celdas con conteo cero no entran a la suma. Hace falta, porque $\log 0$ no existe, y además son la mayor parte de la matriz: solo se ajustan las celdas distintas de cero.
- Los pares raros pesan poco. Un par que coocurrió dos veces puede ser casualidad, y su logaritmo es ruido.
- Los pares muy frecuentes no dominan: arriba de 100 el peso se queda en 1 en lugar de seguir creciendo.

Ejemplo 14.14 (Dos celdas del renglón de *cherry*)**.** De la tabla de la clase pasada, $X_{cherry,pie} = 442$ y $X_{cherry,computer} = 2$.

Pesos.

$$f(442) = 1, \qquad f(2) = (2/100)^{0.75} = 0.053.$$

Objetivos. $\ln 442 = 6.09$ y $\ln 2 = 0.69$.

Supongamos que con los vectores actuales el lado izquierdo da 5.5 para (*cherry*, *pie*) y 2.0 para (*cherry*, *computer*). Los dos términos de J son

$$\begin{aligned} \textit{pie:} \quad & 1 \cdot (5.5 - 6.09)^2 = 1 \cdot 0.348 = 0.348, \\ \textit{computer:} \quad & 0.053 \cdot (2.0 - 0.69)^2 = 0.053 \cdot 1.72 = 0.091. \end{aligned}$$

El error de *computer* es más del doble, pero cuesta la cuarta parte. El ajuste se concentra en que *cherry* y *pie* queden bien, que es el par con evidencia. La celda de *aardvark*, con conteo 0, ni siquiera aparece en J.

Idea clave

GloVe es factorización de matrices, como LSA en la clase del 28: busca dos matrices de vectores, W y $\tilde{W}$, cuyo producto reproduzca la matriz de log-conteos. La diferencia con la SVD truncada es que no trata igual a todas las celdas: ignora los ceros y pesa cada celda según cuánta evidencia tiene. Es lo que la diapositiva 52 llama extraer la estructura latente de la matriz de coocurrencias. Junta las estadísticas globales de los métodos de conteo con el tipo de vectores que dan los de predicción.

Igual que en negative sampling, al final cada palabra tiene dos vectores, w_i y $\tilde{w}_i$, y se usa la suma $w_i + \tilde{w}_i$.

Cuál funciona mejor

Depende de la tarea que se esté haciendo. Ninguno gana siempre.

	Qué aprende	**Palabras nuevas**	**Costo**
CBOW	Predecir la palabra central con su contexto	No	El más barato: un ejemplo por posición
Skip-gram	Predecir el contexto con la palabra central	No	$2C$ ejemplos por posición; mejores vectores para palabras raras
FastText	Skip-gram con n-gramas de caracteres	Sí, con sus n-gramas	Cada palabra es una suma de n-gramas
GloVe	Ajustar la matriz de coocurrencias global	No	Se cuenta una vez; se ajustan solo las celdas distintas de cero

Con poco texto o mucho vocabulario raro conviene skip-gram. Si el idioma tiene mucha morfología o el texto trae errores de dedo, FastText, porque las variantes comparten n-gramas. GloVe entrena sobre las celdas distintas de cero y no sobre el corpus, así que después de contar, entrenar es rápido. La forma segura de escoger es probar con la tarea final.

Visualizar embeddings

Los embeddings tienen cientos de dimensiones y no se pueden ver directamente. Para inspeccionarlos se proyectan a dos. Con PCA o SVD, la de la clase del 28, se conservan las direcciones de mayor varianza;

con t-SNE se conservan las vecindades: lo que estaba cerca en el espacio original queda cerca en el plano.

Ejemplo 14.15 (Diapositiva 53)**.** Una proyección t-SNE de embeddings de 60 dimensiones entrenados para análisis de sentimiento (Li et al., 2015). Salen tres grupos:

- palabras funcionales: *to*, *by*, *that*, *now*, *a*, *i*, *you*, *than*, *with*, *is*;
- positivas: *good*, *nice*, *very good*, *amazing*, *terrific*, *fantastic*, *wonderful*, *incredibly good*;
- negativas: *bad*, *worse*, *worst*, *dislike*, *not good*, *incredibly bad*.

Las palabras de significado parecido quedan cerca en el espacio.

Ojito...

Que *good* y *bad* queden lejos es por la tarea con la que se entrenaron, sentimiento. En embeddings generales, como los de Word2Vec o GloVe, los antónimos suelen quedar **cerca**: *good* y *bad* aparecen en los mismos contextos (*the movie was* ____), y para la hipótesis distribucional eso es parecido. Además, t-SNE deforma las distancias grandes: dos grupos lejanos en el dibujo no dicen cuánto están separados en el espacio original.

Propiedades semánticas de los embeddings

Son los tipos de relación entre palabras que quedan capturados en la geometría de los vectores: qué quiere decir que dos vectores estén cerca, y qué quieren decir las direcciones entre ellos.

El tamaño de la ventana cambia la similitud

Hay distintos tipos de similitud o asociación, y cuál capturan los vectores depende sobre todo de la ventana de contexto:

- **Ventanas cortas** dan representaciones un poco más sintácticas: las palabras más parecidas son del mismo tipo y se pueden intercambiar en la oración.
- **Ventanas largas** hacen que las palabras de mayor coseno con la objetivo sean palabras relacionadas por tema, pero no parecidas.

Ejemplo 14.16 (*hogwarts*, de Levy y Goldberg (2014))**.** Vecinos más cercanos con skip-gram y dos tamaños de ventana:

Ventana ± 5	**Ventana** ± 2
dumbledore	*evernight*
hallows	*sunnydale*
half-blood	*garderobe*
malfoy	*blandings*
snape	*collinwood*

Con ± 5 sale el mundo de Harry Potter: personajes y palabras de los títulos de los libros. Con ± 2 salen otros lugares y escuelas de ficción, como *Sunnydale*, de *Buffy*, o *Evernight*, de una saga de vampiros. Son cosas **del mismo tipo** que Hogwarts, aunque no tengan nada que ver con Harry Potter.

Es lo que ya se había visto con las matrices término-término de la clase pasada: ventanas chicas juntan palabras intercambiables y ventanas grandes juntan palabras del mismo tema. Las dos asociaciones tienen nombre:

- **Coocurrencia de primer orden**, o asociación sintagmática: las palabras suelen aparecer una cerca de la otra. *escribió* con *libro* o *poema*.
- **Coocurrencia de segundo orden**, o asociación paradigmática: las palabras tienen vecinos parecidos, aunque casi nunca aparezcan juntas. *escribió* con *dijo* o *comentó*.

Analogías

La propiedad más famosa: algunas relaciones entre palabras quedan como direcciones constantes en el espacio. La diferencia entre *rey* y *hombre* se parece a la diferencia entre *reina* y *mujer*, así que

$$v_{rey} - v_{hombre} + v_{mujer} \approx v_{reina}.$$

Para resolver *"hombre es a rey como mujer es a ?"* se calcula el lado izquierdo y se busca la palabra de mayor coseno con el resultado. Funciona igual con $v_{París} - v_{Francia} + v_{Italia} \approx v_{Roma}$.

Ejemplo 14.17 (Con vectores de juguete)**.** Si la primera componente fuera qué tan masculina es la palabra y la segunda qué tanto tiene que ver con la realeza:

$$v_{rey} = (0.9,\ 0.8), \quad v_{hombre} = (0.9,\ 0.1), \quad v_{mujer} = (0.1,\ 0.1),$$

$$v_{rey} - v_{hombre} + v_{mujer} = (0.9 - 0.9 + 0.1,\ 0.8 - 0.1 + 0.1) = (0.1,\ 0.8),$$

que es justo donde estaría *reina*: poco masculina, muy de la realeza. Restar *hombre* quita lo masculino y sumar *mujer* pone lo femenino; la realeza no se toca.

Ojito...

En embeddings reales ninguna componente sola significa "masculino"; la dirección existe, pero repartida entre todas. Y las analogías funcionan menos de lo que sugieren los ejemplos famosos: sirven sobre todo con palabras frecuentes y con ciertas relaciones, y al buscar la respuesta hay que excluir las tres palabras de entrada, porque la más cercana al resultado suele ser *rey* misma. También heredan los sesgos del corpus: Bolukbasi et al. (2016) encontraron que *man* es a *computer programmer* como *woman* es a *homemaker*.

Embeddings contextualizados

Todo lo de hoy produce embeddings estáticos: un vector representa cada palabra. El problema son las palabras polisémicas, como *mouse*, ratón animal o ratón de computadora, o *bat*, murciélago o bate. El uso de una palabra depende de su contexto, pero su vector no.

Ejemplo 14.18 (*mouse*)**.** Word2Vec le da a *mouse* un solo vector, entrenado con todas sus apariciones. Termina en algún punto entre el sentido de roedor y el de computadora, cargado hacia el que sea más frecuente en el corpus, y no representa bien a ninguno.

Un embedding contextualizado calcula un vector distinto para cada aparición, a partir de la oración en la que está: *cheese-loving mouse* queda cerca del roedor y *click on the mouse* cerca de la computadora. Es lo que dibuja la diapositiva 57: una flecha por cada uso.

A esto también se le llama **representación dinámica**: la palabra ya no tiene un vector, tiene una función que le da uno según el contexto.

Idea clave

Es el problema de desambiguación de sentidos de la clase del 21 de septiembre, atacado desde la representación: en lugar de escoger un sentido de WordNet, el vector mismo cambia con el contexto. Los modelos que hacen esto, como ELMo y BERT, ya no son una tabla de vectores sino una red completa que hay que correr sobre la oración.

15. De las redes recurrentes al Transformer

7 de octubre de 2026

Resumen rápido. Segunda parte de los embeddings: los contextualizados, donde el vector de una palabra depende de la oración en la que aparece. Se recorrió la línea de modelos que llevó a ellos. Las redes recurrentes leen la oración palabra por palabra con una memoria; la LSTM le arregla a esa memoria lo corta; ELMo, el primer modelo de embeddings contextualizados, corre LSTM en las dos direcciones. El codificador-decodificador transforma una secuencia en otra, la atención le quita su cuello de botella, y el Transformer tira la recurrencia y se queda solo con la atención.

Estáticos contra contextualizados

Word2Vec, FastText y GloVe producen embeddings estáticos: cada palabra tiene un vector fijo. En el fondo son una tabla: se busca la palabra y sale su vector, sin importar en qué oración esté.

Un embedding contextualizado no es una tabla sino una función de la oración completa (diapositiva 58). Recibe las n palabras y devuelve un vector por cada una:

$$g : (w_1, w_2, \ldots, w_n) \longmapsto x_1, x_2, \ldots, x_n \in \mathbb{R}^d,$$

y el vector de la palabra k depende de todas las demás (diapositiva 59):

$$x_k = f(w_k \mid w_1, w_2, \ldots, w_n).$$

Ejemplo 15.1 (*perro*)**.** En *"ese carro se ve muy perro"*, *perro* quiere decir que se ve bien; en *"el perro ladra"*, es el animal. Un embedding estático le da a *perro* un solo vector, entrenado con todas sus apariciones, y como el animal es mucho más frecuente, ese vector queda casi todo del lado del animal. Un embedding contextualizado calcula un vector distinto en cada oración: en la primera, *se ve muy* lo acerca a adjetivos como *bonito* o *chido*; en la segunda, *ladra* lo deja junto a *gato* y *perrito*.

La diapositiva 59 trae el mismo caso en inglés: *play* en *"Elmo and Cookie Monster play a game"* es el verbo jugar, y en *"The Broadway play premiered yesterday"* es una obra de teatro.

Para construir f hace falta un modelo que lea la oración completa y cuya salida en cada posición dependa de las otras palabras. El primero que se usó para eso fueron las redes recurrentes.

Redes neuronales recurrentes

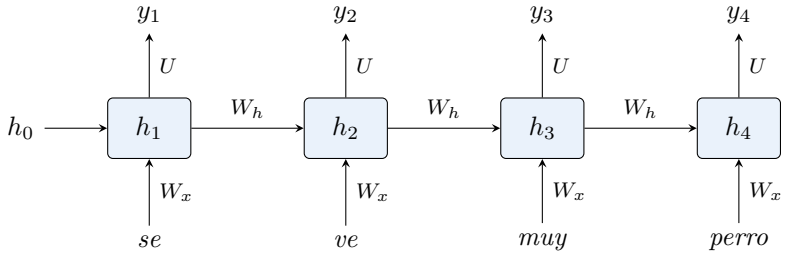
Definición 15.2 (Red neuronal recurrente)**.** Una RNN procesa una secuencia un elemento a la vez. En cada paso t su capa oculta recibe dos cosas: la entrada actual x_t y su propia salida del paso anterior, h_{t-1}. Con eso calcula el nuevo estado h_t, que pasa al paso siguiente.

El estado h_t es la **memoria interna** de la red: un vector de tamaño fijo que resume todo lo que ha leído hasta la palabra t, la experiencia previa con la que interpreta la palabra actual. Las ecuaciones de un paso son

$$h_t = \tanh(W_h\, h_{t-1} + W_x\, x_t + b)\,, \qquad y_t = \mathrm{softmax}(U h_t)\,,$$

con x_t el embedding de la palabra t y $h_0 = 0$. Si la red es un modelo de lenguaje, y_t es la distribución de la palabra siguiente, como en la red de la clase pasada.

Las matrices W_h, W_x y U son las mismas en todos los pasos. Por eso se dibuja desenrollada: no son cuatro redes, es una sola red aplicada cuatro veces, y las flechas horizontales son la memoria que pasa de un paso al siguiente.



Frente al modelo de lenguaje neuronal de la clase pasada, que solo veía una ventana fija de $N-1$ palabras, la RNN puede, en principio, condicionar en todas las anteriores: su historia no tiene un tamaño máximo.

Idea clave

Cada salida h_t ya es un embedding contextualizado de la palabra t: representa la palabra de entrada junto con las anteriores. Pero solo con las anteriores. En *"se ve muy perro"*, h_3 sabe que antes vino *se ve*, pero no sabe que después viene *perro*. Ese hueco es lo que va a cerrar ELMo leyendo también de derecha a izquierda.

Ejemplo 15.3 (Cuánto dura la memoria)**.** Una RNN de juguete con estado escalar, $W_h = 0.5$, $W_x = 1$ y $b = 0$. La primera palabra deja una marca, $x_1 = 1$, y las tres siguientes no aportan nada a esta componente, $x_2 = x_3 = x_4 = 0$:

$$\begin{aligned} h_1 &= \tanh(0.5 \cdot 0 + 1) = \tanh(1) = 0.762, \\ h_2 &= \tanh(0.5 \cdot 0.762) = \tanh(0.381) = 0.363, \\ h_3 &= \tanh(0.5 \cdot 0.363) = \tanh(0.182) = 0.180, \\ h_4 &= \tanh(0.5 \cdot 0.180) = \tanh(0.090) = 0.090. \end{aligned}$$

La marca de la primera palabra se reduce a la mitad en cada paso. Tres palabras después queda el 12 % de lo que había.

Arquitecturas

Según cuántas entradas y salidas se conecten, la misma idea da distintas arquitecturas (diapositiva 62):

Arquitectura	Qué hace	Ejemplo
Uno a uno	una entrada, una salida	red densa común
Uno a muchos	genera una secuencia	describir una imagen
Muchos a uno	resume una secuencia	sentimiento de una reseña
Muchos a muchos	una salida por entrada	etiquetado POS
Codificador-decodificador	secuencia a secuencia	traducción

En *muchos a uno* solo se usa el último estado, h_n, que resume la oración entera. En *muchos a muchos* hay una salida por palabra, que es lo que pide el etiquetado POS de la clase del 9 de septiembre. El codificador-decodificador tiene sección propia más abajo.

Desventajas

Memoria de corto plazo. En cada paso h_t se reescribe completo, mezclando la memoria con la palabra nueva y pasando todo por tanh. Lo reciente pesa mucho más que lo viejo, y la información del principio de la oración se va borrando, como en el ejemplo de arriba. Es la desproporción de la información pasada: no es que la RNN decida olvidar, es que no puede evitarlo.

Ejemplo 15.4 (Una dependencia larga)**.** En *"los perros que viven en la casa de la esquina ladran"*, para conjugar *ladran* en plural hay que recordar *perros*, que quedó siete palabras atrás, y no confundirse con *casa* ni con *esquina*, que están más cerca y son singulares.

Desvanecimiento del gradiente. La RNN se entrena con retropropagación a través del tiempo: el error de una predicción tardía tiene que viajar hacia atrás, paso por paso, hasta las palabras que la causaron. En cada paso el gradiente se multiplica por

$$\frac{\partial h_t}{\partial h_{t-1}} = W_h \left(1 - h_t^2\right).$$

En el ejemplo de juguete eso da 0.434, 0.484 y 0.496, así que

$$\frac{\partial h_4}{\partial h_1} = 0.434 \cdot 0.484 \cdot 0.496 = 0.104.$$

Cada factor vale a lo más 0.5, de modo que tras 20 palabras el gradiente queda multiplicado por a lo más $0.5^{20} \approx 10^{-6}$. La red recibe una señal casi nula para ajustar lo que pasó 20 palabras antes, y no aprende dependencias largas. Si los factores son mayores que 1 pasa lo contrario, el gradiente explota.

LSTM

Las redes de memoria de largo y corto plazo (*Long Short-Term Memory*, Hochreiter y Schmidhuber, 1997) son una RNN diseñada contra esos dos problemas: permiten aprender dependencias largas sin que el gradiente se desvanezca o explote.

Idea clave

La idea central es una **celda de memoria** que puede mantener su estado a lo largo del tiempo (diapositiva 63). Tiene dos partes: una memoria explícita, el vector de estado de la celda c_t, y unas **compuertas** que regulan qué información entra a la memoria y qué sale de ella.

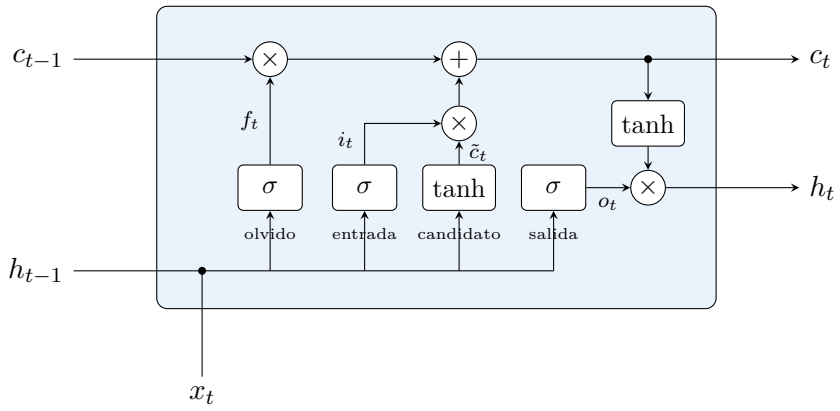
Una compuerta es un vector de números entre 0 y 1, salido de una sigmoide σ, que se multiplica componente a componente ($\odot$) contra otro vector: donde vale 0 bloquea y donde vale 1 deja pasar. Las

tres compuertas y el candidato se calculan a partir de la memoria de trabajo anterior y la palabra actual, concatenadas:

$$
\begin{aligned}
f_t &= \sigma(W_f\,[h_{t-1}; x_t] + b_f) && \text{compuerta de olvido}\\
i_t &= \sigma(W_i\,[h_{t-1}; x_t] + b_i) && \text{compuerta de entrada}\\
\tilde{c}_t &= \tanh(W_c\,[h_{t-1}; x_t] + b_c) && \text{candidato a guardar}\\
o_t &= \sigma(W_o\,[h_{t-1}; x_t] + b_o) && \text{compuerta de salida}
\end{aligned}
$$

y con ellas se actualizan la celda y la salida:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t).$$



Se lee así. La línea de arriba es la memoria de la celda: cruza el paso entero sufriendo solo una multiplicación, por f_t, que decide cuánto de lo viejo se conserva, y una suma, que agrega lo nuevo que i_t dejó pasar del candidato. No se reescribe completa ni se aplasta con tanh en cada paso, como el h_t de la RNN. Abajo, las compuertas se calculan con h_{t-1} y x_t. A la derecha, o_t decide qué parte de la memoria se muestra como salida h_t en este paso.

Por qué ya no se desvanece el gradiente. Por el camino de la memoria,

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t.$$

Si la red aprende a poner $f_t \approx 1$, la información y el gradiente cruzan muchos pasos casi intactos. En la RNN el factor era $W_h(1 - h_t^2)$, que no depende de lo que la red quiera recordar y siempre aplasta.

Ejemplo 15.5 (La misma marca, ahora en una celda)**.** Se repite el ejemplo de la RNN con una LSTM escalar. Para verlo claro, los valores de las compuertas se dan directamente en lugar de calcularlos con pesos. En el primer paso la entrada se abre del todo, $i_1 = 1$, y el candidato es $\tilde{c}_1 = \tanh(1) = 0.762$:

$$c_1 = f_1 \cdot 0 + 1 \cdot 0.762 = 0.762.$$

En los tres pasos siguientes la red decide conservar, con $f_t = 0.95$, y casi no deja entrar nada, con $i_t = 0.05$ y $\tilde{c}_t = 0$:

$$
\begin{aligned}
c_2 &= 0.95 \cdot 0.762 = 0.724,\\
c_3 &= 0.95 \cdot 0.724 = 0.687,\\
c_4 &= 0.95 \cdot 0.687 = 0.653.
\end{aligned}
$$

Tres palabras después sigue el 86 % de la marca, contra el 12 % de la RNN, y el gradiente por la memoria es $0.95^3 = 0.857$ contra 0.104.

Las compuertas no son fijas: la red las calcula en cada paso a partir de la palabra actual, así que aprende cuándo guardar y cuándo soltar. En *"los perros que viven en la casa de la esquina ladran"* puede abrir la entrada en *perros* para guardar el plural, mantener $f_t \approx 1$ durante la subordinada, usar el plural en *ladran* con o_t, y luego olvidarlo.

Ojito...

La LSTM alarga la memoria, pero no la vuelve infinita, y sigue siendo secuencial: para calcular h_{100} hay que tener h_{99}, así que las palabras de una oración no se pueden procesar en paralelo. Esa es la limitación que va a eliminar el Transformer.

ELMo

ELMo (*Embeddings from Language Models*, Peters et al., 2018) es el primer modelo que dio embeddings contextualizados de uso general. Sus vectores son los estados internos de un modelo de lenguaje bidireccional profundo (biLM), hecho con LSTM.

El modelo de lenguaje bidireccional

Un biLM son dos modelos de lenguaje (diapositiva 65). El que va hacia adelante lee de izquierda a derecha y predice cada palabra a partir de las anteriores; el que va hacia atrás lee de derecha a izquierda y predice cada palabra a partir de las siguientes:

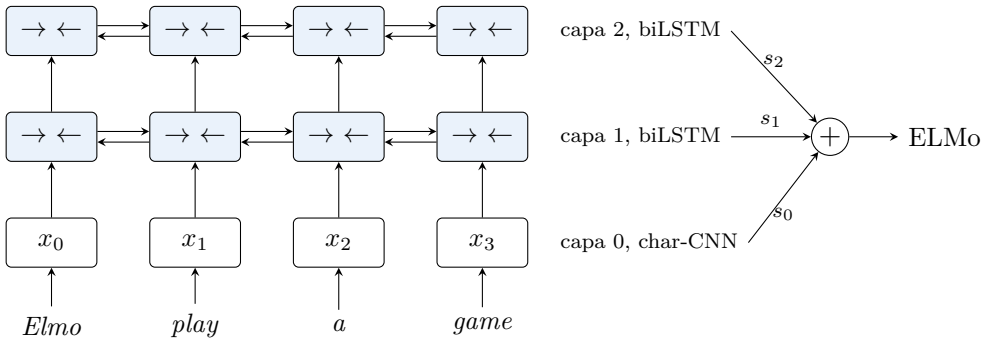
$$p(t_1, \ldots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_1, \ldots, t_{k-1}), \qquad p(t_1, \ldots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_{k+1}, \ldots, t_N).$$

Se entrenan juntos, maximizando la suma de las dos log-verosimilitudes, con autosupervisión como el modelo de lenguaje de la clase pasada. Son dos LSTM separadas y cada una ve un solo lado; lo bidireccional sale de juntar sus estados en cada palabra. Así el vector de una palabra ve hacia adelante y hacia atrás.

La arquitectura

Tiene tres capas (diapositiva 66):

- **Capa 0**: un embedding de la palabra sin contexto, construido a partir de sus caracteres con una red convolucional (char-CNN). Igual que FastText, así no hay palabras fuera del vocabulario.
- **Capas 1 y 2**: dos capas de LSTM bidireccional. En cada una se concatenan el estado hacia adelante y el estado hacia atrás de la palabra: $h_{k,j} = [\overrightarrow{h}_{k,j}; \overleftarrow{h}_{k,j}]$ para la palabra k en la capa j.



La representación ELMo de la palabra k combina sus tres capas con pesos que aprende la tarea que la va a usar:

$$\text{ELMo}_k = \gamma \sum_{j=0}^{2} s_j \, h_{k,j},$$

donde los s_j (los λ_j de la diapositiva) salen de un softmax y suman 1, y γ ajusta la escala.

Cómo se usa. Primero se entrena el biLM sobre un corpus grande y se congela. Para una tarea, digamos análisis de sentimiento, cada oración se pasa por el biLM, se calcula ELMo_k de cada palabra y se le da al modelo de la tarea junto con su embedding de siempre. El modelo de la tarea solo aprende la combinación lineal: los s_j y γ.

Por qué combinar capas. Cada capa captura algo distinto. Peters et al. encontraron que la primera capa de LSTM funciona mejor para etiquetado POS, que es sintaxis, y la segunda para desambiguación de sentidos, que es semántica. Con los pesos, cada tarea toma la mezcla que le sirve.

Ejemplo 15.6 (*play*, en las dos oraciones)**.** En la capa 0 el vector de *play* es idéntico en *"Elmo and Cookie Monster play a game"* y en *"The Broadway play premiered yesterday"*: todavía no hay contexto.

En la capa 1 ya no. La LSTM hacia adelante llega a *play* después de leer *Elmo and Cookie Monster*, un sujeto que pide un verbo, en una oración, y después de leer *The Broadway*, un artículo y un modificador que piden un sustantivo, en la otra. La LSTM hacia atrás llega desde *a game* en una y desde *premiered yesterday* en la otra. Las dos direcciones difieren, así que $h_{play,1}$ y $h_{play,2}$ son distintos en cada oración. En el artículo, los vecinos de *play* en GloVe son puras palabras de deporte, como *playing*, *game* y *players*; con el biLM, el *play* de una oración de teatro queda junto a otro *play* de teatro.

Ejemplo 15.7 (Dos tareas, dos mezclas)**.** Con vectores de juguete de dos componentes para *play*, $h_0 = (0.2,\ 0.9)$, $h_1 = (0.6,\ 0.1)$ y $h_2 = (0.8,\ 0.5)$, y $\gamma = 1$. Una tarea sintáctica que aprendió $s = (0.2,\ 0.6,\ 0.2)$ usa

$$0.2\,(0.2,\ 0.9) + 0.6\,(0.6,\ 0.1) + 0.2\,(0.8,\ 0.5) = (0.56,\ 0.34),$$

y una semántica que aprendió $s = (0.1,\ 0.2,\ 0.7)$ usa

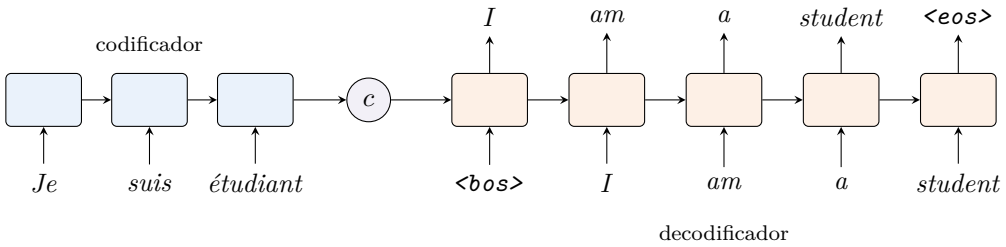
$$0.1\,(0.2,\ 0.9) + 0.2\,(0.6,\ 0.1) + 0.7\,(0.8,\ 0.5) = (0.70,\ 0.46).$$

El mismo biLM congelado le da a cada tarea un vector distinto para la misma palabra en la misma oración.

Codificador-decodificador

Un modelo secuencia a secuencia (*sequence-to-sequence*) toma una secuencia como entrada y produce otra secuencia como salida, posiblemente de otro largo. El caso típico es la traducción automática (diapositiva 67): *"Je suis étudiant"* entra y *"I am a student"* sale.

- **El codificador** es una RNN o LSTM que lee la entrada completa. Su último estado oculto es el **vector de contexto** c: todo lo que entendió de la oración, agrupado en un solo vector de tamaño fijo.
- **El decodificador** es otra RNN que arranca con c como estado inicial y genera la salida una palabra a la vez. En cada paso recibe la palabra que acaba de generar, da una distribución sobre el vocabulario, escoge una palabra y sigue, hasta producir el símbolo de fin `<eos>`.



Ejemplo 15.8 (Traducir *"Je suis étudiant"*, paso a paso)**.** Lo que hace el modelo, en orden:

1. El codificador lee *Je*, *suis* y *étudiant*. Su último estado es c.
2. El decodificador arranca en c y recibe `<bos>`, el símbolo de inicio. La palabra más probable es *I*.
3. Recibe *I* y genera *am*.
4. Recibe *am* y genera *a*.
5. Recibe *a* y genera *student*.
6. Recibe *student* y genera `<eos>`. Se detiene.

Entraron tres palabras y salieron cuatro: por eso no sirve la arquitectura de una salida por entrada. Al entrenar, en cada paso se le da al decodificador la palabra correcta anterior, no la que él generó, para que un error al principio no arruine el resto de la oración.

La diferencia con ELMo es el producto. ELMo da una representación, un vector por palabra de la entrada, y ahí termina. El codificador-decodificador produce una secuencia nueva; la representación, el vector c, es solo el paso intermedio.

Ojito...

El vector c es un cuello de botella. La oración entera, sea de tres palabras o de sesenta, tiene que caber en el mismo vector de tamaño fijo, y el decodificador no ve nada más. La calidad de la traducción cae con el largo de la oración, y lo primero en perderse son las palabras del principio, por la memoria de la LSTM. Además todas las palabras de la salida usan el mismo c, aunque cada una corresponda a palabras distintas de la entrada.

Atención

La atención (Bahdanau, Cho y Bengio, 2014) ataca los dos problemas del vector de contexto (diapositiva 68): que no cabe una oración larga en él, y que la parte de la entrada que importa cambia en cada paso de la salida. La solución es pasarle al decodificador **todos** los estados ocultos del codificador, y que en cada paso decida a cuáles prestarles atención.

En el paso t el decodificador tiene su estado h_t, y el codificador dejó un estado s_k por cada palabra de la entrada (diapositiva 69). Entonces:

1. **Puntaje.** Para cada palabra de la entrada se calcula $\text{score}(h_t, s_k)$: qué tan relevante es la palabra k para el paso t.

2. **Pesos.** Un softmax vuelve los puntajes una distribución sobre las palabras de la entrada:

$$a_k^{(t)} = \frac{\exp \text{score}(h_t, s_k)}{\sum_j \exp \text{score}(h_t, s_j)}.$$

3. **Contexto.** El vector de contexto del paso t es la suma de los estados del codificador ponderada por esos pesos:

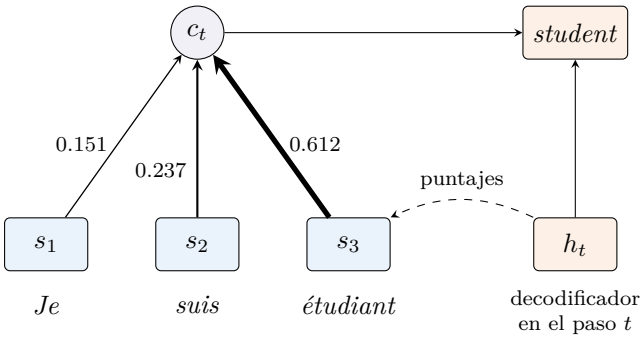
$$c_t = \sum_k a_k^{(t)}\, s_k.$$

4. **Predicción.** El decodificador usa c_t junto con h_t para predecir la palabra siguiente.

Ya no hay un solo c: hay un c_t nuevo en cada paso, armado con las palabras de la entrada que ese paso necesita. El puntaje se puede calcular de varias formas (diapositiva 70):

$$\underbrace{h_t^\top s_k}_{\text{producto punto}}, \qquad \underbrace{h_t^\top W s_k}_{\text{bilineal}}, \qquad \underbrace{w_2^\top \tanh(W_1[h_t; s_k])}_{\text{perceptrón multicapa}}.$$

El producto punto no tiene parámetros; la bilineal aprende una matriz W que relaciona los dos espacios, y el perceptrón aprende una red completa. Bahdanau et al. usaron el perceptrón.



Ejemplo 15.9 (Atención al generar *student*)**.** Con vectores de juguete de dos componentes, los estados del codificador son

$$s_{Je} = (1.0,\ 0.2), \quad s_{suis} = (0.3,\ 1.0), \quad s_{étudiant} = (1.2,\ 1.5),$$

y el decodificador, a punto de generar la cuarta palabra, está en $h_t = (0.5,\ 1.0)$. Con producto punto:

$$\begin{aligned} h_t^\top s_{Je} &= 0.5 \cdot 1.0 + 1.0 \cdot 0.2 = 0.70, \\ h_t^\top s_{suis} &= 0.5 \cdot 0.3 + 1.0 \cdot 1.0 = 1.15, \\ h_t^\top s_{étudiant} &= 0.5 \cdot 1.2 + 1.0 \cdot 1.5 = 2.10. \end{aligned}$$

El softmax:

$$e^{0.70} = 2.014, \quad e^{1.15} = 3.158, \quad e^{2.10} = 8.166, \quad \text{suma} = 13.338,$$

$$a = \left(\frac{2.014}{13.338}, \frac{3.158}{13.338}, \frac{8.166}{13.338}\right) = (0.151,\ 0.237,\ 0.612).$$

El contexto de este paso:

$$\begin{aligned} c_t &= 0.151\,(1.0,\ 0.2) + 0.237\,(0.3,\ 1.0) + 0.612\,(1.2,\ 1.5) \\ &= (0.151,\ 0.030) + (0.071,\ 0.237) + (0.734,\ 0.918) = (0.956,\ 1.185). \end{aligned}$$

El 61 % de la atención cae en *étudiant*, justo la palabra que se está traduciendo. Al generar *I*, los pesos habrían cargado hacia *Je*.

Idea clave

Los pesos de atención son una **alineación suave** entre las dos oraciones: para cada palabra de la salida dicen qué palabras de la entrada se miraron. En el artículo de Bahdanau la tabla de pesos sale casi diagonal, con las inversiones donde los idiomas cambian el orden: *zone économique européenne* queda alineada con *European Economic Area* al revés. Nadie le enseñó la alineación; la aprendió porque le servía para traducir.

Transformer

El Transformer (Vaswani et al., 2017, *"Attention is all you need"*) es un modelo basado únicamente en mecanismos de atención (diapositiva 71). Tira la recurrencia por dos razones:

- **Es secuencial.** En una RNN, h_t necesita a h_{t-1}: una oración de 50 palabras son 50 pasos uno tras otro, sin forma de repartirlos en paralelo.
- **Las dependencias largas cruzan muchos pasos.** Para que la palabra 1 influya en la 50, la información tiene que atravesar 49 estados. Con atención, cualquier par de palabras se conecta en un solo paso.

En el codificador, las palabras se miran entre sí con autoatención, intercambian información y tratan de entenderse en el contexto de la oración completa. En el decodificador también se miran entre sí, y además miran los estados del codificador.

Autoatención

La diferencia con la atención de la sección anterior está en quién mira a quién (diapositiva 72). En la atención, el decodificador mira al codificador: dos secuencias distintas. En la **autoatención** (*self-attention*) las representaciones son de la misma naturaleza: cada palabra de una oración mira a todas las palabras de esa misma oración, ella incluida, junta contexto y actualiza su propia representación. Lo hacen todas a la vez, en paralelo.

Consulta, clave y valor

Cada vector de entrada x_i recibe tres papeles, cada uno con su propia matriz aprendida (diapositiva 73):

$q_i = W_Q\, x_i$	consulta: *"¿quién tiene lo que busco?"*
$k_i = W_K\, x_i$	clave: *"yo tengo esto, dame peso si te sirve"*
$v_i = W_V\, x_i$	valor: *"esta es la información que entrego"*

Para actualizar la palabra i se compara su consulta con la clave de cada palabra, el softmax de esos puntajes da los pesos, y la salida es la suma ponderada de los valores. Con todas las palabras a la vez,

en forma de matriz:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

con d_k la dimensión de las claves.

Por qué tres papeles y no usar x_i directo. Lo que una palabra busca no es lo mismo que lo que ofrece. *Ladra* busca a su sujeto, un sustantivo que pueda ladrar; lo que les ofrece a las demás es ser un verbo. Con matrices separadas, consulta y clave pueden ser distintas.

Por qué dividir entre $\sqrt{d_k}$. El producto punto suma d_k términos, así que crece con la dimensión. Con puntajes grandes el softmax se satura: casi todo el peso cae en una palabra y el gradiente de las demás se vuelve casi cero. Dividir entre $\sqrt{d_k}$ mantiene los puntajes en la misma escala sin importar la dimensión.

Ejemplo 15.10 (Autoatención en *"el perro ladra"*)**.** Con $d_k = 2$, la nueva representación de *ladra*. Su consulta es $q_{ladra} = (1,\ 2)$ y las claves y valores de las tres palabras son

	Clave k	**Valor** v
el	$(0.5,\ 0)$	$(0.2,\ 0.1)$
perro	$(0,\ 1.5)$	$(0.9,\ 0.3)$
ladra	$(1,\ 0.5)$	$(0.1,\ 0.8)$

Puntajes, $q \cdot k$, y escalados entre $\sqrt{2} = 1.414$:

$$(0.5,\ 3.0,\ 2.0) \longrightarrow (0.354,\ 2.121,\ 1.414).$$

El softmax:

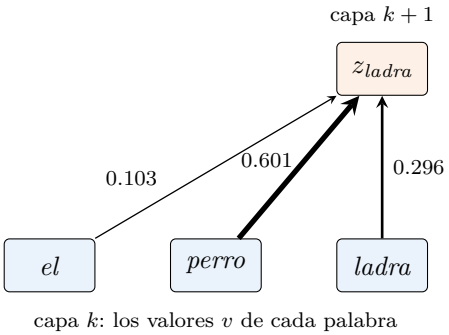
$$e^{0.354} = 1.424, \quad e^{2.121} = 8.342, \quad e^{1.414} = 4.113, \quad \text{suma} = 13.879,$$

$$a = (0.103,\ 0.601,\ 0.296).$$

La salida es la suma ponderada de los valores:

$$\begin{aligned} z_{ladra} &= 0.103\,(0.2,\ 0.1) + 0.601\,(0.9,\ 0.3) + 0.296\,(0.1,\ 0.8) \\ &= (0.021,\ 0.010) + (0.541,\ 0.180) + (0.030,\ 0.237) = (0.592,\ 0.427). \end{aligned}$$

El nuevo vector de *ladra* es 60 % *perro*: la representación del verbo ya incluye quién ladra. Sin dividir entre $\sqrt{2}$, el softmax de $(0.5,\ 3.0,\ 2.0)$ habría dado $(0.057,\ 0.690,\ 0.254)$, más concentrado; con las 64 dimensiones del Transformer original la diferencia sería mucho mayor.



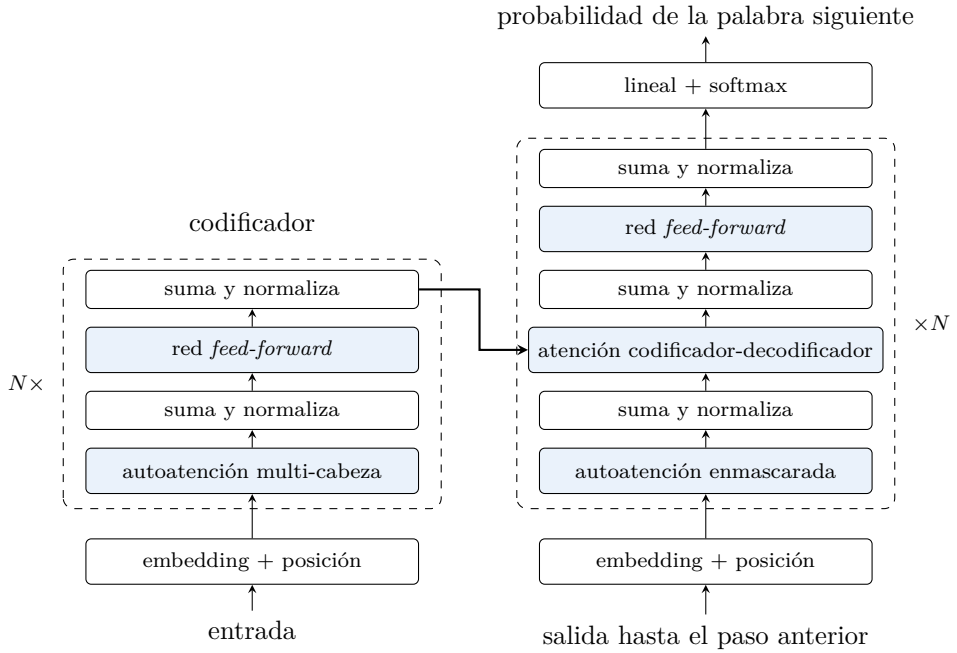
Atención multi-cabeza

Un solo softmax da una sola distribución: *ladra* solo puede repartir su atención de una manera. La atención multi-cabeza (diapositiva 74) corre varias autoatenciones en paralelo, cada una con sus propias W_Q, W_K y W_V. Las cabezas trabajan de forma independiente y al final sus salidas se concatenan y se proyectan con una matriz W_O. Cada cabeza puede seguir una relación distinta: una la de verbo con sujeto, otra la de pronombre con el sustantivo al que se refiere, otra la de cada palabra con sus vecinas. El Transformer original usa 8 cabezas de $d_k = 64$.

Codificación posicional

La autoatención no ve el orden: es una suma ponderada sobre un conjunto de palabras. *"El perro mordió al hombre"* y *"el hombre mordió al perro"* le darían a cada palabra el mismo vector. Por eso la entrada de cada palabra es la suma de dos embeddings (diapositiva 76): el de la palabra y el de su posición, *"la palabra x en la posición k"*. El artículo original usa senos y cosenos de distintas frecuencias; otros modelos aprenden el embedding de posición como cualquier otro.

La arquitectura completa



Es el diagrama de la diapositiva 75, simplificado.

- **Codificador.** N capas iguales (6 en el original). Cada una tiene dos subcapas: autoatención multi-cabeza, donde las palabras se miran entre sí, y una red *feed-forward* que se aplica a cada palabra por separado: después de juntar información de las otras, cada palabra se toma un momento para procesarla.

- **Suma y normaliza.** Cada subcapa va envuelta en una conexión residual y una normalización de capa, $\text{Norm}(x + \text{Subcapa}(x))$. La conexión residual le deja a la información y al gradiente un camino directo que se salta la subcapa, la misma idea que la línea de memoria de la LSTM, y es lo que permite apilar muchas capas.

- **Decodificador.** También N capas, con tres subcapas. La **autoatención enmascarada** deja

que cada palabra mire solo a las anteriores: al generar la palabra t, las siguientes todavía no existen. La **atención codificador-decodificador** toma las consultas del decodificador y las claves y valores del codificador; es la atención de Bahdanau. Al final, una capa lineal y un softmax dan la probabilidad de la palabra siguiente.

Ojito...

La autoatención compara cada palabra con todas las demás: con n palabras son n^2 puntajes por cabeza y por capa. Una RNN hace n pasos. Para oraciones eso no importa y el paralelismo gana de sobra, pero para documentos largos el costo cuadrático es el límite principal del Transformer.

Idea clave

La salida del codificador ya es un embedding contextualizado: un vector por palabra que depende de la oración entera, a través de varias capas de autoatención. Es la función g del principio de la clase, ahora sin recurrencia. Los dos modelos que siguen se quedan cada uno con una mitad del Transformer: GPT usa solo el decodificador, como modelo de lenguaje de izquierda a derecha, y BERT usa solo el codificador, que ve la oración en las dos direcciones (diapositivas 77 a 82).

16. El Transformer a detalle

12 de octubre de 2026

Resumen rápido. Segunda vuelta al Transformer, que la clase del 7 dejó presentado. Usa el esquema codificador-decodificador, pero sus piezas son de autoatención, y la autoatención del codificador no necesita saber nada del decodificador. Los tres conceptos nuevos son query, key y value, y lo que lo hace tan poderoso son las múltiples cabezas de atención: en cada capa hay varias representaciones distintas de cada palabra. Aquí se arma la autoatención completa con matrices, se ve la máscara del decodificador y cómo se genera una traducción paso a paso, se trabajan dos cabezas con números, y se cierra con lo que hay dentro de cada capa: suma y normalización, la red *feed-forward*, la codificación posicional y cuánto pesa el modelo.

Autoatención con matrices

La clase del 7 calculó la nueva representación de una sola palabra, *ladra*, con su consulta y las claves y valores ya dados. En el Transformer se calcula para todas las palabras a la vez, y las tres proyecciones salen de los mismos embeddings de entrada.

Se apilan los n vectores de entrada como renglones de una matriz X de $n \times d$. Cada papel es una multiplicación por una matriz aprendida:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

El renglón i de Q es la query de la palabra i, el de K su key y el de V su value. Después,

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \qquad Z = A\,V.$$

$QK^\top$ es de $n \times n$: su celda (i, j) es $q_i \cdot k_j$, qué tanto le sirve a la palabra i lo que ofrece la palabra j. El softmax se aplica renglón por renglón, así que cada renglón de A suma 1 y dice cómo reparte su atención una palabra. El renglón i de Z es la nueva representación de la palabra i.

Observación. Aquí cada palabra es un renglón y las matrices multiplican por la derecha. En la clase del 7 cada palabra era una columna, $q_i = W_Q\, x_i$. Es la misma operación con las matrices transpuestas; la forma por renglones es la del artículo original y la de casi todo el código.

Ejemplo 16.1 (Autoatención completa en *"el gato duerme"*)**.** Con embeddings de dos componentes y $d_k = 2$:

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{matrix} el \\ gato \\ duerme \end{matrix} \qquad W_Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad W_V = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}.$$

Proyecciones. Multiplicando cada renglón de X:

$$Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 2 & 1 \end{pmatrix}, \qquad K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad V = \begin{pmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 2 \end{pmatrix}.$$

Puntajes. Cada query contra cada key, y luego entre $\sqrt{2} = 1.414$:

$$QK^\top = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 3 \end{pmatrix} \longrightarrow \frac{QK^\top}{\sqrt{2}} = \begin{pmatrix} 0 & 0.707 & 0.707 \\ 0.707 & 0.707 & 1.414 \\ 0.707 & 1.414 & 2.121 \end{pmatrix}.$$

Por ejemplo, la celda (*duerme*, *gato*) es $(2,\ 1)\cdot(1,\ 0) = 2$.

Pesos. Softmax por renglón. Para *el*: $e^0 = 1$, $e^{0.707} = 2.028$, $e^{0.707} = 2.028$, que suman 5.056. Igual con los otros dos:

Mira ↓ **a →**	*el*	*gato*	*duerme*
el	0.198	0.401	0.401
gato	0.248	0.248	0.503
duerme	0.140	0.284	0.576

Salida. Cada renglón de $Z = AV$ es la suma de los values pesada por su renglón de A. Para *el*:

$$z_{el} = 0.198\,(1,\ 0) + 0.401\,(0,\ 2) + 0.401\,(1,\ 2) = (0.599,\ 1.604).$$

Las tres:

$$Z = \begin{pmatrix} 0.599 & 1.604 \\ 0.752 & 1.503 \\ 0.716 & 1.720 \end{pmatrix}.$$

Cada palabra salió con un vector nuevo hecho de pedazos de las otras. *el* entró como $(1,\ 0)$ y salió cargada de *gato* y *duerme*.

Ojito...

A no es simétrica. *el* le da 0.401 a *gato*, pero *gato* le da solo 0.248 a *el*. Si se usara el mismo vector como query y como key, $QK^\top$ sería simétrica y la atención tendría que ser mutua. Con W_Q y W_K separadas, que una palabra busque a otra no obliga a la otra a buscarla de regreso. Es lo que se necesita: un artículo busca a su sustantivo, pero al sustantivo le importa más su verbo que su artículo.

Por qué entre $\sqrt{d_k}$, con números

Si las componentes de q y k son independientes, con media 0 y varianza 1, el producto punto $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ es una suma de d_k términos de varianza 1, y su varianza es d_k. Con $d_k = 64$, la desviación estándar es 8: dos palabras pueden tener puntajes que difieren por 8 solo por azar.

$$\text{softmax}(8,\ 0) = (0.9997,\ 0.0003), \qquad \text{softmax}(1,\ 0) = (0.731,\ 0.269).$$

El primero es prácticamente one-hot: toda la atención va a una palabra y el gradiente hacia las demás es casi cero, así que no aprenden. Dividir entre $\sqrt{64} = 8$ regresa los puntajes a desviación 1, que es el segundo caso.

Las tres atenciones del Transformer

El Transformer usa la misma operación en tres lugares, y lo único que cambia es de dónde salen las queries, las keys y los values (diapositiva 75):

	Queries de	**Keys y values de**	**Quién mira a quién**
Autoatención del codificador	codificador	codificador	cada palabra de la entrada a todas las de la entrada
Autoatención del decodificador	decodificador	decodificador	cada palabra de la salida a las anteriores de la salida
Codificador-decodificador	decodificador	codificador	cada palabra de la salida a toda la entrada

La primera fila es la que dice que en el codificador no hace falta ver qué pasa en el decodificador. La autoatención del codificador solo usa estados del codificador: las palabras de la entrada se entienden entre sí sin saber que existe una salida. El codificador corre una sola vez sobre la entrada completa, y el decodificador consulta su resultado en la tercera fila tantas veces como quiera.

Idea clave

Por eso el codificador se puede usar solo. Si lo que se quiere es representar una oración y no generar otra, basta con la pila del codificador: su salida es un embedding contextualizado de cada palabra. Es lo que hace BERT.

La máscara del decodificador

En la autoatención del decodificador, cada palabra solo puede mirar a las anteriores. La forma de imponerlo es sumarle a los puntajes una máscara con $-\infty$ arriba de la diagonal antes del softmax:

$$M = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix}, \qquad A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right).$$

Como $e^{-\infty} = 0$, las posiciones tapadas reciben peso cero y el softmax reparte todo entre las que quedan.

Ejemplo 16.2 (La misma oración, con máscara)**.** Con los puntajes escalados del ejemplo anterior:

	Puntajes visibles	*el*	*gato*	*duerme*
el	(0)	1	0	0
gato	$(0.707,\ 0.707)$	0.5	0.5	0
duerme	$(0.707,\ 1.414,\ 2.121)$	0.140	0.284	0.576

el solo se ve a sí misma, y su salida es su propio value, $(1,\ 0)$. *gato* reparte entre *el* y ella: $0.5\,(1,\ 0) + 0.5\,(0,\ 2) = (0.5,\ 1)$. *duerme* ya veía todo lo anterior y no cambia.

Generar contra entrenar

Al traducir *"Je suis étudiant"*, el decodificador genera una palabra por paso. Cada palabra que sale se agrega a su entrada para el paso siguiente:

Paso	**Entrada del decodificador**	**Palabra que sale**
1	`<bos>`	*I*
2	`<bos>` *I*	*am*
3	`<bos>` *I am*	*a*
4	`<bos>` *I am a*	*student*
5	`<bos>` *I am a student*	`<eos>`

`<bos>` marca el inicio y `<eos>` el final. En todos los pasos el codificador ya terminó: su salida es la misma y el decodificador la consulta con la atención codificador-decodificador.

Al entrenar no hace falta ir paso por paso, porque la traducción correcta ya se conoce. Se le da al decodificador toda la oración `<bos>` *I am a student* de una vez, recorrida una posición a la derecha respecto a lo que debe predecir, que es *I am a student* `<eos>`. Es el *"outputs (shifted right)"* del diagrama de la diapositiva 75. Las cinco predicciones se calculan en paralelo, y la máscara garantiza que la posición t no vea la palabra que tiene que adivinar.

Ojito...

Sin la máscara el entrenamiento sería trampa: al predecir *student* el decodificador podría mirar *student* en su propia entrada, y aprendería a copiar en lugar de traducir. Al generar no habría nada que copiar y el modelo fallaría.

Múltiples cabezas de atención

Lo que hace tan poderoso al Transformer es que no tiene una atención por capa sino varias. Una cabeza produce una sola distribución por palabra: *duerme* reparte su atención de una sola manera. Con h cabezas, cada una con sus propias matrices, la palabra mira a la oración de h maneras distintas a la vez:

$$\text{head}_i = \text{Attention}(XW_Q^{(i)},\ XW_K^{(i)},\ XW_V^{(i)}),$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O.$$

Las cabezas trabajan de forma independiente. Al final sus salidas se pegan lado a lado y W_O las mezcla y las regresa a la dimensión de entrada, para que la capa siguiente reciba vectores del mismo tamaño.

Ejemplo 16.3 (Dos cabezas sobre *"el gato duerme"*)**.** La cabeza 1 es la del ejemplo anterior. La cabeza 2 tiene

$$W_Q^{(2)} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad W_K^{(2)} = W_V^{(2)} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix},$$

así que $K^{(2)} = V^{(2)} = X$ y $Q^{(2)} = ((1,\ 0),\ (0,\ -1),\ (1,\ -1))$. Sus puntajes escalados y sus pesos:

$$\frac{Q^{(2)} K^{(2)\top}}{\sqrt{2}} = \begin{pmatrix} 0.707 & 0 & 0.707 \\ 0 & -0.707 & -0.707 \\ 0.707 & -0.707 & 0 \end{pmatrix}.$$

	Cabeza 1			**Cabeza 2**		
	el	*gato*	*duerme*	*el*	*gato*	*duerme*
el	0.198	0.401	0.401	0.401	0.198	0.401
gato	0.248	0.248	0.503	0.503	0.248	0.248
duerme	0.140	0.284	0.576	0.576	0.140	0.284

Mismas palabras y mismos embeddings, dos patrones distintos. En la cabeza 1, *duerme* se mira sobre todo a sí misma; en la cabeza 2 mira sobre todo a *el*. *gato* en una mira al verbo y en la otra al artículo.

Concatenar y mezclar. Para *duerme*, la cabeza 1 dio $(0.716,\ 1.720)$ y la cabeza 2

$$0.576\,(1,\ 0) + 0.140\,(0,\ 1) + 0.284\,(1,\ 1) = (0.860,\ 0.424).$$

Pegadas, $(0.716,\ 1.720,\ 0.860,\ 0.424)$, de cuatro componentes. Con

$$W_O = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \\ 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

la salida es $(0.788,\ 1.072)$, de vuelta a dos componentes. Esta W_O solo promedia las dos cabezas; la de un modelo real se aprende y mezcla las componentes como le convenga.

Las dimensiones del Transformer original

Con $d = 512$ y $h = 8$ cabezas, cada cabeza proyecta a $d_k = d/h = 64$ componentes. Para una oración de n palabras:

Pieza	**Tamaño**
Entrada X	$n \times 512$
$W_Q^{(i)}$, $W_K^{(i)}$, $W_V^{(i)}$ de cada cabeza	512×64
$Q^{(i)}$, $K^{(i)}$, $V^{(i)}$ de cada cabeza	$n \times 64$
Pesos $A^{(i)}$ de cada cabeza	$n \times n$
Salida de cada cabeza	$n \times 64$
Las 8 concatenadas	$n \times 512$
W_O	512×512
Salida de la capa	$n \times 512$

Ocho cabezas de 64 cuestan lo mismo que una de 512: el trabajo se reparte, no se multiplica. A cambio, en lugar de una forma de mirar hay ocho.

Distintas representaciones en cada capa

Con varias cabezas, cada capa produce varias representaciones de cada palabra, una por cabeza, cada una armada con un tipo de relación. Y las capas se apilan: la salida de la capa 1 es la entrada de la 2, así que en la segunda capa las palabras se miran ya contextualizadas. Pasa lo mismo que con ELMo: las capas bajas capturan relaciones más locales y sintácticas, y las altas, más semánticas.

Observación. Cuando se revisa qué aprendió cada cabeza, salen patrones reconocibles. Voita et al. (2019) encontraron, en un Transformer de traducción, cabezas que miran a la palabra anterior o a la siguiente, cabezas que siguen relaciones sintácticas como la de sujeto y verbo, y cabezas que se fijan en las palabras raras. También encontraron que muchas cabezas se pueden quitar sin perder casi nada de calidad: no todas aprenden algo útil.

Dentro de cada capa

Suma y normalización

Cada subcapa, sea atención o *feed-forward*, va envuelta en

$$\text{LayerNorm}\,\big(x + \text{Subcapa}(x)\big).$$

La suma es la conexión residual de la clase del 7. La normalización de capa toma el vector de **cada palabra** por separado y lo lleva a media 0 y desviación 1, y después le aplica una escala γ y un corrimiento β aprendidos:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta, \qquad \mu = \frac{1}{d}\sum_i x_i, \quad \sigma = \sqrt{\frac{1}{d}\sum_i (x_i - \mu)^2}.$$

Ejemplo 16.4. Con $x = (1,\ 2,\ 3,\ 6)$, $\mu = 3$ y las desviaciones son $(-2,\ -1,\ 0,\ 3)$. La varianza es $(4 + 1 + 0 + 9)/4 = 3.5$ y $\sigma = 1.871$:

$$\frac{x - \mu}{\sigma} = (-1.069,\ -0.535,\ 0,\ 1.604).$$

Sin importar qué tan grandes vengan los números de la subcapa anterior, la siguiente los recibe en la misma escala. Eso es lo que mantiene estable el entrenamiento cuando se apilan muchas capas.

La red feed-forward

Después de juntar información de las otras palabras, cada palabra pasa sola por una red de dos capas, la misma para todas las posiciones:

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\,W_2 + b_2,$$

con W_1 de 512×2048 y W_2 de 2048×512. La atención mezcla información **entre** palabras; la *feed-forward* procesa lo que cada palabra ya juntó, sin mirar a las demás.

Ejemplo 16.5 (Cuánto pesa una capa)**.** Con $d = 512$ y $d_{\text{ff}} = 2048$, contando sesgos:

$$\begin{aligned}\text{Atención } (W_Q, W_K, W_V, W_O) &: 4 \cdot 512 \cdot 512 + 4 \cdot 512 = 1\,050\,624,\\ \text{Feed-forward} &: 512 \cdot 2048 + 2048 + 2048 \cdot 512 + 512 = 2\,099\,712,\\ \text{Dos normalizaciones} &: 2 \cdot 2 \cdot 512 = 2\,048.\end{aligned}$$

Una capa del codificador suma 3 152 384 parámetros, y dos tercios están en la *feed-forward*, no en la atención. Una del decodificador tiene una atención y una normalización más, 4 204 032. Con 6 y 6 capas son 44.1 millones, más $37\,000 \times 512 = 18.9$ millones de la tabla de embeddings: unos 63 millones. El artículo reporta 65 millones para este modelo; la diferencia está en detalles de qué se cuenta.

Codificación posicional

Como se vio el 7, la autoatención no ve el orden, y la entrada de cada palabra es la suma de su embedding y el de su posición. El artículo original no aprende los de posición; los calcula con senos y cosenos:

$$PE(pos, 2i) = \sin\!\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE(pos, 2i+1) = \cos\!\left(\frac{pos}{10000^{2i/d}}\right).$$

Cada par de componentes es un seno y un coseno de la misma frecuencia, y la frecuencia baja conforme avanza la componente.

Ejemplo 16.6 (Las tres primeras posiciones con $d = 4$)**.** Para $i = 0$ el divisor es 1, y para $i = 1$ es $10000^{2/4} = 100$:

Posición	$\sin(pos)$	$\cos(pos)$	$\sin(pos/100)$	$\cos(pos/100)$
0	0	1	0	1
1	0.841	0.540	0.010	1.000
2	0.909	−0.416	0.020	1.000

Las dos primeras componentes cambian mucho de una posición a la siguiente; las dos últimas casi nada. Es como un reloj: el segundero distingue posiciones vecinas y el horario distingue posiciones lejanas.

Idea clave

Por qué senos y cosenos y no simplemente el número de posición:

- Se quedan entre -1 y 1. Sumar $pos = 500$ directo al embedding lo aplastaría.
- Cada posición tiene un vector distinto, porque las frecuencias distintas no se repiten juntas.
- Pasar de pos a $pos + k$ es una rotación de cada par seno-coseno, la misma para cualquier pos. Una capa lineal puede aprender "la palabra que está k lugares antes" sin importar dónde esté.
- Funcionan con oraciones más largas que las del entrenamiento, porque la fórmula da valores para cualquier posición.

Transformer contra RNN

La tabla del artículo original, con n palabras y vectores de dimensión d:

	Costo por capa	**Pasos en secuencia**	**Camino más largo**
Autoatención	$\mathcal{O}(n^2 d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Recurrente	$\mathcal{O}(nd^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$

- **Pasos en secuencia:** cuántas operaciones tienen que esperar a la anterior. La RNN necesita n; la autoatención calcula todas las posiciones a la vez.
- **Camino más largo:** cuántos pasos separan a dos palabras cualesquiera. En la RNN la primera y la última están a n pasos; en la autoatención, a uno.
- **Costo por capa:** la autoatención es más barata mientras $n < d$. Con $d = 512$ y oraciones de unas decenas de palabras, casi siempre. Con documentos de miles de palabras el n^2 domina, que es el límite que ya marcaba la clase del 7.